MANCHESTER 1824
The University of Manchester

COMP30040 Report

Sentiment analysis of app reviews for requirements engineering

Supervisor: Liping Zhao

Student Name: Zihao Chen
Student ID: 10879229

Submission Date: April 18, 2024

Contents

Contents 1

List of Figures 3

List of Tables 5

1 Introduction 8
1.1 Context and Motivation . . . 8
1.2 Project Aims and Objectives . . . 8
1.3 Report Structure . . . 9

2 Background and Related Work 10
2.1 Background of General Theories and Techniques . . . 10
2.1.1 App Review Analysis . . . 10
2.1.2 Sentiment Analysis . . . 11
2.1.3 Natural Language Processing Definition . . . 12
2.1.4 Machine Learning Definition . . . 12
2.1.5 General Deep Learning Architectures and Pretrained Language Models . . . 13
2.1.6 History of Learning Word Embedding . . . 16
2.1.7 Distillation Technique . . . 16
2.2 Traditional Machine Learning Models Used in Project . . . 17
2.2.1 K-Nearest Neighbors . . . 17
2.2.2 Naive Bayes . . . 18
2.2.3 Support Vector Machine . . . 18
2.3 Pretrained Language Models Used in Project . . . 19
2.3.1 RoBERTa . . . 19
2.3.2 DistilBERT . . . 19
2.3.3 BART . . . 19
2.3.4 Loss Function in Fine-Tuning Pretrained Language Models . . . 21
2.3.5 Optimizer in Fine-Tuning Pretrained Language Models . . . 21
2.3.6 Specific Techniques in Fine-Tuning Pretrained Language Models . . . 21
2.4 Related Work . . . 21
2.4.1 General Research about App Review Analysis . . . 21
2.4.2 Sentiment Analysis on App Reviews . . . 22
2.4.3 State-of-Art Techniques Used in Related Work . . . 22
2.5 Research Gaps and Intended Project Contributions . . . 23

3 Research Methodology 25
3.1 Data Collection . . . 25
3.2 Data Preprocessing . . . 25
3.3 Data Labelling . . . 25
3.3.1 SentiStrength . . . 26
3.3.2 Annotation Guideline and Cohen Kappa . . . 26
3.3.3 Rules of Annotation Guidelines . . . 26
3.4 Natural Language Processing Pipelines . . . 27
3.4.1 Tokenization . . . 28
3.4.2 Lemmatization . . . 28
3.4.3 Stemming . . . 28
3.4.4 Stopwords . . . 28
3.5 Model Selection . . . 28
3.6 Training Traditional Machine Learning Models . . . 29
3.6.1 Feature Selection . . . 29
3.6.2 Validation and Hyperparameters . . . 29
3.7 Fine-Tuning Pretrained Language Models . . . 30
3.7.1 Prediction Head . . . 30
3.7.2 Frozen Layers . . . 30
3.7.3 Validation and Hyperparameters . . . 30
3.8 Data Augmentation . . . 30

3.9 Evaluation Metrics . . . 31
3.10 Experiment Environment . . . 31

4 Results Analysis **32**
4.1 SentiStrength Performance . . . 32
4.2 Data Attributes . . . 32
4.2.1 Quantitative Analysis . . . 34
4.2.2 Qualitative Analysis . . . 34
4.3 Confusion Matrix and Kappa Results . . . 35
4.3.1 General Analysis . . . 36
4.4 Ablation Study for Language Model Data Augmentation . . . 37
4.4.1 Quantitative Analysis . . . 37
4.4.2 Qualitative Analysis . . . 38
4.5 Multi-Class Experiments Results . . . 38
4.5.1 Quantitative Analysis . . . 41
4.5.2 Qualitative Analysis . . . 41
4.6 Multi-Label Experiments Results . . . 42
4.6.1 Quantitative Analysis . . . 45
4.6.2 Qualitative Analysis . . . 45
4.7 Experiment Running Time with Accuracy . . . 46
4.7.1 Quantitative Analysis . . . 46
4.7.2 Qualitative Analysis . . . 46

5 Research Evaluation **47**
5.1 Influences of GPU Configurations . . . 47
5.2 Influences of Hyperparameters . . . 47
5.2.1 Traditional Machine Learning Models . . . 47
5.2.2 Pretrained Language Models . . . 47
5.3 Comparison with Related Work . . . 49
5.4 Validity Threats . . . 50
5.4.1 Construct Validity . . . 50
5.4.2 Internal Validity . . . 50
5.4.3 External Validity . . . 50
5.5 Summary of Key Research Findings . . . 51
5.6 Reflections . . . 51

6 Conclusion and Future Work **52**
6.1 Achievements and Contributions . . . 52
6.2 Suggestions for Future Work . . . 52

A Snowballing Flowers **53**

References **59**

Word Count: 14977

List of Figures

1.1 Recent Surge in App Downloads . . . 8
2.1 Sentiment Analysis Framework [90] . . . 11
2.2 General Structure of Supervised Learning [100] . . . 12
2.3 More Effective DP as Data Grew [100] . . . 12
2.4 LSTM Structure [99] . . . 13
2.5 Encoder-Decoder Structure with Attention [76] . . . 13
2.6 Transformer Structure [113] . . . 14
2.7 Multi-Head Attention in Transformer [113] . . . 14
2.8 BERT Input Embedding [19] . . . 15
2.9 Fine-tuning BERT for Classification [35] . . . 15
2.10 Fine-tuning BERT for Sentiment Classification [74] . . . 16
2.11 Distillation . . . 16
2.12 BART . . . 20
2.13 Different Denoising Approaches . . . 20
2.14 BART for Classification . . . 20
2.15 Dropout . . . 22
2.16 Retrain BERT for RE [2] . . . 23
3.1 SentiStrength API [62] . . . 26
3.2 Different NLP Pipelines Used in Project . . . 28
3.3 Popular NB and SVM [16] . . . 29
4.1 Sentiment Category Distribution After Manual Checking . . . 32
4.2 Sentiment Category Distribution Without Manual Checking . . . 32
4.3 Category Distribution After Synonyms Data Augmentation . . . 33
4.4 Positive Scores Distribution After Manual Checking . . . 33
4.5 Negative Scores Distribution After Manual Checking . . . 33
4.6 Positive Scores Distribution Without Manual Checking . . . 33
4.7 Negative Scores Distribution Without Manual Checking . . . 33
4.8 Sentiment Score Pair Distribution . . . 33
4.9 Smooth Positive Scores Distribution Without Manual Checking . . . 33
4.10 Smooth Negative Scores Distribution Without Manual Checking . . . 33
4.11 Review Length Distribution-a . . . 34
4.12 Review Length Distribution-b . . . 34
4.13 Confusion Matrix for Sentiment Category with First Annotator . . . 35
4.14 Confusion Matrix for Sentiment Category with Second Annotator . . . 35
4.15 Confusion Matrix for Sentiment Category with Third Annotator . . . 35
4.16 Confusion Matrix for Sentiment Category with Fourth Annotator . . . 35
4.17 Confusion Matrix for Sentiment Category with Fifth Annotator . . . 35
4.18 Confusion Matrix for Sentiment Scores with First Annotator . . . 36
4.19 Confusion Matrix for Sentiment Scores with Second Annotator . . . 36
4.20 Confusion Matrix for Sentiment Scores with Third Annotator . . . 36
4.21 Confusion Matrix for Sentiment Scores with Fourth Annotator . . . 36
4.22 Confusion Matrix for Sentiment Scores with Fifth Annotator . . . 36
4.23 Ablation Study Observing Dropout . . . 37
4.24 Dropout Legend . . . 37
4.25 Ablation Study Observing Weight Decay . . . 38
4.26 Weight Decay Legend . . . 38
4.27 Accuracy on Multi-Class Experiments Across Various Pipelines . . . 39
4.28 RoBERTa F1-Scores on Multi-Class Experiments Across Various Pipelines . . . 39
4.29 KNN F1-Scores on Multi-Class Experiments Across Various Pipelines . . . 39
4.30 DistilBERT F1-Scores on Multi-Class Experiments Across Various Pipelines . . . 39
4.31 NB F1-Scores on Multi-Class Experiments Across Various Pipelines . . . 39
4.32 BART F1-Scores on Multi-Class Experiments Across Various Pipelines . . . 39
4.33 SVM F1-Scores on Multi-Class Experiments Across Various Pipelines . . . 39
4.34 Accuracy on Multi-Label Experiments Across Various Pipelines . . . 42
4.35 RoBERTa F1-Scores on Multi-Label Experiments Across Various Pipelines for Positive Sentiment Scores . . . 42
4.36 DistilBERT F1-Scores on Multi-Label Experiments Across Various Pipelines for Positive Sentiment Scores . . . 42

4.37 BART F1-Scores on Multi-Label Experiments Across Various Pipelines for Positive Sentiment Scores . . . 42
4.38 RoBERTa F1-Scores on Multi-Label Experiments Across Various Pipelines for Negative Sentiment Scores . . . 43
4.39 DistilBERT F1-Scores on Multi-Label Experiments Across Various Pipelines for Negative Sentiment Scores . . . 43
4.40 BART F1-Scores on Multi-Label Experiments Across Various Pipelines for Negative Sentiment Scores . . . 43
4.41 Linguistic Knowledge Flow in Encoder Framework [88] . . . 46
4.42 Linguistic Knowledge Flow in Auto-Regressive Framework[88] . . . 46
5.1 Different Batch Performance for Absolute Step . . . 48
5.2 Different Batch Performance for Relative Step . . . 48
5.3 Batch Legend . . . 48
5.4 Different Learning Rate Performance . . . 48
5.5 Learning Rate Legend . . . 48
5.6 Different Dropout Performance . . . 49
5.7 Dropout Legend . . . 49
5.8 Different Weight Decay Performance . . . 49
5.9 Weight Decay Legend . . . 49
5.10 Different Max Length Performance . . . 49
5.11 Max Length Legend . . . 49
5.12 Different Frozen Layer Performance . . . 49
5.13 Freeze Legend . . . 49

List of Tables

1.1 Project Objectives with Success Criteria . . . 9
2.1 Start Set for Snowballing . . . 10
3.1 Rules for Selecting Representative Comments . . . 27
3.2 Rules for Scoring Comments . . . 27
4.1 Warning Rate of SentiStrength . . . 32
4.2 Length Statistical Information of Reviews . . . 34
4.3 Kappa Values for Sentiment Category . . . 35
4.4 Kappa Values for Sentiment Scores . . . 36
4.5 Multi-Class Experiments Accuracy Results . . . 40
4.6 Multi-Class Experiments Results (Precision, Recall, and F1-score) for Traditional ML Models . . . 40
4.7 Multi-Class Experiments Results (Precision, Recall, and F1-score) for PLMs . . . 40
4.8 Multi-Label Experiments Accuracy Results . . . 43
4.9 Multi-Label Experiments Results (Precision, Recall, and F1-score) for Positive Sentiment Scores on PLMs . . . 44
4.10 Multi-Label Experiments Results (Precision, Recall, and F1-score) for Negative Sentiment Scores on PLMs . . . 44
4.11 Running Time with Accuracy of PLMs and Traditional ML Models . . . 46
5.1 Running Time for Different Numbers of GPU . . . 47
5.2 Running Time for Different Kinds of GPU . . . 47
5.3 Hyperparameters of All PLMs . . . 48
5.4 Running Time for Different max_length . . . 49
5.5 F1-Scores Compared with State-of-Art Research . . . 50

Acknowledgement

Over the past three years, Professor Liping Zhao has provided me with unwavering guidance and support. No level of grammatical enhancement can fully convey the depth of my gratitude. I can only express how proud and fortunate I feel to be her student. She introduced me to the world of AI, and her promising insights will benefit me for a lifetime. Working alongside her has been a truly enjoyable experience, and I hope to have the opportunity to continue my studies with her in the future.

Besides, I would like to express my heartfelt gratitude to my grandparents, parents, and sister. Their love has always been a source of inspiration and encouragement for me.

I also owe thanks to Kai Dong, Zhimeng Liu, Haochong Luo, Zhengtao Luo, and Haojing Tong for their assistance in ensuring database quality.

Abstract

App reviews contained rich requirement information with potential sentiments. Developers could prioritize improving features disliked by users based on emotions conveyed. The field of Natural Language Processing for Requirements Engineering (NLP4RE) utilized Natural Language Processing (NLP) techniques to tackle such problems, but more effort was required.

Traditional Machine Learning (ML) models, including K-Nearest Neighbors (KNN), Naive Bayes (NB), and Support Vector Machine (SVM) were benchmarked against Pre-trained Language Models (PLMs) including RoBERTa, DistilBERT, and BART for multi-class sentiment classification (positive, negative, and neutral) of app reviews. All PLMs, with F1-scores for positive and negative reviews exceeding 0.86, surpassed traditional ML ones. And DistilBERT achieved 0.80 accuracy, with fewer parameters, performing similarly to RoBERTa and BART, which both achieved 0.81 accuracy.

Granular multi-label classification using PLMs was also conducted, where reviews would have both positive (+1, +2, +3, +4, +5) and negative (-1, -2, -3, -4, -5) scores. All PLMs could achieve over 0.92 F1-scores for "+1" label. But efficiency of encoder-decoder architectures utilized by BART for Sentiment Analysis (SA) might not be as satisfying as encoder-only ones (RoBERTa and DistilBERT). Besides, DistilBERT underperformed in complex multi-label assignments.

Data augmentation was implemented in both traditional ML models and PLMs. The former utilized "synonym" dictionaries to increase the dataset size. The latter introduced "blank nosing" as a form of regularization to prevent overfitting. Through ablation study, it was found "blank nosing" could replace dropout and weight decay during fine-tuning. Moreover, critical analyses were conducted to explain why introducing noises appeared to contradict with aims of noise reduction in preprocessing.

The impact of different NLP pipelines on results was evaluated. Traditional ML models might benefit from them such as stemming and lemmatization. In contrast, they probably caused negative effects on PLMs in multi-class tasks. Nonetheless, these pipelines like lemmatization were found to be promising to improve PLM results in complex multi-label tasks.

Through Snowballing, 121 articles were summarized, providing extensive references across all aspects of this project. Furthermore, for pre-training and fine-tuning models, a high-quality database equipped with sentiment scores and labels for Requirement Engineering (RE) was developed, complemented by two annotation guidelines: one selecting representative reviews and the other outlining scoring processes.

1 Introduction

1.1 Context and Motivation

Requirement Engineering (RE) could comprehensively coordinate objectives to be achieved, expected attributes, and potential limitations encountered during software development. Typically, Requirement represented multiple solutions about features and reliability of platforms, commonly described by natural language [35, 120, 11, 5]. Participants were traditionally invited to engage in qualitative requirement gathering approaches, such as one-on-one dialogues, seminars, and group sessions [57, 50, 51], analyzing their insights about the software[16] to ultimately benefit themselves [57]. However, with the emergence of computerized techniques [12, 29, 50, 52] refining data administration and strategic planning, in "Fourth Industrial Revolution" (Industry 4.0) [103], participants could share [77] their comments and insights of digital services via "Application Distribution Platform" or "App Store" [57, 29, 112, 75]. These postings had contributed to a more diverse set of requirements for software applications [4, 55, 63], attracting considerable attention from RE [77, 40, 25]. This uptrend coincided with a surge in app downloads in recent years [1], shown in Figure 1.1.

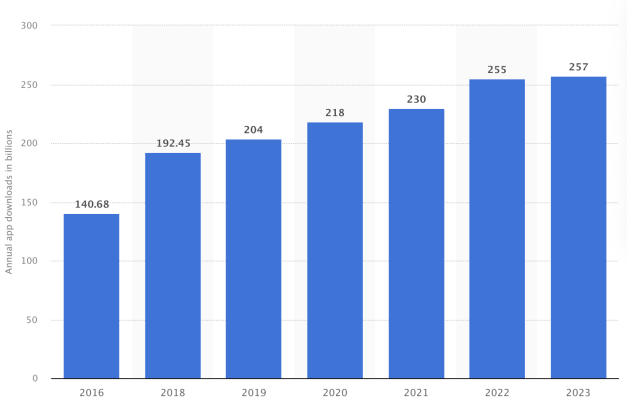


Figure 1.1: Recent Surge in App Downloads

Nevertheless, such "data-driven RE" introduced some difficulties, such as advanced assistance systems and significant manpower to intelligently handle all requirements [57, 50, 16, 55]. To address this issue, an increasing number of investigators started employing Natural Language Processing (NLP) techniques for analyzing requirements [73], and there had been a thriving research area called "Natural Language Processing for Requirements Engineering" (NLP4RE) for this purpose. **But advice on selecting proper NLP techniques still needed refinements [120].** Furthermore, Sentiment Analysis (SA) was a typical discipline of Computer Science [22], and it could be considered as a subtask of requirement classification within NLP4RE [120]. **Research on SA in app reviews also remained inadequate and required further development [102].**

1.2 Project Aims and Objectives

The aim of this project was leveraging traditional Machine Learning (ML) models and advanced pre-trained language models (PLMs) for SA on app reviews. Multi-class classifiers were achieved using both PLMs and traditional ML ones to categorize the review sentiment (positive, negative, or neutral). Moreover, PLMs were employed to facilitate multi-label classifiers, providing each review with a granular positive score from 1 to 5 and a negative one from -1 to -5. The project further investigated the impact of various NLP pipelines on model performance. Additionally, it involved fine-tuning PLMs on multiple GPUs. A high-quality dataset for SA at RE-level was also constructed. To enable critical analyses of this project, corresponding measurable objectives were presented in Table 1.1.

[1] https://www.statista.com/statistics/271644/worldwide-free-and-paid-mobile-app-store-downloads/

Table 1.1: Project Objectives with Success Criteria

Project Objectives	**Success Criteria**
Sentiment scores with related categories of app reviews generated by `SentiStrength` were manually inspected.	Annotators were invited to perform double manual inspections. Using Kappa criteria to compare their labels with authors', evaluating dataset quality.
Explore performance of traditional ML models including K-Nearest Neighbors (KNN), Naive Bayes (NB), and Support Vector Machine (SVM), and PLMs (RoBERTa, DistilBERT, and BART) on multi-class and multi-label classifications for SA.	By conducting multiple experiments and recording metrics such as accuracy and F1-score, combined with respective architectures, quantitative and qualitative analyses should be performed.
App reviews were processed through different NLP pipelines such as stemming and lemmatization.	By integrating their principles, analyses of their metrics like accuracy on multi-class and multi-label tasks should be executed, sharing suitable suggestions with NLP4RE.
Carrying out the PLM fine-tuning on multiple GPUs.	Observing whether increasing the number of GPU could reduce training time.
Data augmentation was implemented to enhance the model robustness.	Investigating the impact of that especially on PLMs, through ablation analyses.
Conducting comparisons of this research with other state-of-the-art studies.	Searching for related papers on SA of app reviews and critically comparing results like F1-score.

1.3 Report Structure

The remaining report is organized as follows:

- Chapter 2 provides a comprehensive literature review and uncovered some potential concerns.
- Chapter 3 outlines the research methodology of each experiment process.
- Chapter 4 deeply analyzes experiment results quantitatively and qualitatively.
- Chapter 5 evaluates results with different GPU, hyper-parameters, and the state-of-the-art. Validity threats are examined. And findings from this project are critically summarized.
- Chapter 6 concludes main contributions, future work assumptions, and pertinent suggestions.

2 Background and Related Work

This chapter presents a literature review for the background and related work about this project. Initially, a comprehensive overview of relevant fields such as SA and NLP is provided in 2.1. The knowledge of traditional ML models and PLMs is thoroughly explained in 2.2 and 2.3, respectively. Articles in similar domains are also summarized in 2.4 with critical discussions of limitations and intended project contributions in 2.5.

There were two methods to support that. One was Systematic Literature Review (SLR) discovering papers by typing keywords on digital libraries. However, it presented some issues [10, 115]:

1. Appropriate choices were required for online libraries such as ACM and IEEE.
2. Different searching options were complex like publication time.
3. Configurations of searching keywords were difficult such as "App Store reviews" and "application review".

The other one was "Snowballing", reading bibliographies of papers, and iteratively continuing it [10]. It was more effective than SLR to accumulate the expertise for specific themes. In this report, the majority of citations were identified through Snowballing, which had three steps [115]:

1. Finding appropriate initial papers, also named as "start set".
2. Backward Snowballing: in bibliographies of an article, searching for suitable papers based on rules such as topics and publication outlets.
3. Forward Snowballing: utilizing tools to search for **articles that had cited the current article**.

In this project, Forward Snowballing was discarded, and five main areas were prioritized when searching papers through Backward Snowballing: **NLP, ML or Deep Learning (DP), app review analysis, SA, and RE**. Besides, start set selections were essential [10] and they were from suggestions of the advising professor, shown in Table 2.1.

Table 2.1: Start Set for Snowballing

Article Name	Reference
A review of machine learning algorithms for identification and classification of non-functional requirements	[10]
A Machine Learning Approach for Hierarchical Classification of Software Requirements	[11]
Attention Is All You Need	[113]
Emergent Abilities of Large Language Models	[114]
Machine Learning: Algorithms, Real-World Applications and Research Directions	[100]
Recent Advances in Natural Language Processing via Large Pre-Trained Language Models: A Survey	[71]
Retraining a BERT Model for Transfer Learning in Requirements Engineering: A Preliminary Study	[2]
Zero-shot learning for requirements classification: An exploratory study	[5]
BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding	[19]
GPT-3: What's it good for?	[17]

In this paper, Backward Snowballing was conducted by two to three iterations. Specifically, articles about SA were primarily accumulated through two iterations of [100], while app review analysis was mainly gathered through two iterations of [5]. Selected ones often served both SA and app review analysis, proving efficiency of Snowballing, whose processes were also visualized in Appendix A. Each green block in images represented the name of the current paper, including some well-known models like T5 [88]. The text above explained how these citations supported the current paper.

2.1 Background of General Theories and Techniques

2.1.1 App Review Analysis

The majority of software applications were implemented by external firms, associations, and independent developers [23], making them likely to be immensely profitable [63]. Participants were also encouraged to share productive reviews and stars on App Store [32, 60] such as Google Play and Apple App Store [77, 96, 55, 81, 56, 24], and these reviews, also named "user-generated content" [52], probably contained error records and improvement suggestions [96, 75, 61, 32, 56]. Besides, there was a term called "User Story" to represent ones including customer engagements [55, 57].

It was known the workflow of software development was iterative [14, 30], therefore, merging and analyzing reviews into the product lifecycle [96, 75, 29] could enhance expected performance of comprehensive functionalities [77, 3, 14, 16]. Additionally, through text classification tasks such as SA and "Functional/Non-Functional Requirement Classification" [5, 10] on reviews, different individuals developing the software could gain own tasks [57, 40] such as framework scripts for programming specialists [55] and strategies for supervisors [12, 56]. Participants could also benefit from that. For one thing, App Store had narrowed the distance between users and software supervisors [77, 81, 4, 16]. For another thing, they could determine the need for this software [112, 56] by observing peer reviews [25, 13, 52].

2.1.2 Sentiment Analysis

Individuals could convey their views for many topics on the Internet [90, 95, 51], comprising textual information and emotional insights [18]; the latter was more likely to represent underlying attitudes [111]. Initially, SA was employed in analyses of business and governmental statements [62, 67, 22, 84], and software engineering (SE) also tried this technology given its capabilities were gradually uncovered [61]. SA was a form of digital research designed to analyze potential perspectives and feelings within given texts based on specific themes [73, 41, 47, 72, 50], also named "Opinion Mining" [51, 22, 67, 90, 100, 65]. It could be utilized by enterprises to effectively [80] investigate stakeholders' stances [65, 25, 57], thereby adjusting appropriate policies [13, 73, 90, 100, 112], especially those Seed-Stage corporations [22, 52]. For example, software teams probably paid more attention to requirements with negative emotions [16, 63, 112, 24, 30, 57]. [86, 95, 81, 84, 32] argued SA was just assessing sentiment orientations (positive, negative, or neutral) within the input. However, SA should be considered as "A Multi-Faceted Problem" [90, 51], shown in Figure 2.1.

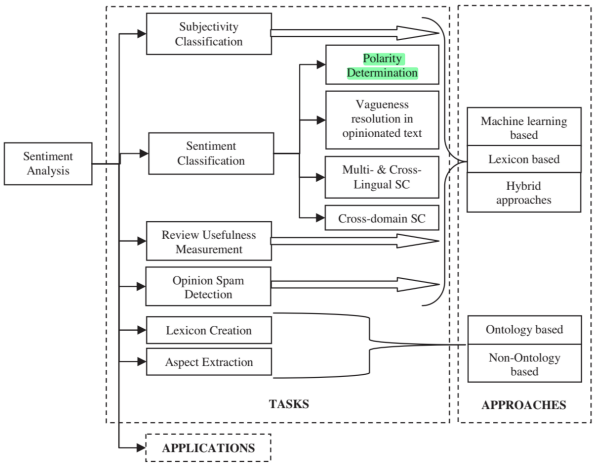


Figure 2.1: Sentiment Analysis Framework [90]

Two predominant directions could be "Sentiment Classification" and "Subjectivity Classification" [110]. The **former** recognized emotions of provided documents [47]. One of key areas of it should be "Polarity Determination" [64, 16] and it was precisely the concept mentioned in early cited papers [86, 95, 81, 84, 32]. The **latter** detected if documents expressed subjectivity or objectivity [51, 107]. Specifically, subjectivity referred to subjective "facts" for items, occurrences, and attributes. Objectivity referred to individual objective "opinions" for emotions or perceptions regarding those items [52]. Sometimes, Sentiment and Subjectivity Classification were employed in conjunction, first assessing whether the content was subjective or objective, and then only investigating emotions of subjective ones [22, 73, 111, 110]. However, it remained debates, for instance, the journalism probably be identified as positive or negative but usually lacking subjective statements [13].

In terms of SA strategies, mainstream studies divided them into two main routes. The first one was dictionary-based models: by employing specific methods to create a dictionary including emotion scores for every word and punctuation mark, when in used, displaying the emotion score of every "token" in content and conducting customized analyses [62]. The second one was ML methods [26, 95].

Besides, SA could be classified in three different scales: "document-level", "sentence-level", and "aspect-level". The first two derived the overall emotion of words and were similar, since a document consisted of several sentences. The third one could compensate for their shortcomings, allowing further analyses to check

each item emotion within the input [22, 67].

2.1.3 Natural Language Processing Definition

To address SA or other text classifications, NLP was indispensable, since content like app reviews was readable for people but not for machines [16]. The aim of NLP was utilizing digital methods to acquire, comprehend, and generate verbal texts [100, 120, 76]. Furthermore, NLP encompassed two main directions: "symbolic NLP" and "statistical NLP". The former was linguistic frameworks or human-engineered guidelines accumulating specific language understandings. However, it probably was not adaptable to manually develop principles when analyzing unconventional and syntactically incorrect files and too complex to maintain. The latter was the priority for most scholars, utilizing quantitative approaches and substantial textual datasets to construct broad linguistic models [49] without examining language patterns [120].

2.1.4 Machine Learning Definition

Nowadays, NLP4RE would prefer to utilize ML, especially DP to tackle challenges related to requirements [5, 11], converting RE into knowledge-based frameworks [10]. Requirement classifications including SA, were predominantly presented as supervised learning, shown in Figure 2.2, fitting a mapped relationship to match each given requirement to labels, grounded on requirement-label pairings [11, 100]. "Multi-Class" meant requirements had **"more than two incompatible [100] classes"** [11]. "Multi-Label" meant requirements would hold **more than two labels concurrently**, implemented by sophisticated algorithms. For example, a piece of news could belong to both technique and business [100].

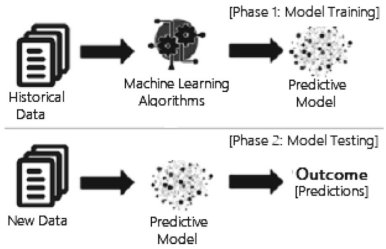


Figure 2.2: General Structure of Supervised Learning [100]

The success of ML relied on the data property and efficient algorithm itself [98]. DP could derive profound results from extensive data shown in Figure 2.3, trained by minimizing "partial derivative" of neutral networks through "gradient descent" known as "back-propagation" [92, 76] and introducing the non-linearity through activation functions [58], compared to conventional ML [100, 99, 15].



Figure 2.3: More Effective DP as Data Grew [100]

However, RE lacked structured samples [10, 120] especially app reviews, since manually annotating was costly, labor-intensive, and susceptible to mistakes [121], affecting deep learning employments [5]. DP might cause unsatisfying outcomes without enough data [101, 118, 120, 87]. The dilemma could be improved by "Transfer Learning" [99], leveraging knowledge from other similar [5, 121] tasks to enhance the current project with limited input-output pairs [5]. It had been applied to many domains such as NLP and Computer Vision [99], breaking away premises of supervised learning: forecasting systems operated on singular data to complete assignments and datasets exhibited consistent patterns during training and inference phases [79, 91].

2.1.5 General Deep Learning Architectures and Pretrained Language Models

Recurrent Neural Network (RNN) was a DP architecture, analyzing given sentences [72] and deriving a condition cell prior to processing subsequent data points. Representations learnt from RNN were weakened with training epochs. Long Short-Term Memory(LSTM) shown in Figure 2.4, enhanced RNN [76, 99, 59], tackling "gradient vanishing" and "gradient exploding" [72] by introducing gates to manage sequential knowledge during training [76]. As sequences were progressively learned, each LSTM node along the sequential direction, would hold own "memory cell". Generally [99]:

- "Forget Gate" considered how much knowledge should be discarded from earlier "memory cell".
- "Input Gate" coordinated how much knowledge should be appended to current "memory cell".
- "Output Gate" determined how much information should be passed to next "memory cell" or output layers.

Therefore, LSTM could learn longer representations and manage gradients during training [76].

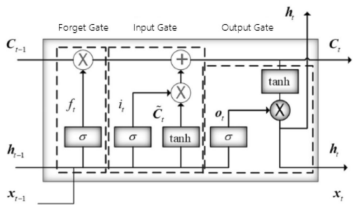


Figure 2.4: LSTM Structure [99]

However, RNN and LSTM cannot run in parallel. Specifically, outputs of each hidden layer had to wait previous results as a part of input [69, 72]. Besides, they failed to consider the relative importance of elements within the sequence [76]. [7] proposed attention mechanism with encoder-decoder structure, shown in Figure 2.5. It could learn patterns between elements without length trouble [7], by training models to catch detailed knowledge [24]. But RNN was still needed [76].

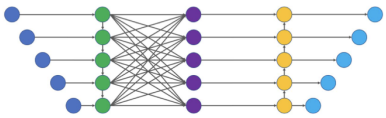


Figure 2.5: Encoder-Decoder Structure with Attention [76]

Transformer architecture significantly improved these issues. It solely depended on self-attention, listed in Equation 2.1, to identify global relationships among input and output [72]. Self-attention was the function matching the "query" (Q) and the pair of "key (K) - value (V)" to results. And each V was based on "dot-product" of Q and K, ensuring rapid underlying computations. $\frac{1}{\sqrt{d_k}}$ was a form of regularization to offset the extreme growth of "dot product", for smoother gradients during training [113].

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \qquad [113] \tag{2.1}$$

Transformer was shown in Figure 2.6. The left part was the Encoder block and the right one was the Decoder block. Its initial task was language translation. Training and test data, added positional information of tokens through sine and cosine functions, were passed to Encoder and Decoder respectively [113].

Token representations were learnt through self-attention and fully connected feed-forward networks [88] in Encoder, with "Residual Connection" [34] and "Layer Normalization" [6] served as intervening components facilitating information flow [113]. "Residual Connection" [34] was the mapping function where the output of a layer was derived from the input and output of the former layer [76]. "Layer Normalization" [6] could coordinate distributions of mean and variance along the batch direction of data, thereby rendering models less sensitive to the sequence length.

Decoder was similar to Encoder, but it included a component to manage self-attention between Encoder and Decoder. This was achieved by leveraging Encoder's K and V alongside Decoder's Q , thus facilitating

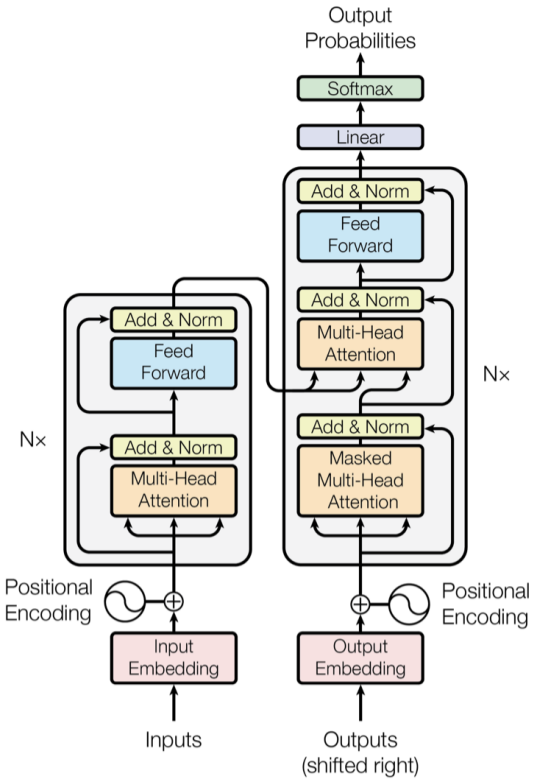


Figure 2.6: Transformer Structure [113]

relationships between training and test data. By stacking Encoder and Decoder [76], Transformer effectively learnt knowledge from sequences [113].

Besides, there was also a crucial component: Multi-Head Attention, shown in Figure 2.7. Its principle was mapping computed attention scores through multiple "linear projections" [113, 76].

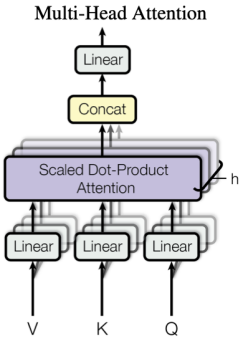


Figure 2.7: Multi-Head Attention in Transformer [113]

Successes [87, 19, 87, 2] of Transformer had several factors [113, 72, 76]:

- Computational complexity was optimized.
- Maximally reducing structures containing sequential operations, without recursive models like RNN, allowing to conduct large-scale pre-training universal language representations in numerous GPUs.
- Length concerns of data were addressed through self-attention: sequence information could be fully analyzed, and the length had minimal impact during back-propagation.

Bidirectional Encoder Representations from Transformers (BERT) was primarily composed of stacked encoder blocks of Transformer. It employed transfer learning to solve a series of NLP problems [19]. Specifically,

it could pre-train general language representations [35] on large amounts of unlabelled data available on the Internet [5, 87, 49], because of similar language essences, then they were fine-tuned in downstream assignments [88, 71, 2, 76]. One possible advantage was projects with limited training data could benefit from that [19, 35].

BERT also constructed the contextual word embedding at WordPiece level with a set having 30,000 fixed tokens [19] not the word level [94]. This could solve Out-Of-Vocabulary (OOV) [76] issues where words were not appeared in pre-training data but showed in fresh sequences. And the fixed size of WordPiece token set enhanced algorithm effectiveness. Besides, the first token of each content was fixed as "[CLS]" [19, 71, 35].

To get general language knowledge, BERT used two pre-training tasks: Masked Language Model (MLM) and Next Sentence Prediction (NSP) [94, 76]:

- MLM: hiding 15% WordPiece tokens randomly in given content using special tags "[MASK]". BERT was trained to recover those tokens [35, 88, 71]. To solve the issue that "[MASK]" did not present when fine-tuning, **those 15% masked tokens**, would still be "[MASK]" in 80% cases, keep original elements without masking in 10% cases, and replaced by random words in 10% cases [19]. This was also named as "BERT-style denoising" [88], which could learn language knowledge from two directions simultaneously, considering both forward and backward content [19, 71].
- NSP: To expand scopes of fine-tuning, "segment" [53] pairs (one segment probably had several sentences) were extracted from unlabelled pre-training data, with 50% pairs having sequential relationships and the other 50% not, to facilitate binary classification [35]. The "[SEP]" token was inserted between segments to separate them [19]. However, effectiveness of this task was still queried [53].

Besides, BERT injected positional information of tokens and sentences by extra layers [19] shown in Figure 2.8.

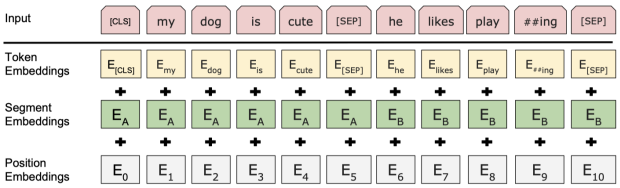


Figure 2.8: BERT Input Embedding [19]

For conducting NLP tasks such as text classifications, it was only required to append some output layers shown in Figure 2.9, also called "prediction heads" to an already trained BERT framework [71]. "[CLS]" of the last hidden layer was extracted, passed to "prediction heads" for predicting [19, 35] shown in Figure 2.10.

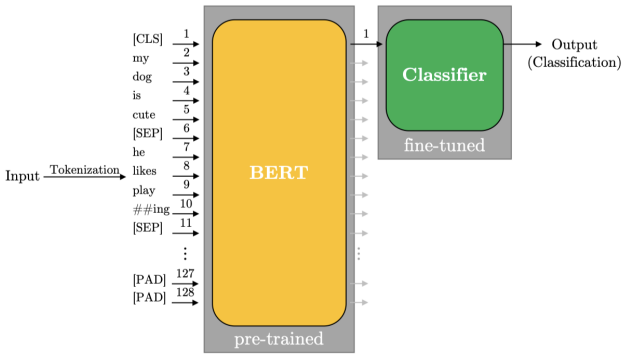


Figure 2.9: Fine-tuning BERT for Classification [35]

There were two BERT configurations. The base version had 12 encoder blocks, 768 fixed hidden size, 12 self-attention heads, and about 110M parameters; the large one had 24 encoder blocks, 1024 fixed hidden size, 16 self-attention heads, and about 340M parameters [19].

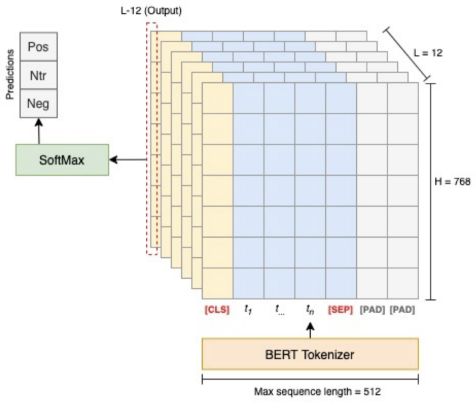


Figure 2.10: Fine-tuning BERT for Sentiment Classification [74]

2.1.6 History of Learning Word Embedding

Traditional language models such as "Skip-gram" [70] and "Word2Vec" [68] could depict the "static" [2] embedding of natural language. But they were more likely to learn compact vector information [20] with some issues. For one thing, "This is good" and "This is not good" potentially held homogeneous characteristics [5], which was not reasonable. For another thing, when a word had distinct meanings yet reflecting the identical vector, it restricted algorithm capabilities, as it failed to distinguish different word meanings [20, 35]. [83] proposed "dynamic" [5] word representations employing multiple levels of LSTMs in two directions [71]. This style could catch improved vocabulary representations and distinguish "polysemy". Besides, [83] also pioneered paradigms of adding classification heads on the top of PLMs for fine-tuning.

BERT also adopted analogical approaches [35]. It designed MLM to catch effective word vectors within stacked Encoders [71, 5] and address "polysemy" issues using Multi-Head Attention [35], attaining impressive performance in numerous NLP assignments [20, 2].

2.1.7 Distillation Technique

Combined frameworks especially multiple DP models presented challenges such as delay and computational assets in implementations for edge users. [36] proposed the general "distillation" concept. During training, knowledge from a vast and rich database could be learnt as **a teacher model**, but when deploying, through distillation, this entire body of knowledge was simplified as **a student model**, as shown in 2.11 [2]. This **student model** was deployed for inference.

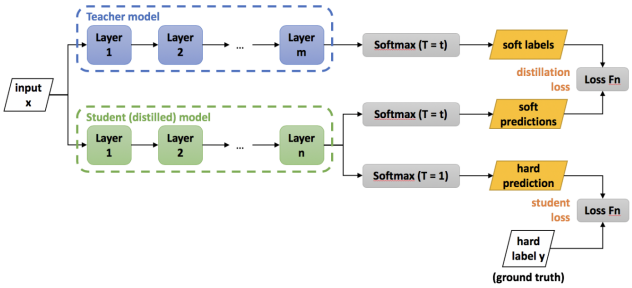


Figure 2.11: Distillation

The core idea was during classification, confidence scores predicted by models for different categories after many training epochs might increase significantly. For instance, the prediction for correct classes probably gained 0.98 probability, while predictions for other categories might be as low as 0.1 or even 0.00001. Intuitively, this discrepancy probably caused losses of potential knowledge, for instance, the degree of distinction between the correct category and those categories with merely 0.1 or 0.00001 probabilities [36, 97].

[2] https://devopedia.org/knowledge-distillation

The distillation addressed this by altering the original softmax function listed in Equation 2.3, and introducing a variable "distillation temperature T" listed in Equation 2.4, which **decreased differences between confidence scores**. For instance, "hard labels" (1 and 0) were probably transformed into "soft labels" such as 0.72 and 0.28, thereby conveying more information. Employing a greater T would derive more diffuse probability distributions across categories. [36, 97].

In knowledge distillation processes, **the teacher model** would output soft targets through temperature-scaled softmax function listed in Equation 2.4, then **student model** would be trained with those soft targets, decreasing cross-entropy distillation losses [36, 97] shown in Equation 2.2.

$$L_{ce} = -\sum_i t_i \cdot \log(s_i) \qquad [97] \tag{2.2}$$

t_i denoted "soft" probability distributions provided by **the teacher model**, including probabilities for categories that were not skewed towards extremes. On the other hand, s_i denoted probability distributions predicted by **the student model**. The student model reduced cross-entropy losses L_{ce}, aligning "soft labels" provided by the teacher model. Student and teacher models would share the same temperature during distilling. When deployment, the temperature of student model would be fixed as "1" returning the original softmax function listed in Equation 2.3 [36, 97]. In this case, t_i in Equation 2.2 would return "hard labels" (1 or 0).

$$f(z_j) = \frac{\exp(z_j)}{\sum_{k=1}^{N} \exp(z_k)} \qquad [45] \tag{2.3}$$

$$q_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \qquad [36] \tag{2.4}$$

2.2 Traditional Machine Learning Models Used in Project

2.2.1 K-Nearest Neighbors

KNN leveraged training data and categorized fresh observations according to proximity metrics. Assignments were operated by plurality voting among the k closest training entities for each test instance [58, 89]. The primary challenge was selecting an ideal number of adjacent entities to include [100]. One potential drawback was not efficient to deal with huge data, since its "lazy learning" had to conserve whole datasets [58].

Its general expressions were listed in Equation 2.5 and Equation 2.6. Given a set of adjacent entities $N_k(x)$ for a fresh observation x, the probability that x belonged to a class c given K neighbors could be calculated, using Equation 2.5. t indicated the category to which the instance was likely to belong. The anticipated category y for the point x was then determined by the class c with the highest probability [58] of Equation 2.5, listed in Equation 2.6.

$$p(t = c|x, K) = \frac{1}{K} \sum_{i \in N_k(x)} 1(t_i = c) \qquad [58] \tag{2.5}$$

$$y = \arg\max_c p(t = c|x, K) \qquad [58] \tag{2.6}$$

There were numerous distance measures available. Euclidean distance $d(x, y)$ between two points x and y was calculated as listed in Equation 2.7. Here, x_k denoted the k-th attribute of the entity x, y_k represented the k-th attribute of the entity y, and n was the aggregate count of attributes within data spaces [118].

$$d(x, y) = \sqrt{\sum_{k=1}^{n} (x_k - y_k)^2} \qquad [118] \tag{2.7}$$

Manhattan distance between two points x and y was defined as listed in Equation 2.8, which calculated the total of absolute deviations within corresponding elements across two samples [85].

$$MD(x, y) = \sum_{i=1}^{n} |x_i - y_i| \qquad [85] \tag{2.8}$$

Cosine distance, also termed "angular distance", between two points x and y was based on the cosine similarity quantifying the angular discrepancy between two samples. The cosine distance was calculated by deducting values of cosine similarity from one, as listed in Equation [85].

$$CosD(x,y) = 1 - \frac{\sum_{i=1}^{n} x_i y_i}{\sqrt{\sum_{i=1}^{n} x_i^2}\sqrt{\sum_{i=1}^{n} y_i^2}} \qquad [85] \tag{2.9}$$

Besides, there were two methods to derive weights for neighbors. The "uniform" approach assigned equal weights to all samples in a neighborhood area, calculating the influence of each point in the standard formula. The "distance" measure assigned weights based on "inverse" of the interval, making models sensitive to nearby points, emphasizing the effect of nearer samples [3].

2.2.2 Naive Bayes

NB [116, 119] leveraged Bayes' Theorem, assuming each attribute was statistically uncorrelated [55]. It could achieve satisfying performance even with limited datasets [89, 58], though results sometimes were constrained by that independent assumption [100].

Considering a set D comprising samples each characterized by a series of content features along with related categories. These features were denoted by the n-dimensional vector $\mathbf{X} = (x_1, x_2, \ldots, x_n)$ and associated with categories $C_1, C_2, \ldots, C_m$. NB assigned an instance $\mathbf{X}$ to a label C_i maximizing the "posterior probability", with assumptions of independent features. The "posterior probability" could be derived, as listed in Equation 2.10. The term $P(\mathbf{X})$ was a fixed value considering all categories. Besides, $P(C_i)$ could be disregarded, as assumptions of class equivalence that $P(C_1) = P(C_2) = \cdots = P(C_m)$ [98].

$$P(C_i|\mathbf{X}) = \frac{P(\mathbf{X}|C_i)P(C_i)}{P(\mathbf{X})} \qquad [98] \tag{2.10}$$

To further explain the methodology, the likelihood $P(\mathbf{X}|C_i)$ was formulated as the product of probabilities of each feature x_k conditioned on the class C_i, as listed in Equation 2.11 and Equation 2.12 [98].

$$P(\mathbf{X}|C_i) = \prod_{k=1}^{n} P(x_k|C_i) \qquad [98] \tag{2.11}$$

$$P(\mathbf{X}|C_i) = P(x_1|C_i) \times P(x_2|C_i) \times \cdots \times P(x_n|C_i) \qquad [98] \tag{2.12}$$

Nonetheless, algorithms would yield zero for the likelihood $P(\mathbf{X}|C_i) = P(x_1|C_i) \times P(x_2|C_i) \times \cdots \times P(x_n|C_i)$, when encountering the sample having a feature x_i that was not represented in training datasets for the specific category C_i. This absence invariably caused the null probability product, thereby compromising the performance of classifiers. "Laplace Estimator" could be added as a regularization constant, the category likelihood could be calculated by Equation 2.13, solving null likelihood issues [98].

$$P(C = c_i) = \frac{n_c + k}{N + n \times k} \tag{2.13}$$

- $P(C = c_i)$ signified the adjusted probability of category C after smoothing.
- n_c was the count of occurrences for **the specific category** in training datasets.
- k was referred to as Laplace smoothing parameter, such as 0.1 and 1.
- N was the total count of samples in training datasets.
- n was the count of distinct categories.

2.2.3 Support Vector Machine

SVM [116, 119] could build the "N-dimensional hyperplane" [82, 100, 95], splitting categories of training datasets with a "maximum margin". In forecasting a novel observation, models would analyze its position relative to that "N-dimensional hyperplane" [58, 118]. If this hyperplane had significant separation, also termed "margin" from the closest training sample, models were robuster [100]. These closest training samples were also named as "support vectors" [58]. SVM was more costly and time-consuming [89] but effective for some multivariate datasets [58]. There were different kernels for SVM to derive decision boundaries, such as linear, polynomial, and radial basis function (RBF) [100, 118].

[3] https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html

The linear SVM could be refined as "constrained optimization" questions, as listed in Equation 2.14. The goal was to achieve the best separation within categories by finding the decision boundary to maximize the "margin". The orientation of this boundary in feature spaces was determined by $\mathbf{w}$, while its positional bias from the baseline was decided by b. Besides, to accommodate outliers of datasets, each training sample $\mathbf{x}_i$ was associated with a slack variable ξ_i, quantifying the degree for samples breaching the margin. This slack variable was non-negative, penalizing anomalies with regularization parameter C indicating punishment intensities [58].

$$\begin{aligned} \text{minimize}_{\mathbf{w},b,\boldsymbol{\xi}} \quad & f(\mathbf{w},b,\boldsymbol{\xi}) = \frac{1}{2}\mathbf{w}^T\mathbf{w} + C\sum_{i=1}^{n}\xi_i \\ \text{subject to} \quad & y_i(\mathbf{w}^T\mathbf{x}_i + b) - 1 + \xi_i \geq 0 \quad i = 1,\ldots,n, \\ & \xi_i \geq 0 \quad i = 1,\ldots,n \qquad [58]. \end{aligned} \tag{2.14}$$

However, the linear kernel probably failed to effectively handle the reality data [118]. Polynomial and RBF kernels could project the initial dataset into a space of increased dimensions, where it still became divisible by hyperplanes [38]. Polynomial kernel algorithms were listed in Equation 2.15. The result was contingent on the orientation of samples through "dot product" in original feature spaces. d represented the complexity of transformed spaces [82].

$$K(\mathbf{x},\mathbf{x}_i) = (1 + \mathbf{x}\mathbf{x_i}^T)^d \qquad [82] \tag{2.15}$$

RBF kernel algorithms were listed in Equation 2.16, creating an expected "elevation" for every sample within the dataset. γ could control the spread of "elevations": larger values produced a tighter "elevation", whereas smaller ones made wider influence [82].

$$K(\mathbf{x},\mathbf{x}_i) = e^{-\gamma\|\mathbf{x}-\mathbf{x}_i\|^2} \qquad [82] \tag{2.16}$$

2.3 Pretrained Language Models Used in Project

2.3.1 RoBERTa

RoBERTa represented an enhanced version of BERT [72, 97], revealing BERT had not fully converged and discarding NSP from pre-training tasks. It was subject to pre-training on broader corpora with larger batch scales and extended training periods. Additionally, it improved MLM through dynamic masking techniques. RoBERTa would hide different parts of input data, each time the content fed into models, to prevent predicting identical masked tags consistently across successive iterations. Furthermore, the byte-based tokenization was employed, comprising a set of 50,000 tokens [53]. Commonly encountered terms would remain themselves, whereas infrequently occurring ones were segmented into sub-units [106].

2.3.2 DistilBERT

DistilBERT adhered same pre-training assignments as RoBERTa, while preserving identical training datasets to BERT [97].

DistilBERT was mainly pre-trained through distillation techniques mentioned in 2.1.7, minimizing MLM losses and distillation losses. It also considered their combined losses. Moreover, "cosine similarity" was found to be favourable for adjusting hidden representations of teacher and student models. The primary method to simplify large teacher models was decreasing encoder blocks to 6 [97].

Compared with BERT, DistilBERT emerged as approximately 40% fewer parameters, 60% faster, and holding 97% of BERT abilities for language comprehension. It responded to some concerns of AI community. The model could be pre-trained on large corpora, but there should be a smarter improved version during fine-tuning with fewer computing resources and more elasticities on edge devices. Besides, it could offer possible suggestions regarding environmental issues [97, 65, 54].

2.3.3 BART

Bidirectional and Auto-Regressive Transformers (BART) [46] combined bidirectional encoders with left-to-right auto-regressive decoders, like Transformer [113], and it could gain similar effectiveness to RoBERTa in NLP downstream assignments. Owing to decoder blocks, it could also contribute to language production and comprehension. [46] argued current "BERT-style denoising" (MLM) approaches possessed a certain degree

of task specificity, constraining model capabilities. To facilitate BART supporting different kinds of tasks, it employed various methods to corrupt input sequences, subsequently restoring them using decoders, which was also called "denoising", as shown in Figure 2.12.

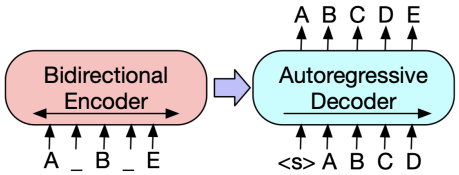


Figure 2.12: BART
[46]

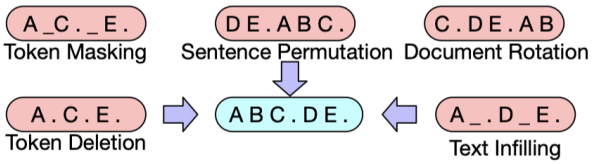


Figure 2.13: Different Denoising Approaches
[46]

Base version of BART had 6 encoder and 6 decoder blocks. In addition to components of Transformer as mentioned before in Figure 2.6, every decoder operated an additional attention mechanism with the last layer of the encoder. Moreover, it removed Feed-Forward Network (FFN) architectures from encoders. Cross-entropy losses were derived between initial input sequences and results of decoders. Five methods were employed to corrupt text in BART [46], also shown in Figure 2.13:

- Token Masking: similar to BERT, tokens were randomly replaced by "[MASK]".
- Token Deletion: some tokens were randomly dropped and BART had to figure out which units were removed.
- Text Infilling: it probably was the most challenging one. A series of **"continuous tokens" were selected, also named "span"**, and replaced by a single "[MASK]". BART was required to analyze omitted tokens in every span and take into account the length of original content.
- Sentence Permutation: the input sequence was split into several parts and mixed.
- Document Rotation: a token was randomly selected, and moving the segment from this token to subsequent ones, to the beginning. BART would predict the first token of original content.

It was profound that hidden states of the last decoder were extracted to prediction heads, considering whole decoder states [46], as shown in Figure 2.14, similar to "[CLS]" of BERT [19].

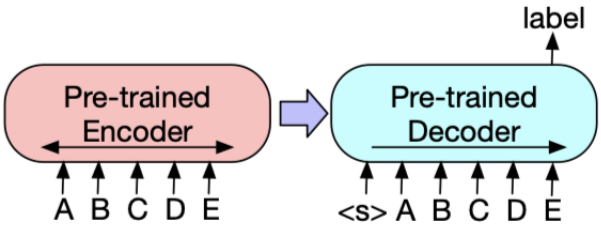


Figure 2.14: BART for Classification
[46]

2.3.4 Loss Function in Fine-Tuning Pretrained Language Models

As mentioned in 2.1.7, the cross-entropy loss function would be used when fine-tuning PLMs for both multi-class and multi-label classifications, calculating differences between probability scores predicted by models and actual labels as shown in Equation 2.17.

$$H(p, q) = -\sum_{x} p(x) \log q(x) \qquad [35] \tag{2.17}$$

- $p(x)$ denoted the true probability
- $q(x)$ denoted the predicted likelihood
- x signified various labels

For the correct label x, $p(x)$ would be 1 and $p(x')$ would be set to 0 for any other label x'. Consequently, the loss function penalized those incorrect predictions.

2.3.5 Optimizer in Fine-Tuning Pretrained Language Models

Loss functions could be reformulated as assignments of minimizing the objective function in relation to parameters. In this project, adaptive moment estimation (Adam) was used as the optimizer. Because its high computational efficiency and minimal memory demands [44] were particularly suitable for projects with numerous parameters, such as fine-tuning PLMs. The core Adam algorithm was listed in Algorithm Table 10.

In the loop, after each step was updated and deriving the current gradient g_t, the algorithm firstly calculated the average of past gradients and the current gradient, then calculated the average of the squares of past gradients and the square of the current gradient. Here, m_{t-1} was the average value from the previous step, α was the learning rate, and θ was the parameter. Subsequently, β_1^t and β_2^t decreased as the step increased, thereby $\hat{m}_t$ and $\hat{v}_t$ decreased, dynamically correcting "past knowledge". For instance, the θ parameter would be adjusted cautiously if $\hat{v}_t$ became larger. Besides, ϵ ensured the denominator was not zero [44].

Algorithm 1 Adam Algorithm Pseudocode [44]

while θ_t not converged **do**
  $t \leftarrow t + 1$ ▷ Update the step
  $g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$ ▷ Get gradients at the specific step t
  $m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ ▷ Update biased first moment estimate
  $v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ ▷ Update biased second raw moment estimate
  $\hat{m}_t \leftarrow \frac{m_t}{(1-\beta_1^t)}$ ▷ Compute bias-corrected first moment estimate
  $\hat{v}_t \leftarrow \frac{v_t}{(1-\beta_2^t)}$ ▷ Compute bias-corrected second raw moment estimate
  $\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\epsilon}$ ▷ Update parameters
end while
return θ_t ▷ Return parameters

2.3.6 Specific Techniques in Fine-Tuning Pretrained Language Models

Dropout would stochastically close some neurons within the network as shown in Figure 2.15, enhancing the model generalization [76] and preventing overfitting due to limited and noisy data [104].

Batch Normalisation improved "Internal Covariate Shift" issues by adjusting the distribution of **each** feature within a mini-batch, fitting "zero means and the unit variances", speeding up the convergence [39].

ReLU function was listed in Equation 2.18, introducing the nonlinearity.

$$f(z) = \max(0, z) \qquad [45] \tag{2.18}$$

2.4 Related Work

2.4.1 General Research about App Review Analysis

Investigations of application markets could be viewed as a significant and independent field within SE development [61]. There were various interesting yet practical studies on app reviews, coordinating relationships

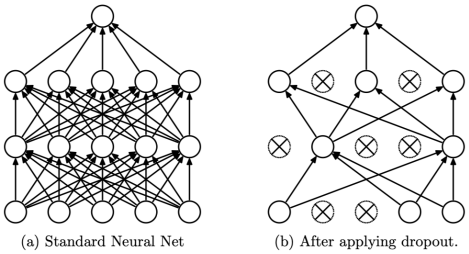


Figure 2.15: Dropout
[104]

between participants and applications [40]. [77] claimed participants probably expressed fewer words in reviews if they enjoyed applications, and revealed a phenomenon that the emergence of comments might have potential dynamic relationships with certain version publications. Similarly, [112] argued people were more likely to post detailed requirements when marking applications as 1 or 2 score. Besides, [102] explored comments of Bangladeshi agriculture programs and found within recognized "Human Values" related to software projects such as "accessibility" and "security", only half were identified in submitting feedback. [40] offered "SAFE" algorithm to retrieve specific software functionalities from user requirements, thereby enabling relevant software managers to organize strategies on different themes, and this model reduced the dependency on training samples. Moreover, [14] proposed "AR-Miner" architecture with "topic modeling" techniques, to obtain and visualize more insightful requirements, assisting product managers to uncover valuable ideas and suggestions in parallel. [33] considering essences of comments evolved over time, published "AOBTM" framework to derive appropriate number of history samples, analyzing several application versions with current trends.

2.4.2 Sentiment Analysis on App Reviews

SA was a sustaining study area [67] and it could also be employed in mining app reviews. [24] provided "RRGen" algorithm and considered sentiment scores contained in reviews as a feature, to self-reply participants' comments, which could improve their perceptions of products. From the view of "Crowd-Based RE", [43] offered "CrowdRE-Arg" model to discover focal points of contention, and part of the algorithm was composed by taking into account the emotion orientation of reviews. But it appeared not to share details about test labels used for SA. Other papers were also attempting to use SA as part of the project to analyze app reviews. Both [28] and [56] classified reviews into several categories such as error records and improvement suggestions, utilizing SA as one of main techniques from the process-level [28] or viewing sentiment scores as a feature from the data-level [56]. [32] conducted an in-depth examination and clustering of comments by assigning different weights to specific features and corresponding emotion scores through "topic modeling" and SA. [66] undertook a similar research to [32] but concentrating on tourism applications. [23] published "WisCom" to investigate opinions on various levels, including SA. However, it assumed review sentiments were just determined by scores given from users.

[30] adopted a hybrid approach integrating traditional ML methods, such as SVM and NB, to categorize comments. The proportion of words carrying positive or negative emotions was calculated as an attribute of data. Similarly, [89] conducted SA on user feedback using traditional ML techniques such as SVM and KNN, and combined them using "Bagging" approach, comparing their performance.

[29] also combined SA to return classified and representative requirements and it could pay more attention to unconventional or quieter participants. [60] not only intelligently analyzed relevant functionalities of products, but also collected comment sentiments, attempting to identify consumer inclinations. [75] published "SIMBA" architecture, attempting to augment similar frameworks previously mentioned, with capabilities to analyze reviews in various languages.

2.4.3 State-of-Art Techniques Used in Related Work

In terms of multi-class assignments, [84] downloaded pre-trained Italian BERT and fine-tuned it for SA on Italian tweets. They reiterated the notion that, to some extent, the purpose of employing pre-trained models was to obtain universal language representations. They highlighted the perspective: choosing pre-trained models based on the standard Italian corpus rather than tweet content was due to the difficulty in tweet collection

and the desire to hold representations from a broader spectrum. By the way, conducting SA on tweets could be referenced by this project, as tweets and app reviews shared common characteristics [96], elaborated in 2.5. [1] processed the similar fine-tuning project on comments of COVID-19 exposure notification applications and introduced a new category, "technique issue", in addition to traditional classes (positive, negative, and neutral). [59] compared results of LSTM and DistilBERT for SA on "Ride-Sharing" product feedback. However, the process of data labelling, such as the categorizing into positive, negative, and neutral, was not detailed. [74] also fine-tuned BERT for SA but in Indonesian, and they viewed user ratings of 1 and 2 as negative, 3 as neutral, 4 and 5 as positive. [35] fine-tuned BERT with public RE databases, and performed experiments such as a detailed categorization of non-functional requirements (NFRs).

Unlike these works, [2] utilized available RE datasets to "re-train" BERT specifically for RE, as shown in Figure 2.16, achieving significant outcomes. Besides, [5] employed a novel architecture "zero-shot learning" to conduct RE tasks with limited even no training samples and gained promising performance, pioneering a new RE direction.

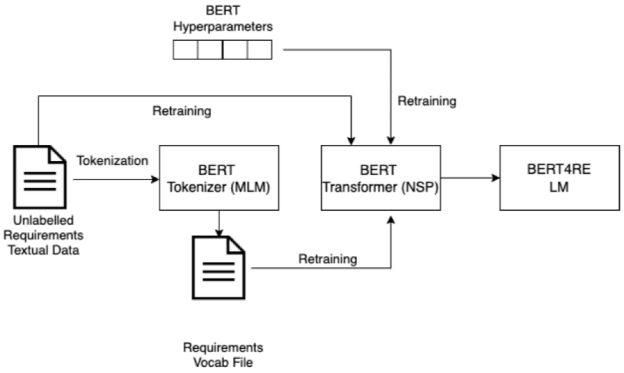


Figure 2.16: Retrain BERT for RE [2]

Furthermore, some studies used typical DP architectures as prediction heads of pre-trained models to generate outputs. [48] combined outputs of two RoBERTa with a proprietary "cross-attention mechanism" and CNN, to perform the aspect-level SA. [106] positioned LSTM over pre-trained RoBERTa for SA and utilized data augmentation through oversampling, to address data imbalance. [9] also attempted to stack LSTM or RNN on the top of pre-trained RoBERTa for SA.

In terms of multi-label assignments, [37] constructed multi-label tasks by combining sentiment classification with subjective-objective categorization. [108] processed the new kind of aspect-level SA, leveraging advantages of transfer learning for BERT to automatically analyze each document into numerous entity combinations. Conventional aspect-level SA probably required predetermined certain features reflecting to specific emotions.

2.5 Research Gaps and Intended Project Contributions

While transfer learning had proven effective for RE tasks, exemplified by NoRBERT [35], which successfully fine-tuned on typical RE datasets. Such successes might not be equally realized within app reviews. Because the specific vocabulary was more likely to hold distinct interpretative biases across various fields [90, 2, 95, 73] also known as "context feature bias" [121]. Therefore, if one wished to conduct SA or other forms of mining on app reviews using PLMs, it was imperative to select databases pertinent to the specific domain.

Unfortunately, this also posed significant challenges for collecting datasets. Firstly, a substantial portion of such app reviews were quite brief [25, 3, 60, 109, 37], as individuals might input them through handheld devices [24], which were less conducive to operate. Besides, owing to continuous updates of the product, words pertaining to different themes emerged in comments across various time periods [75, 23, 50, 33, 61], known as "data-drift" [16]. Furthermore, unlike news publications or scholarly papers, the integrity of these reviews was concerned in numerous studies [8, 3, 60, 81, 32, 55, 56, 3, 50, 8, 13, 73, 90], such as spelling errors [24], colloquial or idiomatic documents [95, 112, 25, 109], and some sentences merely containing emotions or ratings without any other opinions [55, 66, 56]. According to relevant papers, only one-third of the participant feedback were probably beneficial to developers [30, 14], just like "looking for a needle in a hay-stack" [50]. Due to the complexity of intricate semantic understandings in requirement documents [43], these different kinds of data issues

would further affect SA projects [25, 66]. Therefore, it was essential to continuously [16, 72] derive separate resources for app reviews.

Besides, conducting manual examinations of raw data labels was exponentially resource-intensive and difficult [96, 28, 109, 50, 16, 72]. In RE community, this issue was significantly magnified. [120, 16, 11, 10]. For another thing, [16] argued just few papers had assessed the quality of data labelling. It was more likely to be evident that database qualities had been overlooked by many articles.

Moreover, among those studies on fine-tuning PLMs for SA mentioned in 2.4.3, only [74, 106] provided corresponding outcomes or runtime under different hyper-parameters. [35] briefly mentioned batch size 16 yielded better results. [84, 1, 59, 9, 94] merely listed the selection of hyper-parameters without specific reasons. [53] also argued hyper-parameters of PLMs were crucial to outcomes but high cost of training limited detailed adjustments. These practices probably caused challenges for those attempting to replicate the research or integrate it into their own scholarly endeavors, thereby reducing the accessibility for academic purposes.

In terms of SA studies, the majority [106, 1, 59, 48, 9, 74, 105] tended to employ various PLMs for multi-class experiments, subsequently comparing effects between model variants or occasionally adding an additional category. Regarding multi-label studies, [37] merely combined sentiment with subjective-objective classifications to construct the multi-label architecture, and [108] built the multi-label through aspect-level SA, with matching sentiments with specific entities.

In response to aforementioned issues such as dataset qualities, hyper-parameters, and SA research, intended contributions of this project are:

1. Mitigating the gap in understanding how to select NLP tools within NLP4RE [120], investigating influences of various NLP pipelines on results.
2. Evaluating the effect of different hyper-parameters on test results.
3. Sharing a manually inspected dataset of app reviews to serve as a knowledge repository for pre-training or fine-tuning models on “high-quality data” [76].
4. Conducting sentiment classifications on reviews using traditional ML models and PLMs through multi-class and multi-label experiments.

3 Research Methodology

This chapter outlines the project methodology, beginning by dataset sources in 3.1, discussing data preprocessing in 3.2 and labelling in 3.3. Different NLP pipelins are explained in 3.4, followed by reasons of model selections in 3.5. Implementation details for benchmark models are in 3.6 and 3.7 for PLMs. Two data augmentation are conducted in 3.8. Evaluation metrics are in 3.9 and project environment is in 3.10.

3.1 Data Collection

The dataset titled "Top 20 Play Store App Reviews (Daily Update)", utilized in this project, was sourced from Kaggle [4]. Kaggle had officially recognized its quality. Furthermore, studies such as [9, 2, 106, 108] also utilized Kaggle as data sources.

This database compiled reviews of the top 20 applications on Google Play Store, collected through `google-play-scraper` API [5] into a unified dataset, also called "Collected through Application Programming Interface" [90]. It was acceptable, as evidenced by [63, 16] on app review analyses within SE both claimed a considerable number of experiments were conducted with data from Google Play Store. Besides, reviews from higher-ranked products would encompass more information [55], thereby rendering the final dataset invaluable for future fine-tuning researches. Additionally, collecting reviews from various applications aided in the database with broader characteristics [32, 120] also named "between-app analysis". In contrast, applications with lower rankings were not considered in this project due to their insufficient provisions of comments [56, 55].

3.2 Data Preprocessing

Not only to ensure database applicability, but also to guarantee high-quality data did not compromise results [40, 84], meticulous data preprocessing was prioritized:

1. Remove columns of "review id" and "app name" from the dataset and only retain reviews and related scores from users.
2. Drop missing values and duplicates from the dataset [59], ensuring that if multiple instances were identical, only one was kept.
3. Utilize `emoji` API [6] to convert any emoji presented in the review into textual representations [2].
4. Exclude characters while keeping letters and digits using `Regular Expression` [7].
5. Preserve three punctuation marks: period (.) comma (,) and apostrophe ('), to simplify the process of manually inspecting labels.
6. Utilize `lower()` function to transform all uppercase letters in reviews into the lowercase [31, 43, 14, 24, 50, 11, 94, 23, 75] for normalisation.
7. Eliminate feedback shorter than 8 words [2] to ensure each comment contained relatively substantial content regarding requirements [55, 56]. However, it was important to emphasize that previously mentioned punctuation marks were also included in this calculation.
8. Utilize `langdetect` API [8] to remove non-English reviews [88, 2].

3.3 Data Labelling

There existed only a slight dependency between the number of stars submitted by participants and the mood of comments [61, 3, 62]. Such a mechanism provided fewer insights to management groups, since one comment probably contained both positive and negative feelings [32, 60, 24, 56]. Consequently, `SentiStrength` API [9] was used as a dictionary-based [8] tool to automatically generate emotional scores for reviews, as shown in Figure 3.1.

[4] https://www.kaggle.com/datasets/odins0n/top-20-play-store-app-reviews-daily-update?select=Candy+Crush+Saga.csv
[5] https://www.kaggle.com/code/odins0n/play-store-app-reviews-scrapper-daily-update/notebook
[6] https://pypi.org/project/emoji/
[7] https://docs.python.org/3/library/re.html
[8] https://pypi.org/project/langdetect/
[9] https://pypi.org/project/PySentiStrength/



Tool	Approach	Scores	Languages	Technology	License
SentiStrength	Lexicon based	Estimates strength of positive and negative sentiments	English (partly tested: Dutch, Finnish, German, Italian, Russian, Spanish, Turkish)	Command line (Java)	Commercial, closed source (free for academic research)

Figure 3.1: SentiStrength API [62]

3.3.1 SentiStrength

`SentiStrength` could automatically generate **both** a positive socre from 1 to 5, and a negative score from -1 to -5 for each review [61]. 1 and -1 meant the lack of corresponding emotions. 5 and -5 meant filled with the corresponding emotions [31, 32, 75, 8]. It ensured granular analyses of emotions [111], thereby supporting multi-label experiments. Then by comparing absolute values of two kinds of scores, the sentiment with the larger value determined overall sentiment orientation of the review [24, 55, 8] to support multi-class experiments. If both scores were equal, the sentence was labelled as neutral.

Although [56, 61] claimed `SentiStrength` could perform well on brief sentences, the primary reason for selecting it as the starting point for data labelling [62] was it yielded favorable results with non-structured content [32, 111]. Its mechanism was simple: it was actually an emotion lexicon that assigned emotion grades to each word in the given input, ultimately deriving scores by calculating the difference between the maximum and minimum value among those words [29, 8]. `SentiStrength` could also update its rules with additional data provided [32, 62], though it was not considered in this project.

Besides, this project compared the inconsistency **between sentiment labels derived from original scores (1 and 2 for negative, 3 for neutral, 4 and 5 for positive [60, 61, 26]) submitted by users and labels calculated by SentiStrength scores (emotion with higher absolute value as corresponding sentiment category; if equal, as neutral)** to observe `SentiStrength` overall performance. The own-created Equation 3.1 was proposed to measure the disagreement between them. It was aimed to observe whether `SentiStrength` could serve as the initial labelling tool for related experiments. Relying on participant ratings to represent sentiment was not precise, as people might be coerced into scoring the software [112]. Here, offering only a rough overview.

$$\text{Warning Ratio} = \frac{\text{number of reviews with categories that were not same in two scoring systems}}{\text{length of the whole dataset}} \tag{3.1}$$

3.3.2 Annotation Guideline and Cohen Kappa

After labelling, 8000 samples were randomly selected as the initial dataset. Subsequently, they were filtered to select representative [57] ones, and their sentiment scores and categories were manually inspected by the author. An annotation guideline [31, 75, 96] was established to dictate which data should be eliminated.

Furthermore, the dataset **was divided into five equal parts and distributed** among various individuals to assess the agreement on labelling. Each person received an additional annotation guideline that introduced this project and outlined potential scoring rules. However, adherence to these rules was not mandatory. Finally, the Cohen's Kappa, as listed in Equation 3.2 was used to express the consistency [16, 10, 43] among us.

$$K = \frac{P_o - P_e}{1 - P_e} \qquad [27] \tag{3.2}$$

- The observed proportion of consistence, denoted as P_o was the probability that two agents agreed on the given rating [27]. For instance, if there were 100 items to be labelled and two annotators agreed on 80 of them, then P_o would be 0.8.
- P_e represented the probability of chance agreement [27], reflecting the likelihood of two annotators reaching consensus by chance, when labelling was done in a random manner.
- $1 - P_e$ could normalize the numerator scaling the final outcome from -1 to +1 [27].

3.3.3 Rules of Annotation Guidelines

Annotation guidelines for scoring and filtering representative comments were developed based on suggestions found in papers. For filtering rules in Table 3.1:

Table 3.1: Rules for Selecting Representative Comments

Rule Number	Rules
1	1. Drop reviews that only appeared to be transformed from python emoji api, such as: "smile face smile face". If the review included both suggestions and these "emoji words", it would be kept.
2	2. Drop reviews that only contained emotional praises, such as: "This app is fantastic, I love it so much.". However, if the praise included discussions about app features, like "This software is really great, especially the design of navigation bar.", it would be kept.
3	3. Drop reviews that only contained emotional complaints, such as: "Ahhhhh, this software is really terrible! How could you design it like this?". However, if the complaint included discussions about app features, like "This software is really terrible, you should redesign the user interface.", it would be kept.
4	4. Drop politically related reviews even with possible software suggestions.
5	5. Reviews expressing long-term affection for a game or other platforms were not considered, as they cannot provide useful information on current or past features.
6	6. If reviews described short videos or similar platforms as "interesting", it would be kept, as this may suggest certain recommendation algorithms were effective.
7	7. If reviews contained the praise/complaint about certain versions, it would be kept, as this may indicate feature designs in the new version update probably (not) be favorable.
8	8. If reviews included positive feedback for the existing app feature, it could be kept as the evidence of good quality.
9	9. Appropriate inferences could be accepted. For example, a comment like "This software is great, I can communicate very well with my family everyday." could be kept. Because it may imply positive feedback on the communication feature, and suggest that this back-and-forth communication seemed to have few bugs in good usability. It even hinted at a certain level of user experience availability, as family members of various ages were likely to be able to operate this app.
10	10. Drop reviews that were incomprehensible without specific logic, as less useful information could be extracted and time-consuming.
11	11. Drop reviews that contained the possible bias or made the annotator uncomfortable, such as "insults" and "toxicity".
12	12. If there was a really tricky review: As long as this review was considered to be insightful for software developers or management, it could be kept.

Rule 1, 2, 3, and 5 were created inspired by [77, 57, 55]. Participant feedback, only representing "evaluative" content, accounted for a significant proportion [77]. However, as they cannot contain information regarding specific requirements and offered little aid to RE [55, 57, 102], they were omitted. Rule 6 was own-created, when short video platforms or similar products continually delivered things to users through algorithms, praises from them probably implied appropriate recommendation systems. Rule 7 was essential, since reviews were dynamic as previously mentioned, incorporating certain temporal information [107, 12, 50] and locating issues to the specific release [16]. Rule 8 could inform programmers about which features were favoured or disliked by users [23]. Participants probably also expressed many negative words but ultimately left a positive or moderate summary [107]. Therefore, it was impractical to encapsulate all rules, thereby Rule 9 and 12 offered the flexibility. Rule 10 was created to filter out comments lacking good structure once again. Rule 11 was necessary considering ethics issues [65].

For scoring rules in Table 3.2:

Table 3.2: Rules for Scoring Comments

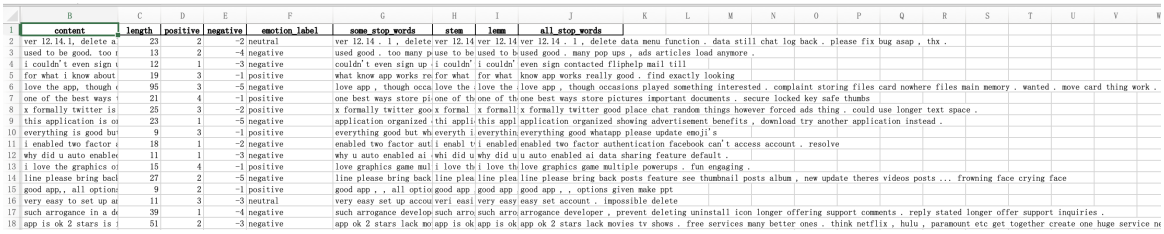
Rule Number	Rules
1	1. Some phrases in sentences might confuse you, such as "thumb up", which were actually emojis turned into English words. When considering emotional intensity, allocate more weights to them.
2	2. When making a judgment, you could firstly determine if the review had only one type of emotion: if there was only one emotion, you only need to assess the emotional intensity, scoring within 2∼5 or -2∼-5. (Here, there was no 1 or -1 because if there was only one type of emotion, it must be within 2∼5 or -2∼-5; the other non-present emotion could be directly set to 1 or -1.)
3	3. The length/structure of a sentence could also be a basis for judgment (e.g. if a phrase was repeated many times or a comment listed many similar interrogative sentences), or certain specific adjectives and adverbs.
4	4. For some very cold comments like "UI was accepted." then directly rate it as (-1, 1).
5	5. *Encourage appropriate association: For example, if a comment expressed negative emotions about security but positive emotions about UI design, and you were really torn about which emotional score prevailed, it could be marked with more negative scores and fewer positive scores. Because evidently, security was more important than UI design.*
6	6. If you encountered a comment where the positive and negative scores were neck and neck in terms of emotional intensity, just set it to the corresponding level of neutral, such as (3, -3) or even (-5, 5).
7	7. If a sentence had two kinds of emotions, do not panic or fear. Consider the scores for each emotion separately, and then compare their absolute values to determine the emotion_label.

Rule 1 emphasized assigning more weights to the emoji. This rule was reasonable, as some studies mentioned the appearing emoticon could map the entire sentence sentiment [26, 110]. Rules 2 and 4 re-emphasized scoring mechanisms. Rule 3 was inspired by `SentiStrength` [110, 111], where one of rules involved scoring adjustments based on the repetition of letters. Here, it was assumed that repetitive patterns probably conveyed the stronger intensity. Rule 5 and 6 also provided the flexibility of rules, preventing issues faced by Rule 9 and Rule 12 in **previous filtering rules**.

3.4 Natural Language Processing Pipelines

To offer NLP suggestions to RE, various NLP pipelines were separately applied, as shown in Figure 3.2.

- content: reviews without NLP pipeline.
- some_stop_words: reviews after removing some stopwords
- stem: reviews after stemming.
- lemm: reviews after lemmatization.
- all_stop_words: reviews after removing all stopwords



	content	length	positive	negative	emotion_label	some_stop_words	stem	lemm	all_stop_words
2	ver 12.14.1, delete a	23	2	-2	neutral	ver 12.14 . 1 , delete	ver 12.14	ver 12.14	ver 12.14 . 1 , delete data menu function . data still chat log back . please fix bug asap , thx .
3	used to be good. too r	13	2	-4	negative	used good . too many p	use to be	used to b	used good . many pop ups , ads articles load anymore .
4	i couldn't even sign i	12	1	-3	negative	couldn't even sign up .	i couldn'	i couldn'	even sign contacted fliphelp mail till
5	for what i know about	19	3	-1	positive	what know app works re	for what	for what	know app works really good . find exactly looking
6	love the app, though c	95	3	-5	negative	love app , though occa	love the	love the	love app , though occasions played something interested . complaint storing files card nowhere files main memory . wanted . move card thing work .
7	one of the best ways t	21	4	-1	positive	one best ways store pi	one of th	one of th	one best ways store pictures important documents . secure locked key safe thumbs
8	x formally twitter is	25	3	-2	positive	x formally twitter goo	x formal	x formall	x formally twitter good place chat random things however forced ads thing . could use longer text space .
9	this application is ot	23	1	-5	negative	application organized	thi appli	this appl	application organized showing advertisement benefits , download try another application instead .
10	everything is good but	9	3	-1	positive	everything good but wh	everyth i	everythin	everything good whatapp please update emoji's
11	i enabled two factor a	18	1	-2	negative	enabled two factor aut	i enabl t	i enabled	enabled two factor authentication facebook can't access account . resolve
12	why did u auto enablec	11	1	-3	negative	why u auto enabled ai	whi did u	why did u	u auto enabled ai data sharing feature default .
13	i love the graphics of	15	4	-1	positive	love graphics game mul	i love th	i love th	love graphics game multiple powerups . fun engaging .
14	line please bring back	27	2	-5	negative	line please bring back	line plea	line plea	line please bring back posts feature see thumbnail posts album , new update theres videos posts ... frowning face crying face
15	good app, , all option	9	2	-1	positive	good app , , all optio	good app	good app	good app , , options given make ppt
16	very easy to set up ac	11	3	-3	neutral	very easy set up accou	veri easi	very easy	easy set account . impossible delete
17	such arrogance in a de	39	1	-4	negative	such arrogance develop	such arro	such arro	arrogance developer , prevent deleting uninstall icon longer offering support comments . reply stated longer offer support inquiries .
18	app is ok 2 stars is f	51	2	-3	negative	app ok 2 stars lack mo	app is ok	app is ok	app ok 2 stars lack movies tv shows . free services many better ones . think netflix , hulu , paramount etc get together create one huge service ne

Figure 3.2: Different NLP Pipelines Used in Project

3.4.1 Tokenization

Tokenization [11, 90, 95] could segment these comments into individual units [66] for subsequent operations [42] such as stemming. The NLTK `TweetTokenizer` [10] [2] was used to apply that because, as previously mentioned, reviews shared similar characteristics with tweets avoiding possible parsing errors.

3.4.2 Lemmatization

Lemmatization [11, 24, 10, 94] could revert certain words to the original version keeping the same meaning [32, 50, 29], according to a source dictionary. For example, "fixing", "fixed", and "fixes" would be changed as "fix" [56, 55]. The NLTK `WordNetLemmatizer` [11] was used to apply that.

3.4.3 Stemming

Stemming [31, 81, 10, 30, 26, 90, 95] could remove "prefixes" and "suffixes" from specific words [66, 8, 78, 55], converting them into forms that were more commonly encountered in the grammar [50]. However, it probably caused some potential mistakes such as "over stemming" and "under stemming", failing to properly remove certain affixes [42]. Besides, stemming probably focused on the morphology of content rather than semantics. For instance, "better" could be reverted as "good" through lemmatization, but "goods" probably also be changed as "good" through stemming [55, 56]. The NLTK `PorterStemmer` [12] [47] was used and it would pay more attention to the suffix elimination.

3.4.4 Stopwords

In English sentences, stopwords such as "was" and "the" [66] referred to those frequently occurred [42, 60] but did not contribute to the overall meaning [56, 50, 29, 25], serving merely to construct the grammar. Removing them [31, 32, 81, 8, 11, 10, 94, 30, 26, 90, 95] might decrease unnecessary noise in data [56, 55, 3, 78]. However, certain words containing important information within requirements might also be mistakenly categorized as stopwords [109, 93] such as "should" and "must" [55, 56]. Removing them potentially affected the judgment of the emotion intensity [66]. Therefore, two stopword lists were respectively adopted: one from the public NLTK source [13], and the other custom-created one consisting some personal pronouns and articles.

These pipelines could decrease irrelevant features of datasets, such as removing stopwords and reverting some words to original versions [42].

3.5 Model Selection

ML was widely utilized in analyzing app reviews, with SVM and NB being more popular [50, 10], as shwon in Figure 3.3. SVM was also the popular tool in SA [90]. Besides, KNN, also termed "lazy learning" was a classical model and it did not build a universal representation space [100]. Therefore, incorporating it into the model list was meaningful. In this project, NB, SVM, and KNN were selected as traditional ML models for benchmarking.

In terms of advanced PLMs, there were three main architectures: "decoder-based", "encoder-based", and "encoder-decoder-based" [71]. Among them, "encoder-based" frameworks like BERT were more likely to be favored [84] and appropriate for review classifications [120, 71], since it was good at generating a predictive feature for every given series [88, 5]. Consequently, an improved version of BERT, RoBERTa [48, 53], was

[10] https://www.nltk.org/api/nltk.tokenize.casual.html
[11] https://www.nltk.org/api/nltk.stem.wordnet.html
[12] https://www.nltk.org/howto/stem.html
[13] https://www.nltk.org/search.html?q=stopwords

Table 12 Distribution of machine learning techniques used in primary studies in the period form 2010 to December 31, 2020

Type	Machine Learning Techniques	No. Studies	Percentage
Supervised	Naïve Bayes	43	24%
	Support Vector Machine	39	21%
	Decision Tree	31	187%
	Logistic Regression	23	13%
	Random Forest	20	1%
	Neural Network	12	7%
	Linear Regression	7	4%
	K-Nearest Neighbor	4	2%
Unsupervised	Latent Dirichlet Allocation	36	20%
	K-Means	8	4%

Figure 3.3: Popular NB and SVM [16]

employed in this project. Furthermore, DistilBERT [97] was considered, because it was lighter and quicker to deploy on edge devices, compared to BERT, which could be assumed as promising for RE. The "decoder-based" architectures [35], such as GPT-2 and GPT-3 auto-regressive models, were more suitable for linguistic production assignments [46], such as textual translation and condensation [5], without fully leveraging pre-training capabilities. Thus not considered in this project. BART belonging to "encoder-decoder-based" architectures [71], was also selected to explore the paradigm divergence between "encoder-decoder-based" and "encoder-only" designs. Besides, "encoder-decoder-based" was assumed to efficiently solve text classifications and generation assignments simultaneously [88].

3.6 Training Traditional Machine Learning Models

3.6.1 Feature Selection

Term Frequency-Inverse Document Frequency (TF-IDF) [31, 43, 75, 10, 94, 30, 86, 90] was a highly recognized vocabulary value-assigned feature selection approach, used in this project. It could derive the relative significance of words across all documents (reviews) [11, 3], as listed in Equation 3.3.

$$tf - idf_{(t,d,D)} = tf_{(t,d)} \times idf_{(t,D)} \qquad [3] \tag{3.3}$$

- The term frequency, denoted as $\text{tf}(t, d)$, represented the number of occurrences of the term t in the review d.
- The inverse document frequency, denoted as $\text{idf}(t, D)$, calculated the importance of the term t across a set of reviews D. It was determined by, dividing the total count of reviews N by **the count of documents including the term** $\text{df}(t)$, and subsequently operating the logarithm of results, as listed in Equation 3.4:

$$\text{idf}(t, D) = \log \frac{N}{\text{df}(t)} \qquad [3] \tag{3.4}$$

Terms with higher tf-idf scores meant they were prevalent within a singular review yet infrequent across the entire corpus [50]. And terms with lower tf-idf scores tended to be those were common or even ubiquitous across the corpus [3, 55, 56, 78, 89]. Subsequently, respective weights of words determined by tf-idf scores could be further analyzed through various ML models [10]. The `TfidfVectorizer` [14] was used from scikit-learn library in this project to derive tf-idf scores.

3.6.2 Validation and Hyperparameters

The dataset was split into 80% for training (and validation), and 20% for testing. 10-fold cross-validation (CV) [3, 30, 43, 96, 18] was utilized on training data to identify the most suitable hyper-parameters. Specifically, training data was randomly partitioned into 10 equal subsets, with 9 subsets being used for training and one subset for validation to produce accuracy results. This process would be repeated 10 times, with each of 10 subsets used exactly once for validation. And the average accuracy from 10 iterations was calculated to one setting of hyper-parameters [10].

KNN Hyperparameters:

[14] https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

- the number of nearest neighbors from 1 to 40.
- different distance measures: Euclidean, Manhattan and Cosine.
- two weight metrics: one based on general functions and the other sensitive to closer data points.

NB Hyperparameters:

- different Laplace smoothing coefficients.

SVM Hyperparameters:

- different penalty coefficients C.
- different kernel functions: linear, polynomial, and RBF.

Then, the best hyper-parameters were applied to testing data, which was not used in cross-validation, ensuring the experiment reliability [11].

3.7 Fine-Tuning Pretrained Language Models

3.7.1 Prediction Head

Similar to [35], the prediction head was constructed: a layer of dropout [104] followed by a standard linear layer, then a batch normalization layer [39], a ReLU activation function layer [45], and the final output layer. To accommodate differences between multi-class and multi-label assignments, output dimensions of the final layer should be adjusted as 3 (for multi-class) or 10 (for multi-label).

3.7.2 Frozen Layers

Word representations would be learnt and stored in all layers of PLMs. Top-level layers were more likely to capture potential patterns of whole input sequences [20]. And low-level layers would pay more attention to the basic linguistics knowledge [83].

Compared with large corpora utilized by huge PLMs, the data in this project for fine-tuning was relatively minuscule, exposed to the risk of overfitting [104]. Hence, it was meaningful to explore the performance if forbidding parameters updated in low-level layers, keeping the original linguistics knowledge and only fine-tuning the high-level layers.

3.7.3 Validation and Hyperparameters

Owing to the expensive computing cost [11], PLMs were fine-tuned without CV. Dataset was split into 80% for training, 5% for validation, and 15% for testing. Similar strategies were also found in [1]. The number of epoch in this project was fixed as 20. As the epoch count increased, the classifier would perform excellently on the training set learnt multiple times, but show poorly on test data, known as "overfitting" [35]. And some hyper-parameters probably performed well **only** on the specific test data. The validation set was utilized for evaluating interim results, tackling these concerns. Eventual results were derived from 15% test data [30] to maximize the experiment dependability. Multi-class and multi-label experiments would share the same hyper-parameters.

3.8 Data Augmentation

In this project, two data augmentation methods were implemented. The first one involved augmenting the quantity of samples in traditional ML, to enhance generalization abilities of models. It was implemented by utilizing synonym dictionaries to randomly replace words or phases in reviews [106].

However, there had been many debates regarding data augmentation for language models, especially PLMs. In other disciplines, such as the image recognition, images could be rotated or resized; for the sound recognition, sound waves could be slightly adjusted. To the best of knowledge, there was no universally recognized strategy for enhancing language models based on neutral networks. In this project, following instructions from [117], published by the Stanford University, after reviews had been tokenized, those tokens would be randomly replaced by "[UNK]" tokens as "blank noising", artificially introducing the noise as a form of regularization [117].

3.9 Evaluation Metrics

In this project, four metrics were utilized for testing the model performance: precision, recall, accuracy, and F1-score. The accuracy was primarily used to get a general impression of the model performance when comparing different hyper-parameters. Precision, Recall, and F1-Score were used as metrics to compare final model performance, and they were also common indicators employed in most studies [10, 16, 28, 31, 43, 50, 55, 66, 107, 47, 86].

The precision function was listed in Equation 3.5. True Positive (TP) was data points, who were positive and predicted also as positive. False Positive (FP) was data points, who were negative but predicted as positive. Precision could describe the degree of reliability when the framework predicted a category [27].

$$\text{Precision} = \frac{TP}{TP + FP} \qquad [27] \tag{3.5}$$

The recall function was listed in Equation 3.6. False Negative (FN) was data points, who were positive but predicted as negative. Recall could describe the capability of a framework to find all correct instances of the certain category [27].

$$\text{Recall} = \frac{TP}{TP + FN} \qquad [27] \tag{3.6}$$

Accuracy denoted the probability that the framework correctly predicted the category of a given data point, as listed in Equation 3.7. However, it probably masked the deficiency of models in predicting the specific category accurately, especially for imbalanced data [27].

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \qquad [27] \tag{3.7}$$

The F1-score performed the model performance by assigning the equal importance to precision and recall [50], integrating them together as the "harmonic mean" [10] as listed in Equation 3.8. There were also other strategies to allocate weights between them [27].

$$F1\text{-Score} = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} \qquad [27] \tag{3.8}$$

3.10 Experiment Environment

`SentiStrength` required the current environment to contain both `python >= 3.7` and `java JRE >= 1.8.0`. For traditional ML experiments, KNN, NB, and SVM classifiers were achieved by `KNeighborsClassifier`, `MultinomialNB`, and `SVC` from scikit-learn [15]. To find appropriate hyper-parameters, `GridSearchCV` [16] was used to exhaustively test the performance of each hyper-parameter, and then conducted the cross-validation (CV) [21]. `time` was used to record running time. Jupyter Notebook files (.ipynb format) were selected to execute these experiments.

To fine-tune PLMs, PyTorch [17] was imported with encapsulated neutral network layers. Learning rate and weight decay of the Adam optimizer were adjusted, while other parameters were default values from PyTorch [18] with β_1 being 0.9, β_2 being 0.999 and ϵ being 1e-8. Three PLMs: RoBERTa, DistilBERT, and BART were downloaded from Hugging Face [19], and `transformers` was imported to operate these models. **Three PLMs were all uncased and base versions**. `TensorBoard` and `logging` were utilized to record these experiments. `DistributedDataParallel` was used to support training on multiple GPU devices. The experiment processes were programmed on Python files, which were **efficient** to adjust hyper-parameters and GPUs, but executed through `bash` commands. `argparse` was imported to receive command parameters. However, bash commands were not executed in the terminal due to the challenge for recording the history. A clever solution was to running them through Jupyter Notebook files, also allowing to use `time` to record. The server website for the multiple GPU environment was Featurize [20] with 4 "V100 Tesla 16 GB and 8 cores Xeon Gold 5218R".

[15] https://scikit-learn.org/stable/index.html
[16] https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html
[17] https://pytorch.org/docs/stable/index.html
[18] https://pytorch.org/docs/stable/generated/torch.optim.Adam.html
[19] https://huggingface.co/
[20] https://featurize.cn/

4 Results Analysis

This chapter shows experiment results with quantitative and qualitative analyses.

4.1 SentiStrength Performance

Data, collected on October-20th-2023, December-14th-2023, and January-28th-2024, were separately gathered. Each of them hold 200,000 reviews. After **same** data preprocessing, the warning ratio mentioned in 3.1 was listed in Table 4.1.

As shown in Table 4.1, `SentiStrength` displayed stable performance, with the warning rate of around 44% across databases of different periods.

It was implied that, after data preprocessing, about 56% participant emotions (derived form submitted scores) were consistent with categories derived from `SentiStrength`. Therefore, sentiment scores and categories from `SentiStrength` could be viewed as an acceptable initial point.

Table 4.1: Warning Rate of SentiStrength

Published time	October 20th, 2023	December 14th, 2023	January 28th, 2024
Number of representative reviews	64,554	62,717	63996
Number of warning tags	28,305	27,271	28492
Warning Rate	0.4385	0.4348	0.4452

4.2 Data Attributes

The dateset published on January 28th, 2024, was selected as the main experiment data due to the latest release. After data cleaning, 63,996 reviews remained. From them, 8000 reviews were randomly selected. Following the manual inspection for sentiment scores, related categories, and reviews by the author, 5,289 representative reviews (containing requirement information) were retained. The distribution of sentiment categories was shown in Figure 4.1. One without manual checking was shown in Figure 4.2.

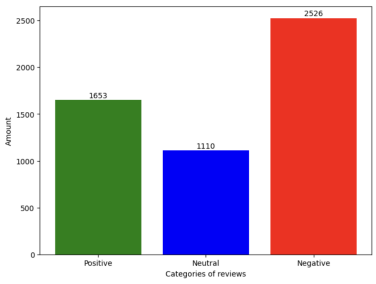


Figure 4.1: Sentiment Category Distribution After Manual Checking

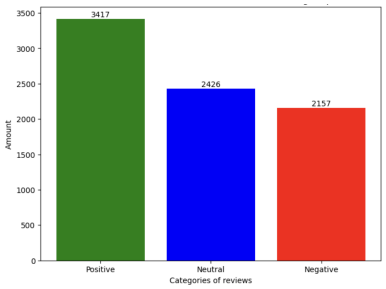


Figure 4.2: Sentiment Category Distribution Without Manual Checking

Besides, the category distribution after "synonyms" data augmentation was shown in Figure 4.3.

The distribution of sentiment scores were shown in Figure 4.4 for positive and Figure 4.5 for negative.

The distribution of scores without manual checking were shown in Figure 4.6 and Figure 4.7.

During multi-label experiments on scores without manual checking, as there was only one instance for "+5" shown in Figure 4.6 , an error would occur. One possible solution was to extract 30 reviews with (1, -1) scores, changing the positive score of first 15 to "+5" and the negative score of last 15 to "-5", allowing models to continue training. Since comments with (1, -1) scores were numerous in the original dataset as shown in Figure 4.8, discarding some of them would not impact overall results. Then, score distributions were slightly improved, shown in Figure 4.9 and Figure 4.10.

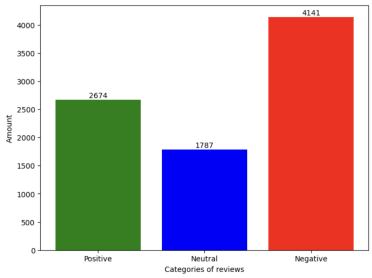


Figure 4.3: Category Distribution After Synonyms Data Augmentation

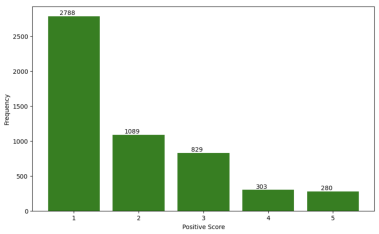


Figure 4.4: Positive Scores Distribution After Manual Checking

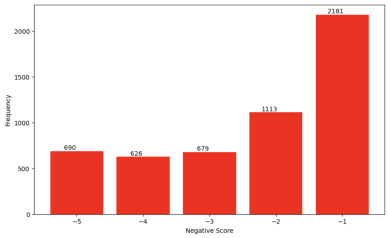


Figure 4.5: Negative Scores Distribution After Manual Checking

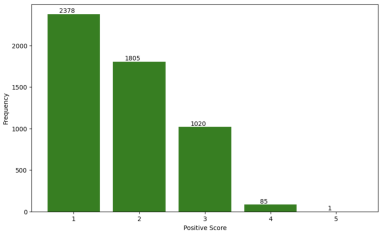


Figure 4.6: Positive Scores Distribution Without Manual Checking

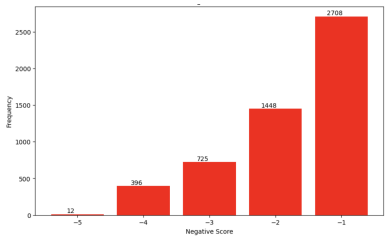


Figure 4.7: Negative Scores Distribution Without Manual Checking

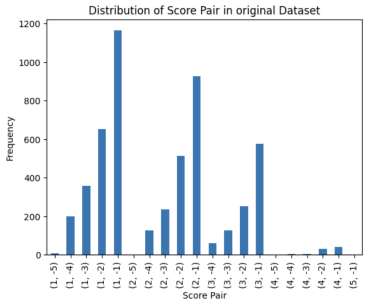


Figure 4.8: Sentiment Score Pair Distribution

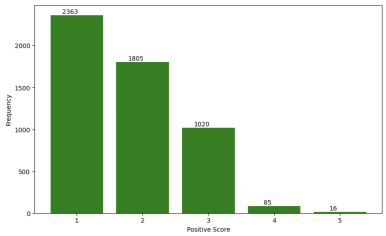


Figure 4.9: Smooth Positive Scores Distribution Without Manual Checking

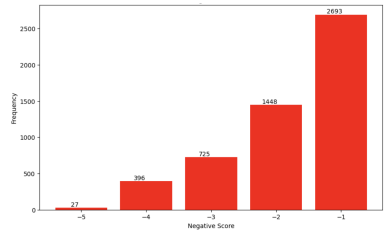


Figure 4.10: Smooth Negative Scores Distribution Without Manual Checking

The length distribution of reviews was shown in Figure 4.11 and Figure 4.12, and after removing the punctuation, statistical information was compiled and listed in Table 4.2 with some items:

- Count: the sum count of reviews at the specific category.
- Mean / Standard Deviation: the mean and standard deviation of the review length.
- Minimum / Maximum: the minimum and maximum review length at the specific category.
- 25%, 50%, 75%: comment length quartiles at 25%, 50% (median), 75% at the specific category.

Table 4.2: Length Statistical Information of Reviews

Sentiment Category	Count	Mean	Standard Deviation	Minimum	25%	50%	75%	Maximum
negative	2526	34.37	23.48	7	16.0	27	45	113
neutral	1110	24.74	19.11	8	12.0	18	30	104
positive	1653	24.35	19.95	7	12.0	18	30	418

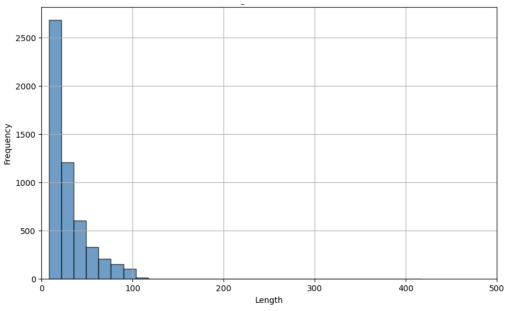


Figure 4.11: Review Length Distribution-a

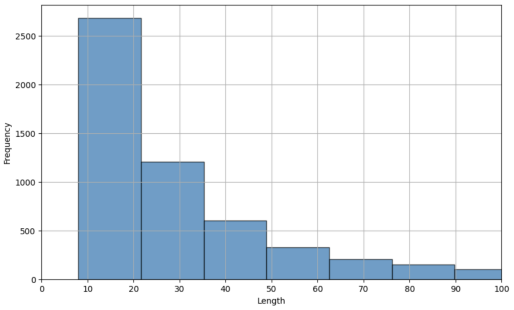


Figure 4.12: Review Length Distribution-b

4.2.1 Quantitative Analysis

For multi-class categories, manually checked dataset had very unbalanced distributions as shown in Figure 4.1, with negative reviews being the most common 2526, followed by positive ones 1653, and only 1110 neutral ones. As for data from `SentiStrength`, it also presented imbalance shown in Figure 4.2, and interestingly, it had higher proportions of positive ones. After "synonyms" augmentation, the number of reviews had increased, but the imbalance still remained shown in Figure 4.3.

For multi-label scores, manually checked dataset shown in Figure 4.4, revealed that for positive scores, "1" was the most frequent 2788. Distributions of "2" and "3" were similar, as well as for "4" and "5"; for negative scores shown in Figure 4.5, the number of "-1" was the most frequent 2181. "-2", "-3", "-4", and "-5" had similar distributions. Dataset from `SentiStrength` displayed abnormal distributions with almost no reviews with scores at "5" or "-5". And there were concentrations around scores "1", "2", "3", "-1", "-2", and "-3" shown in Figure 4.6 and Figure 4.7.

As shown in Table 4.2, the average length of negative reviews was longer, while that value of neutral and positive ones was nearly identical. Given different data quantities, this probably introduced a bias. However, by observing their standard deviations to measure the volatility, the impact appeared to be insignificant. Furthermore, by examining quartiles: 25%, 50%, 75%, it was clearly demonstrated the length of negative reviews were generally longer than neutral and positive ones.

4.2.2 Qualitative Analysis

Imbalanced inspected data [11] was acceptable, as each product might hold different emotional distributions [62], and fewer neutral comments were existed reported in [96, 1, 59]. The length of negative comments was longer in the same background, containing enough requirement information [96]. Besides, the length of reviews was related to different software categories [112].

It could be assumed that users typically posted reviews when having strong feelings about some aspects [8]. Besides, analyzing comments from only one day might introduce biases, as positive feedback probably increased in next day due to software improvements. For example, during the pandemic, Zoom received a significant amount of negative feedback [33]. Regarding distributions for both sentiment inspected categories and categories from `SentiStrength`, it was speculated training data used to develop `SentiStrength` rules was collected from various areas [111]. These rules might not be entirely applicable in RE, leading to a different distribution with post-inspection data.

Simultaneous advantages and disadvantages within products [107] caused complex scenarios, where sentiment score distributions could be utilized for further analyses. **Fewer "-1" scores than "+1", meant negative emotion probably was more common in all reviews** shown in Figure 4.4 and Figure 4.5. Furthermore, negative sentiment distributions (-2, -3, -4, -5) were more balanced and concentrated compared to positive

ones, with fewer comments at positive scores "+4" and "+5". **This also indicated negative sentiments contained in comments were more prevalent than positive**, after removing unrepresentative and shorter ones. Abnormal score distributions from `SentiStrength` were not absolutely wrong, but it probably provided less information with developers, making them hard to consider priorities.

4.3 Confusion Matrix and Kappa Results

Confusion matrix [27] was used to intuitively show differences between labels from the author and labels from others:

- "Annotator A" in each figure meant the author.
- "Annotator B" meant others.

In terms of sentiment categories, they were shown in Figure 4.13, 4.14, 4.15, 4.16, and 4.17 with five different students. Kappa values were listed in Table 4.3.

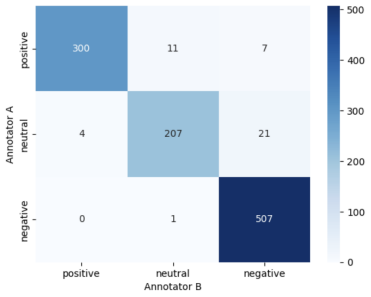


Figure 4.13: Confusion Matrix for Sentiment Category with First Annotator

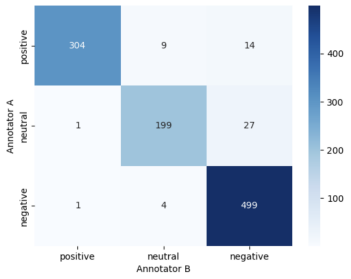


Figure 4.14: Confusion Matrix for Sentiment Category with Second Annotator

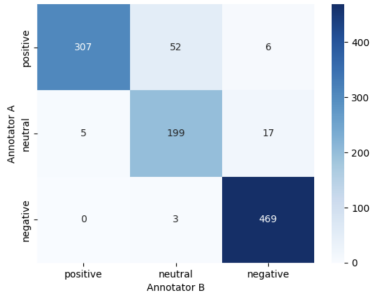


Figure 4.15: Confusion Matrix for Sentiment Category with Third Annotator

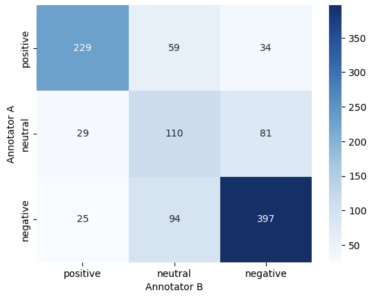


Figure 4.16: Confusion Matrix for Sentiment Category with Fourth Annotator

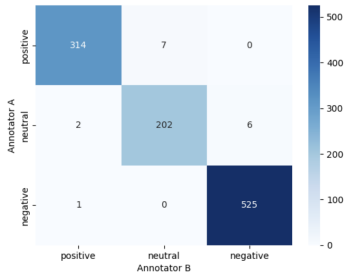


Figure 4.17: Confusion Matrix for Sentiment Category with Fifth Annotator

Table 4.3: Kappa Values for Sentiment Category

First Annotator	Second Annotator	Third Annotator	Fourth Annotator	Fifth Annotator	Average Value
0.9335	0.9152	0.8775	0.5176	0.9756	0.8439

In terms of sentiment scores, they were shown in Figure 4.18, 4.19, 4.20, 4.21, and 4.22. Average Kappa values between positive and negative scores, were listed in Table 4.4.

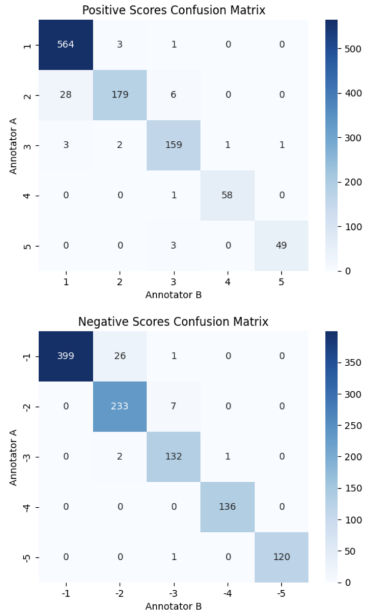


Figure 4.18: Confusion Matrix for Sentiment Scores with First Annotator

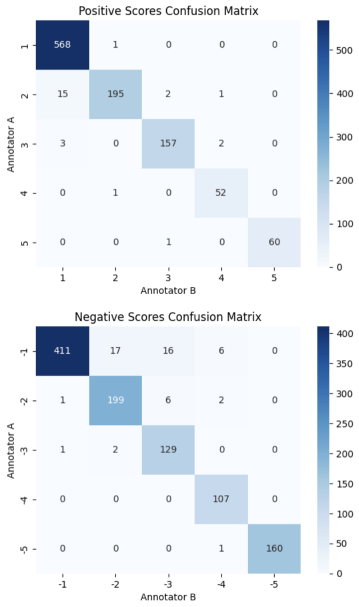


Figure 4.19: Confusion Matrix for Sentiment Scores with Second Annotator

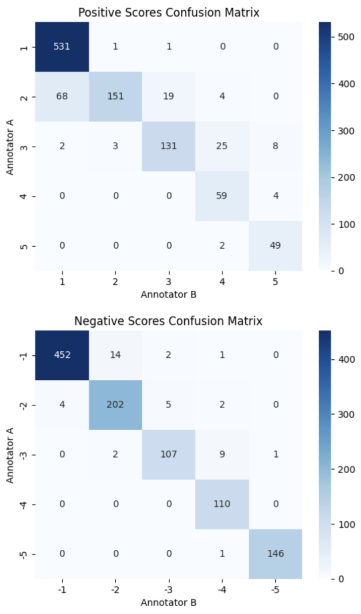


Figure 4.20: Confusion Matrix for Sentiment Scores with Third Annotator

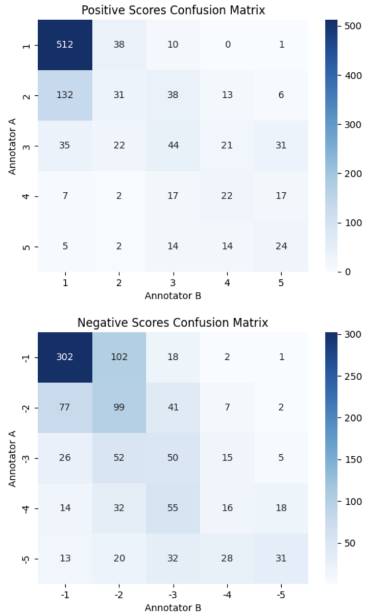


Figure 4.21: Confusion Matrix for Sentiment Scores with Fourth Annotator

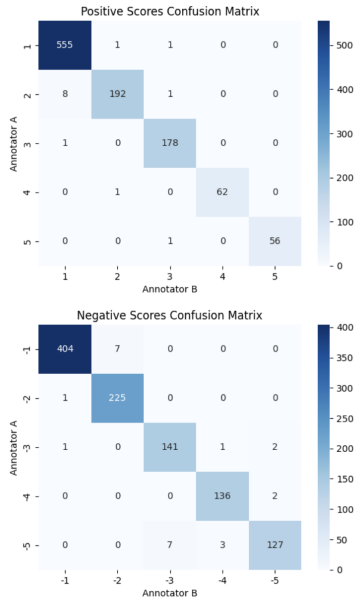


Figure 4.22: Confusion Matrix for Sentiment Scores with Fifth Annotator

Table 4.4: Kappa Values for Sentiment Scores

	First Annotator	Second Annotator	Third Annotator	Fourth Annotator	Fifth Annotator	Average Kappa for Five Annotators
Kappa for Postive Scores	0.9268	0.9613	0.8007	0.3414	0.9796	0.8020
Kappa for Negative Scores	0.9518	0.9337	0.9465	0.2788	0.9698	0.8161
Average Kappa Between Positive and Negative Scores for Each Annotator	0.9393	0.9475	0.8736	0.3101	0.9747	0.8090

4.3.1 General Analysis

For Kappa criteria [27]:

- 0.81 to 0.99 meant almost perfect consistency.

- 0.61 to 0.80 meant considerable consistency.
- 0.41 to 0.60 meant moderate consistency.
- 0.21 to 0.40 meant fair consistency.

As shown in Table 4.3, the first, second, third, and fifth annotators strongly agreed with categories from the author. Although consistency with fourth one was moderate with 0.5176, examining details was still useful. Observing the confusion matrix of Figure 4.16, it seemed that he did not endorse some of neutral comments, classifying many neutral comments as negative. Consequently, from the first and third rows, it was noted that he categorized many author's positive and negative comments as neutral or the opposite, **with numbers decreasing accordingly**. This probably suggested that author's overall grading standard was reasonable, but there might be disagreements between us regarding definitions of "emotional boundaries". The average Kappa value of 0.8439 as shown in Table 4.3, indicating almost perfect consistency, suggested sentiment categories after manually checking were reliable.

Table 4.4 exposed same issues. The average Kappa of the fourth annotator between positive and negative scores was just 0.3101, the fair consistency. Unlike Figures 4.18, 4.19, 4.20, and 4.22, colors of diagonals in confusion matrixes were prominent for both positive and negative scores, indicating higher agreements. Color distributions in Figure 4.21, were quite sparse. For positive scores, he agreed with most author's scores of "1". However, he might consider scores of "2" and "3" too definitive, as many author's "2" and "3" were changed by him to "1". The numbers of "4" and "5" decreased as sentiment reduced, which was reasonable due to different "emotional boundaries". For negative scores, he probably agreed with most "-1" and "-2" scores, but there were fewer disagreements for "-3", "-4", and "-5". The **final average** Kappa of average positive and negative Kappa values from five annotators was 0.8090 shown in Table 4.4, suggesting sentiment scores after manually checking were still reliable.

Furthermore, it was important to claim that each of annotators did not manually check all reviews. 5289 reviews were randomly divided into five equal parts. Although the number of reviews for each category or score in each file might differ, the confusion matrix and kappa coefficients focused more on relative proportions rather than absolute counts. Therefore, result analyses were persuasive.

4.4 Ablation Study for Language Model Data Augmentation

Removing "blank nosing" and testing performances of RoBERTa on multi-class experiments with different dropout, shown in Figure 4.23 with Legend 4.24. Same operations were applied to weight decay, shown in Figure 4.25 with Legend 4.26. **Steps** on the x-axis represented `backward()` updating weight parameters was called once.

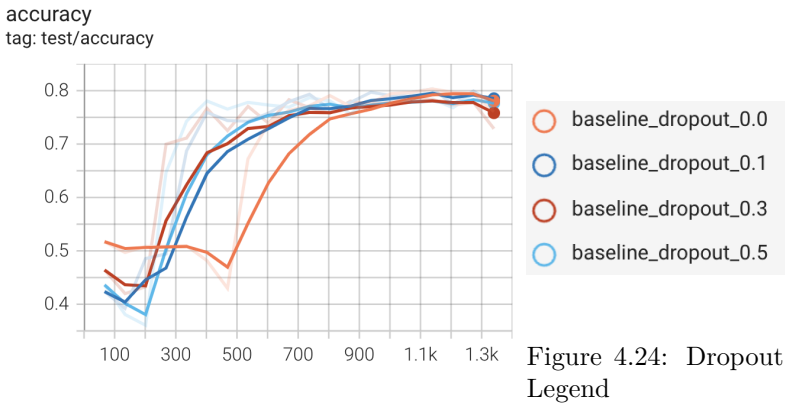


Figure 4.23: Ablation Study Observing Dropout

Figure 4.24: Dropout Legend

4.4.1 Quantitative Analysis

When "blank noising" data augmentation was **removed**, dropout 0.1, 0.3, and 0.5 showed the similar upward trend, but dropout 0.0 did not perform well with slower convergence. Additionally, the performance of weight decay 1e-8 was not satisfying, while larger values such as 1e-5 and 1e-3 performed better.

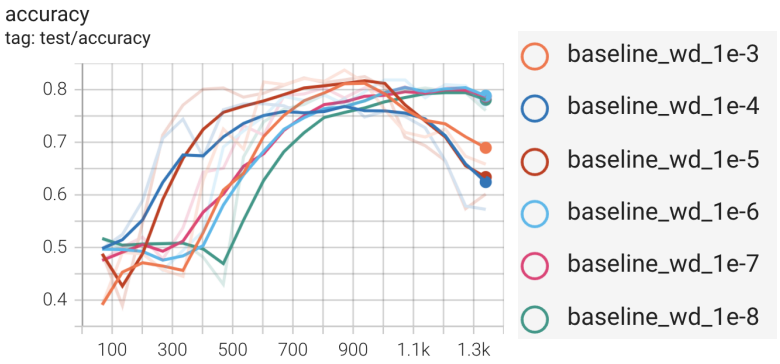


Figure 4.25: Ablation Study Observing Weight Decay

Figure 4.26: Weight Decay Legend

4.4.2 Qualitative Analysis

This probably implied introducing blank noises into training data could have similar effects to dropout and weight decay, **acting as regularization to penalize models or weights, thus to prevent overfitting**. Without adding noises, it became more necessary to employ dropout rates such as 0.1 or 0.3, or higher weight decay.

PLM experiments did not considered "synonyms" data augmentation. One of the principal concerns was neutral networks conducted assignments through complex and dispersed patterns **rather than** "count" mechanisms [117] such as TF-IDF. If two reviews were similar, it probably affected the reliability of results. Specifically, if models had learnt one review representation with some weights during training, it probably encountered a nearly identical pattern. **The model might "memory" the knowledge, not "learn" from data.** Although satisfying scores would be derived, generalization abilities to reality situations were not really enhanced.

Second potential concern was if sentences were replaced through the synonym without altering sentiment scores, this might affect the authenticity of results for multi-label experiments. Since changing the synonym might affect sentiment intensities of sequences, particularly with adjectives and adverbs.

4.5 Multi-Class Experiments Results

Multi-class experiments results were shown in Figure 4.27 for accuracy. Figure 4.28, Figure 4.29, Figure 4.30, Figure 4.31, Figure 4.32, Figure 4.33 showed F1-scores of different models over three sentiment categories, eliminating trade-offs between precision and recall [10]. The x-axis of graphs represented different NLP pipelines. Results were also listed in Table 4.5 for accuracy. Table 4.6 and Table 4.7 for precision, recall and F1-score. Besides, to intuitively compare effects of NLP pipelines, best results obtained by particular techniques within models were highlighted in bold and underlined in tables. Among them, some abbreviations in figures, tables, and analyses meant:

- content: reviews after manually checking. (also called "inspected data" in analyses)
- some stopwords: reviews dropping some stopwords.
- stem: reviews undergone stemming.
- lemm: reviews undergone lemmatization.
- all stopwords: reviews dropping all stopwords.
- original: reviews without manually checking. (also called "uninspected data" in analyses)
- aug: reviews augmented through the "synonym" in traditional ML experiments.

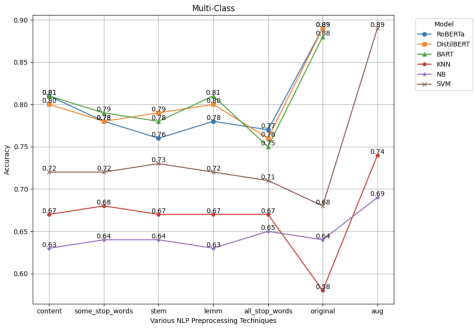


Figure 4.27: Accuracy on Multi-Class Experiments Across Various Pipelines

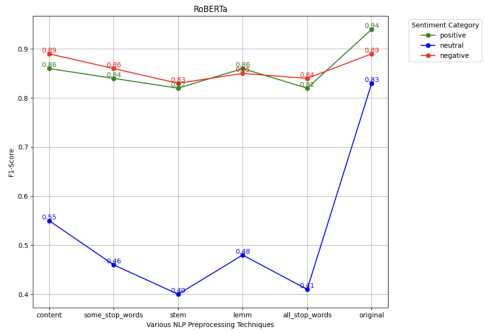


Figure 4.28: RoBERTa F1-Scores on Multi-Class Experiments Across Various Pipelines

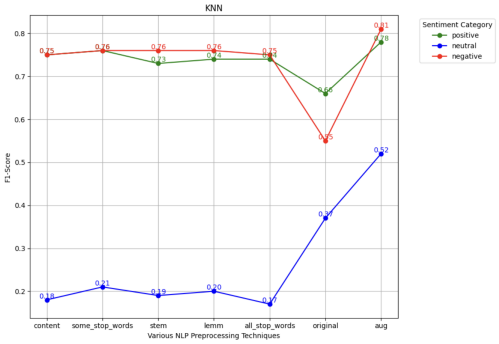


Figure 4.29: KNN F1-Scores on Multi-Class Experiments Across Various Pipelines

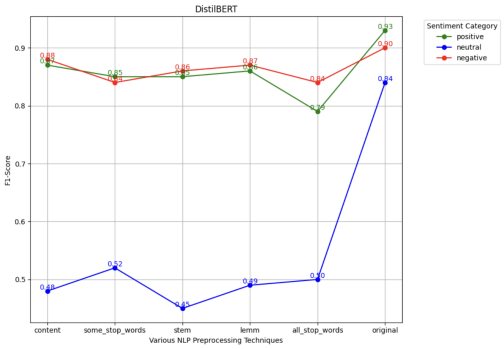


Figure 4.30: DistilBERT F1-Scores on Multi-Class Experiments Across Various Pipelines

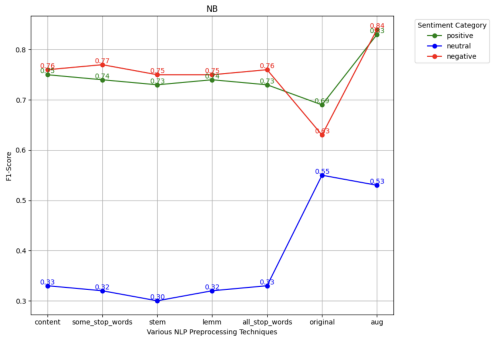


Figure 4.31: NB F1-Scores on Multi-Class Experiments Across Various Pipelines

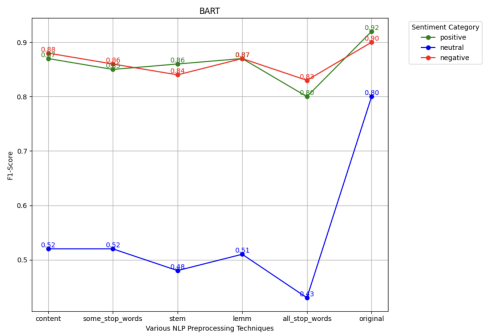


Figure 4.32: BART F1-Scores on Multi-Class Experiments Across Various Pipelines

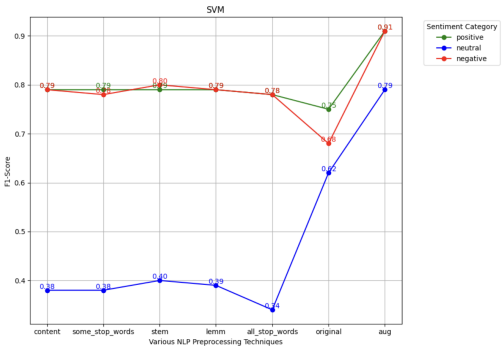


Figure 4.33: SVM F1-Scores on Multi-Class Experiments Across Various Pipelines

Table 4.5: Multi-Class Experiments Accuracy Results

	KNN	NB	SVM	RoBERTa	DistilBERT	BART
content	0.67	0.63	0.72	**0.81**	**0.80**	**0.81**
some stopwords	**0.68**	0.64	0.72	0.78	0.78	0.79
stem	0.67	0.64	**0.73**	0.76	0.79	0.78
lemm	0.67	0.63	0.72	0.78	**0.80**	**0.81**
all stopwords	0.67	**0.65**	0.71	0.77	0.76	0.75
original	0.58	0.64	0.68	0.89	0.89	0.88
aug	0.74	0.69	0.89	\	\	\

Table 4.6: Multi-Class Experiments Results (Precision, Recall, and F1-score) for Traditional ML Models

	Sentiment Categories of KNN									Sentiment Categories of NB									Sentiment Categories of SVM								
	Positive			*Neutral*			*Negative*			*Positive*			*Neutral*			*Negative*			*Positive*			*Neutral*			*Negative*		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
content	0.75	0.75	0.75	0.5	0.11	0.18	0.65	0.89	0.75	**0.79**	**0.7**	**0.75**	**0.52**	0.24	**0.33**	0.67	**0.88**	0.76	**0.8**	0.78	**0.79**	**0.59**	0.28	0.38	0.71	**0.89**	0.79
some stopword	0.75	**0.77**	**0.76**	**0.56**	**0.13**	**0.21**	**0.66**	0.88	**0.76**	**0.79**	0.69	0.74	0.48	0.24	0.32	**0.68**	**0.88**	**0.77**	0.79	0.78	**0.79**	0.54	0.29	0.38	0.71	0.87	0.78
stem	0.74	0.71	0.73	0.49	0.12	0.19	0.65	**0.9**	**0.76**	0.78	0.69	0.73	0.51	0.21	0.3	0.65	0.87	0.75	0.78	**0.8**	**0.79**	0.58	**0.3**	**0.4**	**0.73**	0.88	**0.8**
lemm	**0.76**	0.72	0.74	0.48	**0.13**	0.2	**0.66**	0.89	**0.76**	**0.79**	0.69	0.74	**0.52**	0.23	0.32	0.66	**0.88**	0.75	**0.8**	0.79	**0.79**	0.55	**0.3**	0.39	0.72	0.87	0.79
all stopword	0.72	**0.77**	0.74	0.47	0.1	0.17	**0.66**	0.87	0.75	0.77	**0.7**	0.73	0.47	**0.25**	**0.33**	**0.68**	0.86	0.76	0.79	0.78	0.78	0.54	0.25	0.34	0.7	0.88	0.78
original	0.62	0.79	0.66	0.52	0.53	0.37	0.62	0.49	0.55	0.7	0.69	0.69	0.54	0.55	0.55	0.63	0.63	0.63	0.77	0.74	0.75	0.59	0.66	0.62	0.72	0.64	0.68
aug	0.77	0.72	0.78	0.74	0.25	0.52	0.72	0.91	0.81	0.87	0.8	0.83	0.73	0.42	0.53	0.76	0.93	0.84	0.92	0.9	0.91	0.88	0.71	0.79	0.87	0.95	0.91

Table 4.7: Multi-Class Experiments Results (Precision, Recall, and F1-score) for PLMs

	Sentiment Categories of RoBERTa									Sentiment Categories of DistilBERT									Sentiment Categories of BART								
	Positive			*Neutral*			*Negative*			*Positive*			*Neutral*			*Negative*			*Positive*			*Neutral*			*Negative*		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
content	**0.93**	0.80	**0.86**	**0.54**	**0.55**	**0.55**	**0.86**	**0.92**	**0.89**	**0.86**	**0.87**	**0.87**	**0.56**	0.42	0.48	0.84	**0.92**	**0.88**	0.86	**0.88**	**0.87**	**0.57**	0.48	**0.52**	**0.87**	**0.90**	**0.88**
some stopword	0.81	0.86	0.84	**0.54**	0.39	0.46	0.83	0.89	0.86	0.85	0.84	0.85	0.48	**0.57**	**0.52**	**0.88**	0.81	0.84	**0.88**	0.82	0.85	0.54	**0.51**	**0.52**	0.85	0.89	0.86
stem	0.84	0.80	0.82	0.48	0.34	0.40	0.78	0.89	0.83	0.84	**0.87**	0.85	0.53	0.39	0.45	0.83	0.89	0.86	0.85	0.87	0.86	0.50	0.47	0.48	0.86	0.85	0.84
lemm	0.87	0.85	**0.86**	0.53	0.43	0.48	0.81	0.88	0.85	**0.86**	0.85	0.86	0.54	0.45	0.49	0.84	0.90	0.87	**0.88**	0.87	**0.87**	0.55	0.48	0.51	**0.87**	0.89	0.87
all stopword	0.78	**0.87**	0.82	**0.54**	0.33	0.41	0.81	0.87	0.84	0.83	0.75	0.79	0.49	0.50	0.50	0.81	0.86	0.84	0.86	0.75	0.80	0.48	0.38	0.43	0.80	**0.90**	0.83
original	0.94	0.94	0.94	0.84	0.82	0.83	0.88	0.90	0.89	0.93	0.92	0.93	0.82	0.86	0.84	0.91	0.89	0.90	0.91	0.93	0.92	0.89	0.73	0.80	0.92	0.97	0.90

4.5.1 Quantitative Analysis

For **accuracy** of models shown in Figure 4.27 and Table 4.5, fine-tuned PLMs performed clearly better than traditional ML on inspected reviews. DistilBERT was 0.80 slightly behind RoBERTa 0.81 and BART 0.81, SVM getting 0.72 performed better than KNN 0.67, and NB showed the worst outcome 0.63. Regarding **uninspected reviews**, ML models performed poorly on data without manually checking, except NB, for instance, SVM got 0.72 on inspected data but 0.68 on uninspected data. In contrast, PLMs yielded high scores, for instance, DistilBERT got 0.80 on inspected data but 0.89 on uninspected data. This was an interesting phenomenon.

In terms of different **NLP pipelines** for PLMs, with exceptions of DistilBERT and BART performing identically with lemmatization and inspected reviews, shown in Table 4.5, all other techniques had a certain negative impact, especially removing all stopwords. BART only got 0.75 on reviews dropped all stopwords. For traditional ML, NLP pipelines resulted in insignificant improvements, with removing some stopwords performing well in KNN from 0.67 to 0.68, removing all stopwords in NB from 0.63 to 0.65, and stemming in SVM from 0.72 to 0.73. Moreover, the "synonym" augmentation performed very well in traditional ML models. For example, SVM got 0.89 on augmented data.

Regarding **F1-scores** for each category of **figures** shown in Report 4.5, for **positive and negative reviews**, all three PLMs demonstrated nearly similar results. Within ML, SVM performed the best, while NB and KNN showed similar and moderate results. For **neutral** comments, RoBERTa had the best performance at 0.55, with DistilBERT being the lowest at 0.48 and BART at 0.52. However, they all outperformed ML models, with SVM at 0.38, NB at 0.33, and KNN the lowest at 0.18. In terms of **uninspected reviews**, all models followed similar trends to their accuracy performance. PLM models generally performed better on uninspected data. For instance, RoBERTa got 0.86 on inspected data but 0.94 on uninspected one. While traditional ML behaved differently, performing poorly on positive and negative reviews but much better on neutral reviews. For example, NB got 0.75 on inspected data but 0.69 on uninspected data for positive category. It got 0.33 on inspected data but 0.55 on uninspected data for neutral category. Moreover, "synonym" data augmentation showed good performance across all sentiment categories. For instance, SVM gained 0.38 on neutral reviews but 0.79 on augmented data.

For **NLP pipelines** on PLMs, there were many satisfying results achieved without them shown in Table 4.7, with dropping all stopwords only improved recall of positive reviews for RoBERTa from 0.80 to 0.87. Stemming and lemmatization showed almost no improvement. However, dropping some stopwords performed better in neutral reviews for DistilBERT and BART. For traditional ML shown in Table 4.6, they were greatly benefited from these techniques, with lemmatization and dropping some stopwords being useful for many metrics in KNN and NB, but SVM seemed to benefit more from stemming.

4.5.2 Qualitative Analysis

The "benchmark" mentioned below referred to traditional ML models.

PLMs generally outperformed benchmarks, validating fine-tuning induced by transfer learning was an effective approach in RE [35]. Results from DistilBERT suggested distillation techniques, learning extra knowledge from soft labels, were also applicable in RE. However, it was observed that benchmarks performed poorly on uninspected data, whereas PLMs excelled. That was probably because PLMs with extensive training on large corpora could better understand underlying meanings in uninspected data, even with significant noises. In contrast, benchmarks relied on manually extracted features like TF-IDF, lacking linguistic knowledge of PLMs. Thus, while benchmarks benefited from manually checked data, but struggled on uninspected reviews without complex semantics knowledge.

The performance between PLMs and benchmarks across various NLP pipelines could also be explained by previously discussed reasons. For benchmarks, techniques such as stemming and lemmatization would reduce irrelevant feature dimensions [3, 60, 10], allocating more weights to key features [11, 100, 28]. [84, 3] mentioned stemming could improve SA, validated on performance of NB and SVM. Advantages of removing some stopwords [3] were also validated on performance of KNN and NB, whereas dropping all stopwords demonstrated unstable results on benchmarks. Therefore, removing pre-compiled all stopwords might indeed break meanings of sentences, affecting model performance [93]. By the way, benchmarks, through the synonym augmentation, were exposed to richer dataset and gained comprehensive features, deriving improved outcomes. For PLMs, already equipped with extensive knowledge to **manage various word forms**, those NLP techniques might have the minimal impact, potentially even deviating from sentence semantics, especially dropping all stopwords.

Regarding the performance across each category, all models exhibited fatigue on imbalanced data. This was reasonable, as the distribution where neutral comments were fewer. Classifiers would bias more prevalent categories, while neglecting less common ones [43], also termed "imbalanced learning" [11]. Possible solutions could be Synthetic Minority Oversampling Technique (SMOTE) [50, 106] by supplementing more neutral reviews, or "under-sampling" by decreasing negative reviews. However, increasing data for the category with fewer instances could lead to overfitting, while decreasing data for the category with abundant instances would result in models learning less knowledge [11].

Across all categories, SVM outperformed NB and KNN, suggesting its efficiencies in mapping data to high-dimensional spaces. In contrast, assumptions of word independence in reviews for NB [80] were not satisfying compared to more sophisticated models. Moreover, BART and RoBERTa exhibited similar performance, whereas DistilBERT underperformed on neutral reviews, suggesting distilled models probably required ideal conditions to deploy. Notably, dropping some stopwords was more effective for DistilBERT on neutral reviews, implying eliminating irrelevant features might enhance semantic analyses of distilled models in tough imbalanced situations.

4.6 Multi-Label Experiments Results

Multi-label experiments results were shown in Figure 4.34 for accuracy. Figure 4.35, Figure 4.38, Figure 4.36, Figure 4.39, Figure 4.37, Figure 4.40 showed F1-scores of PLMs with both positive and negative sentiment scores. The x-axis of graphs represented different NLP pipelines. Results were also listed in Table 4.8 for accuracy. Table 4.9 and Table 4.10 for precision, recall and F1-score. Besides, to intuitively compare effects of NLP pipelines, best results obtained by particular techniques within models were highlighted in bold and underlined in tables. The meanings of some abbreviations in figures, tables, and analyses, were the same as those listed at the beginning of the previous section 4.5.

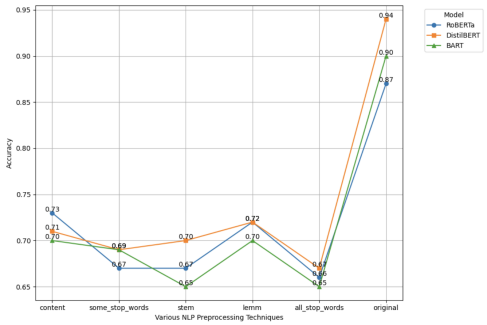


Figure 4.34: Accuracy on Multi-Label Experiments Across Various Pipelines

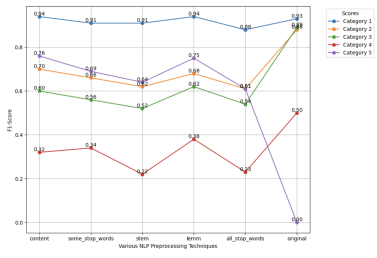


Figure 4.35: RoBERTa F1-Scores on Multi-Label Experiments Across Various Pipelines for Positive Sentiment Scores

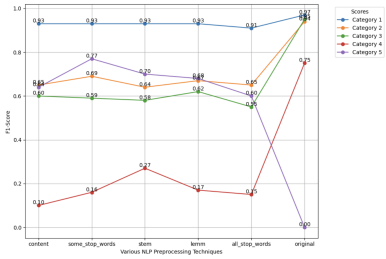


Figure 4.36: DistilBERT F1-Scores on Multi-Label Experiments Across Various Pipelines for Positive Sentiment Scores

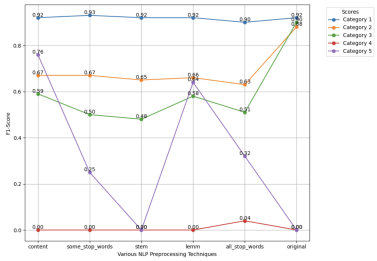


Figure 4.37: BART F1-Scores on Multi-Label Experiments Across Various Pipelines for Positive Sentiment Scores

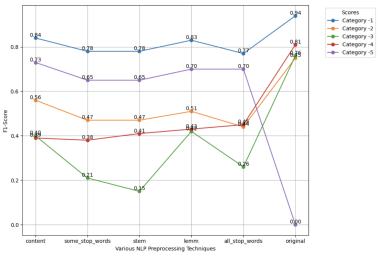


Figure 4.38: RoBERTa F1-Scores on Multi-Label Experiments Across Various Pipelines for Negative Sentiment Scores

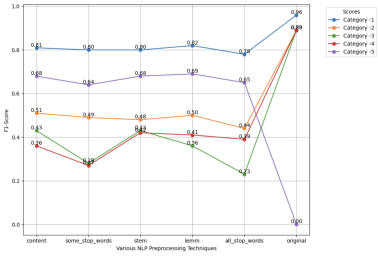


Figure 4.39: DistilBERT F1-Scores on Multi-Label Experiments Across Various Pipelines for Negative Sentiment Scores

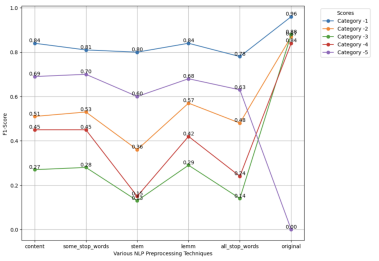


Figure 4.40: BART F1-Scores on Multi-Label Experiments Across Various Pipelines for Negative Sentiment Scores

Table 4.8: Multi-Label Experiments Accuracy Results

		RoBERTa	DistilBERT	BART
content	Accuracy For Positive Sentiment Scores	0.80	0.77	0.77
	Accuracy For Negative Sentiment Scores	0.67	0.64	0.63
	Average Accuracy for Positive and Negative Sentiment Scores	0.73	0.71	0.70
some stopwords	Accuracy For Positive Sentiment Scores	0.76	0.79	0.74
	Accuracy For Negative Sentiment Scores	0.59	0.60	0.64
	Average Accuracy for Positive and Negative Sentiment Scores	0.67	0.69	0.69
stem	Accuracy For Positive Sentiment Scores	0.75	0.77	0.72
	Accuracy For Negative Sentiment Scores	0.60	0.63	0.57
	Average Accuracy for Positive and Negative Sentiment Scores	0.67	0.70	0.65
lemm	Accuracy For Positive Sentiment Scores	0.80	0.79	0.76
	Accuracy For Negative Sentiment Scores	0.65	0.64	0.65
	Average Accuracy for Positive and Negative Sentiment Scores	0.72	0.72	0.70
all stopwords	Accuracy For Positive Sentiment Score	0.73	0.75	0.73
	Accuracy For Negative Sentiment Score	0.59	0.59	0.58
	Average Accuracy for Positive and Negative Sentiment Scores	0.66	0.67	0.65
original	Accuracy For Positive Sentiment Scores	0.90	0.95	0.90
	Accuracy For Negative Sentiment Scores	0.85	0.92	0.91
	Average Accuracy for Positive and Negative Sentiment Scores	0.87	0.94	0.90

Table 4.9: Multi-Label Experiments Results (Precision, Recall, and F1-score) for Positive Sentiment Scores on PLMs

	Socre: 1 of PLMs									Socre: 2 of PLMs									Socre: 3 of PLMs									Socre: 4 of PLMs									Socre: 5 of PLMs								
	RoBERTa			DistilBERT			BART			RoBERTa			DistilBERT			BART			RoBERTa			DistilBERT			BART			RoBERTa			DistilBERT			BART			RoBERTa			DistilBERT			BART		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
content	0.93	**0.95**	**0.94**	**0.91**	0.95	**0.93**	**0.93**	0.91	0.92	**0.67**	**0.73**	**0.70**	0.67	0.63	0.65	**0.66**	0.67	**0.67**	**0.60**	0.61	0.60	0.52	0.69	0.60	**0.49**	**0.73**	**0.59**	**0.63**	0.21	0.32	0.23	0.06	0.10	0.00	0.00	0.00	0.73	**0.80**	**0.76**	**0.79**	0.54	0.64	**0.79**	**0.74**	**0.76**
some stopwords	0.91	0.91	0.91	**0.91**	0.95	**0.93**	0.91	**0.95**	**0.93**	0.66	0.67	0.66	0.73	**0.65**	**0.69**	0.65	0.69	**0.67**	0.54	0.58	0.56	0.53	0.66	0.59	0.42	0.61	0.50	0.39	**0.30**	0.34	0.31	0.11	0.16	0.00	0.00	0.00	0.69	0.68	0.69	0.76	**0.78**	**0.77**	0.57	0.16	0.25
stem	0.89	0.93	0.91	**0.91**	0.94	**0.93**	0.92	0.92	0.92	0.64	0.60	0.62	0.63	**0.65**	0.64	0.59	**0.71**	0.65	0.52	0.53	0.52	**0.55**	0.61	0.58	0.39	0.62	0.48	0.32	0.17	0.22	**0.45**	**0.19**	**0.27**	0.00	0.00	0.00	0.60	0.68	0.64	0.72	0.68	0.70	0.00	0.00	0.00
lemm	**0.94**	0.93	**0.94**	**0.91**	0.95	**0.93**	0.92	0.92	0.92	**0.67**	0.69	0.68	0.72	0.63	0.67	0.65	0.66	0.66	0.58	**0.68**	**0.62**	0.54	**0.75**	**0.62**	**0.49**	0.69	0.58	0.54	**0.30**	**0.38**	0.38	0.11	0.17	0.00	0.00	0.00	0.75	0.76	0.75	**0.79**	0.60	0.68	0.66	0.62	0.64
all stopwords	0.88	0.89	0.88	0.87	**0.96**	0.91	0.89	0.92	0.90	0.59	0.64	0.61	**0.74**	0.57	0.65	0.60	0.65	0.63	0.49	0.61	0.54	0.49	0.64	0.55	0.44	0.63	0.51	0.33	0.17	0.23	0.24	0.11	0.15	1.00	0.02	0.04	**0.83**	0.48	0.61	0.74	0.50	0.60	0.61	0.22	0.32
original	0.93	0.94	0.93	0.95	0.98	0.97	0.94	0.91	0.92	0.90	0.86	0.88	0.95	0.94	0.94	0.86	0.90	0.88	0.85	0.93	0.89	0.95	0.94	0.95	0.87	0.94	0.90	0.57	0.44	0.50	0.86	0.67	0.75	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

Table 4.10: Multi-Label Experiments Results (Precision, Recall, and F1-score) for Negative Sentiment Scores on PLMs

	Socre: -1 of PLMs									Socre: -2 of PLMs									Socre: -3 of PLMs									Socre: -4 of PLMs									Socre: -5 of PLMs								
	RoBERTa			DistilBERT			BART			RoBERTa			DistilBERT			BART			RoBERTa			DistilBERT			BART			RoBERTa			DistilBERT			BART			RoBERTa			DistilBERT			BART		
	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1	P	R	F1
content	**0.85**	0.84	**0.84**	**0.82**	0.81	0.81	0.86	0.81	**0.84**	**0.52**	**0.62**	**0.56**	0.45	**0.59**	**0.51**	0.48	0.54	0.51	**0.47**	0.36	0.40	**0.47**	0.39	**0.43**	0.32	**0.24**	0.27	**0.40**	0.38	0.39	0.41	0.32	0.36	0.40	**0.52**	**0.45**	0.73	**0.73**	**0.73**	0.71	0.66	0.68	**0.71**	0.67	0.69
some stopwords	0.78	0.79	0.78	0.80	0.80	0.80	0.82	0.80	0.81	0.45	0.49	0.47	0.49	0.49	0.49	0.50	0.57	0.53	0.31	0.15	0.21	0.32	0.25	0.28	**0.40**	0.21	0.28	0.32	0.46	0.38	0.30	0.24	0.27	**0.41**	0.49	**0.45**	0.67	0.63	0.65	0.57	**0.74**	0.64	0.65	0.75	**0.70**
stem	0.73	**0.85**	0.78	0.78	0.82	0.80	0.74	**0.87**	0.80	0.48	0.45	0.47	0.45	0.51	0.48	0.43	0.30	0.36	0.26	0.11	0.15	0.44	**0.43**	**0.43**	**0.40**	0.08	0.13	0.35	0.50	0.41	**0.46**	0.38	**0.42**	0.22	0.12	0.15	0.70	0.61	0.65	**0.75**	0.62	0.68	0.46	**0.88**	0.60
lemm	0.84	0.82	0.83	0.78	**0.88**	**0.82**	**0.88**	0.81	**0.84**	0.49	0.54	0.51	**0.50**	0.49	0.50	**0.53**	**0.62**	**0.57**	0.45	**0.39**	**0.42**	0.44	0.31	0.36	0.38	0.23	**0.29**	0.39	0.46	0.43	0.42	0.40	0.41	0.38	0.49	0.42	0.74	0.66	0.70	0.70	0.69	**0.69**	0.65	0.72	0.68
all stopwords	0.77	0.77	0.77	0.76	0.80	0.78	0.77	0.79	0.78	0.44	0.44	0.44	0.46	0.43	0.44	0.45	0.51	0.48	0.32	0.22	0.26	0.32	0.18	0.23	0.29	0.10	0.14	0.36	**0.61**	**0.45**	0.33	**0.49**	0.39	0.31	0.19	0.24	**0.81**	0.61	0.70	0.66	0.65	0.65	0.51	0.82	0.63
original	0.94	0.94	0.94	0.96	0.97	0.96	0.97	0.96	0.96	0.77	0.74	0.75	0.87	0.91	0.89	0.84	0.90	0.87	0.73	0.79	0.76	0.90	0.89	0.89	0.91	0.84	0.88	0.78	0.84	0.81	0.93	0.85	0.89	0.82	0.87	0.84	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

4.6.1 Quantitative Analysis

For **accuracy** of PLMs as shown in Figure 4.34, on inspected reviews, RoBERTa performed best at 0.73, while DistilBERT and BART for 0.71 and 0.70. Notably, their performance on uninspected data was satisfying, with DistilBERT even reaching 0.94. Regarding different NLP pipelines, removing all stopwords had the negative impact on them, for example, RoBERTa dropped from 0.73 to 0.66. Removing some stopwords also had the considerable negative effect on RoBERTa, going from 0.73 to 0.67, but had almost no impact on BART and DistilBERT. Stemming negatively affected both RoBERTa and BART, dropping from 0.73 to 0.67 and from 0.70 to 0.65, respectively. Lemmatization slightly improved DistilBERT from 0.71 to 0.72.

Regarding **F1-scores** for each kind of sentiment score of figures as shown in Report 4.6. For **positive scores**, RoBERTa had same performance as DistilBERT at score "+3" with 0.60, and surpassed DistilBERT on remaining positive ones, such as score "+5" where RoBERTa gained 0.76 compared to DistilBERT's 0.64. RoBERTa slightly outperformed BART on "+1", "+2", and "+3", for example, RoBERTa got 0.94 on "+1" while BART got 0.92. They had identical performance at "+5" with 0.76. Moreover, BART failed to make predictions for "+4", deriving zero F1-score. Additionally, for **uninspected data**, apart from RoBERTa's "+1" dropping from 0.94 to 0.93 and BART's "+4" remaining zero, rest results of PLMs on uninspected reviews showed better or equal performance than inspected ones.

For different **NLP pipelines** shown in Table 4.9, reviews dropped some stopwords and reviews applied lemmatization probably performed better than inspected ones, on 7 and 6 cases in metrics (Precision, Recall, and F1-score) respectively across all PLMs. Stemming might improve 5 cases in metrics for DistilBERT and "+2" recall for BART from 0.67 to 0.71.

For **F1-scores of negative sentiment scores**, similar to previous observations, RoBERTa outperformed DistilBERT in almost all "scores". For instance, RoBERTa gained 0.84 on "-1" while DistilBERT gained 0.81. However, on score "-3", DistilBERT surpassed RoBERTa with 0.43 compared to 0.40. BART and RoBERTa performed consistently on "-1". On score "-4", BART's 0.45 exceeded RoBERTa's 0.39, but in other aspects, it failed to surpass RoBERTa. For example, BART gained 0.69 on "-5" and RoBERTa gained 0.73. For **uninspected reviews**, results of PLMs on uninspected data at "-1", "-2", "-3", and "-4" all exhibited higher F1-scores than inspcected ones.

For different **NLP pipelines** shown in Table 4.10, lemmatization improved about 11 cases in metrics (Precision, Recall, and F1-score) across PLMs, while stemming improved 8 cases. The operations on dropping all or some stopwords were not as effective as before, with only 4 cases improved, respectively.

It was also important to claim all PLMs got zero F1-score for "+5" and "-5" on even smoothed uninspected reviews. Because, as previously mentioned in Report 4.2 shown in Figure 4.9 and Figure 4.10, quantities of samples with +5 or -5 were extremely rare.

4.6.2 Qualitative Analysis

In multi-label experiments, PLMs replicated phenomena observed in multi-class tasks on inspected reviews, also retaining predictive capacities for noisy uninspected reviews mentioned in Report 4.5. RoBERTa surpassed DistilBERT and BART for almost all positive and negative scores, highlighting its standard frameworks in complex tasks, where DistilBERT showed limitations. A possible reason could be DistilBERT was unable to transfer all essential language knowledge during pre-training, to smaller distilled models. Review representations were not effectively analyzed, with less competitiveness on complex tasks.

Interestingly, BART completely failed to predict "+4", also on uninspected data, contrasting with RoBERTa and DistilBERT. A likely reason probably was their distinct architectures; RoBERTa was the "encoder-only" structure, thoroughly learning the corpus bidirectionally as shown in Figure 4.41, from "BERT-style denoising" mentioned in Report 2.1.5. Although BART also employed that in its encoders, it was still based on "encoder-decoder" architectures, with knowledge flowing through the "auto-regressive" framework where a value was only derived from the preceding one [113] when training, shown in Figure 4.42. Moreover, "[CLS]" of BART used for classifications was also extracted from there, while RoBERTa and DistilBERT extracted "[CLS]" features through stacked encoders. Based on zero F1-score of BART for "+4", it could be argued that the performance of "encoder-decoder" architectures might indeed not match that of "encoder-only" ones, even with similar "BERT-style denoising" pre-training tasks, on such complex RE assignments.

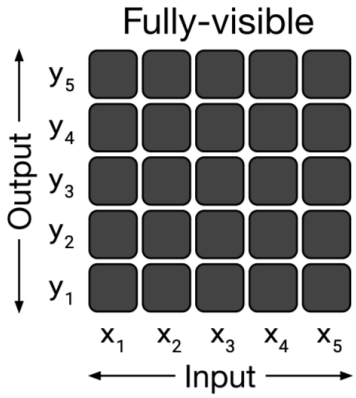


Figure 4.41: Linguistic Knowledge Flow in Encoder Framework [88]

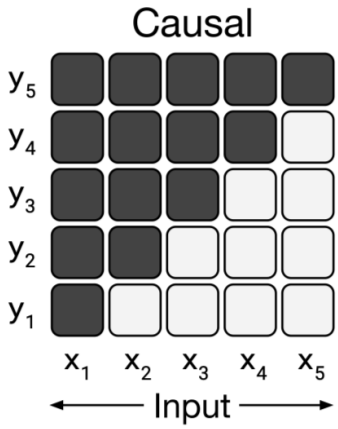


Figure 4.42: Linguistic Knowledge Flow in Auto-Regressive Framework[88]

Additionally, PLMs exhibited lower sensitivity to different NLP pipelines for accuracy given their extensive training corpora including knowledge of word forms. However, for detailed examinations of metrics, lemmatization was relatively acceptable in complex tasks. It reduced data dimensions but following semantics rather than damaging linguistic structures, affecting SA, such as dropping stopwords and stemming. Therefore, NLP pipelines remained promising for detailed metrics, such as precision and F1-score.

4.7 Experiment Running Time with Accuracy

Different running time of PLMs and traditional ML models with accuracy, on multi-class experiments, were listed in Table 4.11.

Table 4.11: Running Time with Accuracy of PLMs and Traditional ML Models

	RoBERTa	DistilBERT	BART	KNN	NB	SVM
GridSearchCV Time for Hyperparameters	\	\	\	79s	0.9s	184s
Running Time on Test Data	\	\	\	5s	3s	31s
Average Time per Epoch for PLMs	17.10s	8.65s	20.10s	\	\	\
Total Time for the Multi-Class Experiment	8min 37s	4min 42s	10min 12s	1min 24s	3.9s	3min 35s
Accuracy	0.81	0.80	0.81	0.67	0.63	0.72

4.7.1 Quantitative Analysis

As shown in Table 4.11, the time for searching benchmark hyper-parameters was not a meaningful reference due to different number of CV experiments. However, their efficiency could be judged based on runtime on test data. NB required only 3 seconds to output, though it had poor accuracy 0.63. KNN and SVM, with more time: 5s and 31s, gained improvements in accuracy, reaching 0.67 and 0.72, respectively. Furthermore, among PLMs, DistilBERT took about only half time of RoBERTa or BART to achieve excellent results with only 0.01 less in accuracy, and each epoch also faster at least twice.

4.7.2 Qualitative Analysis

KNN and NB were relatively simple algorithms, with KNN calculating distances and NB using independent assumptions for rapid predictions, potentially missing complex patterns. SVM performed high-dimensional mapping with nonlinear kernels to optimally separate categories, and costly computation processes yielded better results. BART ran slower than RoBERTa as it fine-tuned both encoders and decoders. The decoder probably had extra complexity and time updating parameters. DistilBERT in particular, stood out by reducing model size but achieved comparable performance to RoBERTa, who had more parameters [71], demonstrating the efficiency of distillation techniques on RE fine-tuning deployment.

5 Research Evaluation

This chapter evaluates results with related factors such as GPU configurations in 5.1 and hyper-parameters in 5.2. A critical comparison with state-of-the-art is in 5.3. Validity threats 5.4 and main research findings 5.5 are also summarized, with some reflections in 5.6.

5.1 Influences of GPU Configurations

Shown in Table 5.1, runtime of multi-class experiments (RoBERTa with epoch5 and batch8) decreased as the number of GPU increased, confirming successes of deploying multi-GPU environments. Runtime of multi-class tasks (RoBERTa with epoch5 and batch16) by different kinds of **FOUR GPUs** was listed in Table 5.2, as potential references for future students.

Table 5.1: Running Time for Different Numbers of GPU

	2 V100	4 V100	6 V100
Average Time per Epoch	1min 01s	52.20s	38.00s
Total Time	6min 23s	5min 38s	4min 28s

Table 5.2: Running Time for Different Kinds of GPU

	4 RTX 3090	4 Tesla V100	4 A4000	4 RTX 3080	4 RTX 3060
Average Time per Epoch	19s	25.2s	28.2s	22.2s	36.2s
Total Time	2min 27s	3min 21s	3min 40s	3min 7s	4min 29s

5.2 Influences of Hyperparameters

5.2.1 Traditional Machine Learning Models

Hyper-parameters of traditional ML models after CV:

- KNN: Euclidean distance, the number of neighbors 33, and distance optimized weighting.
- NB: Laplace constant 0.06.
- SVM: RBF kernel and penalty coefficient 5.

For multi-class experiments of benchmarks, KNN selected 33, a large number, as the neighbor count. This might indicate boundaries between categories in datasets were quite ambiguous, requiring many neighbors to analyze. Euclidean distance with weighted optimization were also selected. It meant closer neighbors, expressing similar features to test data points, had the greater impact on classifications.

NB had altered smoothing coefficients from default 0 to 0.06. This implied test data included some data points not seen during training, but that quantity was relatively small. The training data had covered a lot of the vocabulary appeared in test data.

SVM chose 5 as a moderate penalty size, suggesting boundaries between categories were clear in specific feature spaces. RBF was also selcted as kernel, indicating boundaries were nonlinear in **the original** feature space, utilizing RBF for nonlinear mapping to find better classification hyperplanes.

5.2.2 Pretrained Language Models

Hyper-parameters of PLMs both on multi-class and multi-label experiments, were listed in Table 5.3. Batch meant how many reviews were fed into models once. Max Length meant the maximum length to tokens of input sequences after tokenization (not the number of words). Freeze Layers meant stopping weight parameter updates in lower layers of PLMs. Learning Rate, Dropout, and Weight Decay were mentioned in Report 2.3.5 and 2.3.6.

This project also evaluated different effects of hyper-parameters on results. The baseline task started with learning rate 1e-5, weight decay 1e-5, dropout 0.1, max_length 512, and included “blank noising” augmentation,

Table 5.3: Hyperparameters of All PLMs

Batch per GPU	Learning Rate	Dropout	Weight Decay	Max Length	Freeze Layers
16	1e-5	0	1e-8	256	No

executing multi-class tasks with RoBERTa on inspected data. The criterion was the "variation" of accuracy over 20 epochs. **Steps** on the x-axis represented `backward()` updating weight parameters was called once.

Firstly, batch 8, 16, and 32 were tested, shown in Figure 5.1 and Figure 5.2 with Legend 5.3. When batch was 32, it caused a crash due to insufficient memory. When fixing the x-axis to absolute steps shown in Figure 5.1, batch 16 converged faster than batch 8 and exhibited better performance indicating the model could learn more from enough reviews during each updating parameters. When setting the x-axis to relative steps as shown in Figure 5.2, although batch 8 initially performed well, it was not stable. In contrast, batch 16 maintained robust performance for a while.

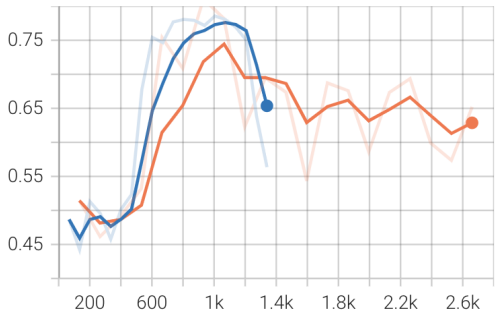


Figure 5.1: Different Batch Performance for Absolute Step

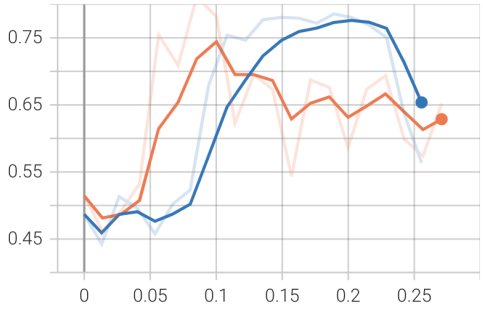


Figure 5.2: Different Batch Performance for Relative Step

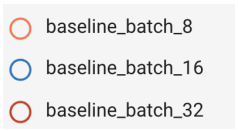


Figure 5.3: Batch Legend

Batch was set as 16. Then, learning rates 1e-5, 2e-5, 3e-5, 9e-5, 1e-4, 3e-4, 9e-4 were tested, shown in Figure 5.4 with Legend 5.5. It was found that smaller learning rates performed better (1e-5, 2e-5, and 3e-5), while larger ones such as 1e-4, derived unsatisfying results. Both 2e-5 and 3e-5 still showed upward trends after 20 epochs, whereas 1e-5 demonstrated stable performance, with relatively smooth convergence and rapid progress. These learning rates were inspired by [53], starting with a relatively small digit 1e-5 and attempting larger ones. Since PLMs had been extensively trained language representations on pre-training assignments, smaller learning rates should be employed [71].

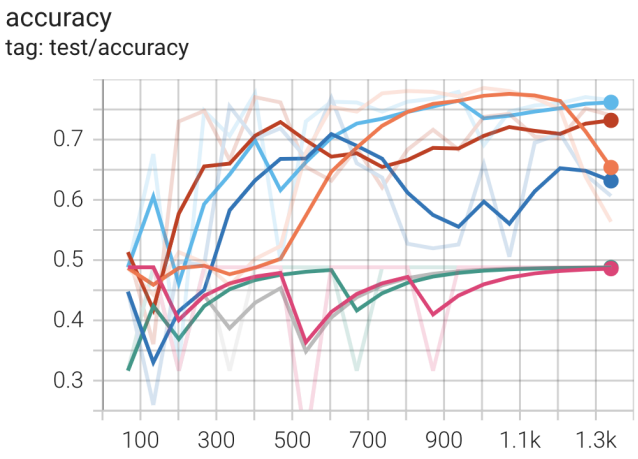


Figure 5.4: Different Learning Rate Performance

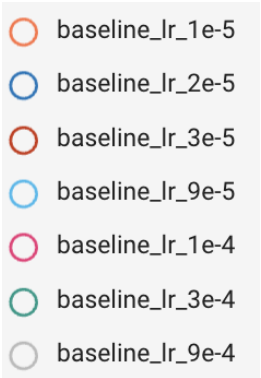


Figure 5.5: Learning Rate Legend

Learning rate was set as 1e-5. Then, dropout 0.0, 0.1, 0.3, 0.5 were tested, shown in Figure 5.6 with Legend 5.7. The absence of dropout leading to better performance suggested the current state might not require the certain regularization to prevent overfitting. Then, weight decay 1e-3, 1e-4, 1e-5, 1e-6, 1e-7, 1e-8 were tested,

shown in Figure 5.8 with Legend 5.9. The performance of low weight decay 1e-8 also confirmed that.

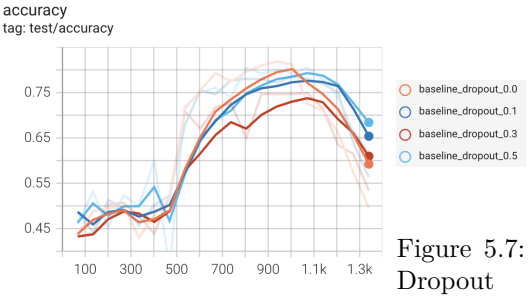


Figure 5.6: Different Dropout Performance

Figure 5.7: Dropout Legend

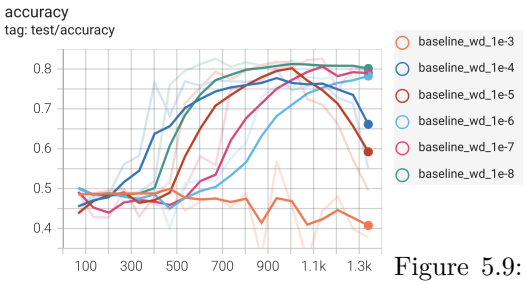


Figure 5.8: Different Weight Decay Performance

Figure 5.9: Weight Decay Legend

Weight Decay was set as 1e-8 without dropout. Then, max_length 256 and 512 were tested, shown in Figure 5.10 with Legend 5.11, and their running time was listed in Table 5.4. 256 max_length doubled overall speeds for running programs and each epoch as shown in Table 5.4. It was reasonable to halve max_length from 512 to 256 to accelerate model convergence, preventing memory overflow [35], given the average length of reviews across various categories was below 50, claimed in Report 4.2 with Table 4.2.

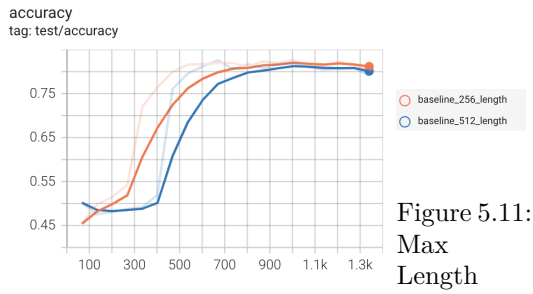


Figure 5.10: Different Max Length Performance

Figure 5.11: Max Length Legend

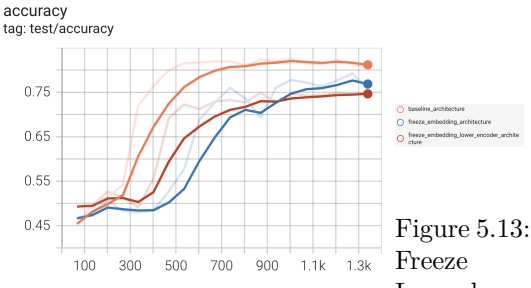


Figure 5.12: Different Frozen Layer Performance

Figure 5.13: Freeze Legend

Table 5.4: Running Time for Different max_length

	max_length 256	max_length 512
Average Time per Epoch	17.05s	35.95s
Total Running Time	8min 37s	16min 33s

Besides, as shown in Figure 5.12 with Legend 5.13, allowing models to update all parameters derived better performance, compared to freezing lower layers or the embedding. It also proved the model was less likely to overfit in this project.

5.3 Comparison with Related Work

A comparison about **F1-Scores** of models with the state-of-art research [1] was explored for multi-class experiments as shown in Table 5.5. Both studies were based on SA of app requirements and both had manually inspected labelling, also with imbalanced neutral instances. It presented an interesting phenomenon their models outperformed this project in positive and negative categories, while some of my models consistently exceeded theirs in the neutral category across benchmarks and PLMs. Two aspects were summarized for critical thinking:

1. Data: their datasets narrowly focused on reviews of virus tracking devices, offering homogeneous positive and negative emotion expressions. This project considered broader fields, from gaming to social media, constructing heterogeneous linguistic features. And the difficulty of predictions for both benchmarks and PLMs was greatly increased, given varying sentiment expressive variations [10]. Therefore, their results on positive and negative categories were more satisfying.

2. Imbalance: neutral reviews in the narrow domain probably limited their models' capacities, whereas this project was exposed diverse features and expressions for neutral reviews across areas, hugely enhancing generalization performance.

Table 5.5: F1-Scores Compared with State-of-Art Research

		Sentiment Categories		
		positive	*neutral*	*negative*
NB	This Project	0.75	**0.33**	0.76
	Ahmad	0.89	0.05	0.89
SVM	This Project	0.79	**0.38**	0.79
	Ahmad	0.88	0.26	0.90
RoBERTa	This Project	0.93	**0.55**	0.89
	Ahmad	0.93	0.32	0.93
DistilBERT	This Project	0.87	**0.48**	0.88
	Ahmad	0.925	0.336	0.923

5.4 Validity Threats

5.4.1 Construct Validity

1. The entire project strictly adhered scientific frameworks in most researches, employing not only accuracy but also precision, recall, and F1-score to examine each category or label, selecting F1-score as the primary metric to mitigate the complementation of precision and recall.

2. Fixed seeds were set for data splitting such as randomly selecting 8000 reviews and model training such as distributing reviews to multiple GPUs, to strongly guarantee the project reproducibility. However, during data cleaning, results after `langdetect` API were not deterministic, probably affecting that.

3. Details such as data augmentation approaches and datasets were shared in github [21] to encourage broader deployments or critiques.

4. Both benchmarks and PLMs employed validation to prevent overfitting, yet PLMs did not undergo CV due to computational resource constraints.

5.4.2 Internal Validity

1. The dataset source was highly recognized by Kaggle, which served as data sources for numerous researches.

2. To filter unrepresentative reviews, an annotation guideline was created for this to reduce the subjective bias.

3. For manually checking, since neither five students nor author were true developers, our perspectives might diverge from the industry. Nonetheless, an annotation guideline was established to reflect the authors' thinking. Subsequent Kappa calculations indicated a degree of broad acceptance, despite each annotator being responsible for a randomly selected portion.

4. All utilized APIs and PLMs were open-source and website links were added.

5.4.3 External Validity

1. Regarding application sources of reviews, potential differences between paid and free products [28, 63, 61], along with different linguistic features across various stores such as Amazon and Apple stores [3, 24], were not provided more attention. Admittedly, results could not fully represent the whole status of comment sections. However, after the preprocessing and manual inspection, the final dataset could be served as corpora with rich linguistic representations for transfer learning in RE. The imbalance within the dataset, probably required possible adjustments in subsequent researches, though a common issue in app review analyses.

[21] https://github.com/zihaoCHEN233/third_year_project

2. This paper employed "Snowballing", but discarding "forward" processes meant it only identified pre-existing publications, potentially introducing biases. For instance, state-of-the-art articles supporting certain viewpoints had to be found through SLR. But some of them were well-known in AI. Additionally, Snowballing primarily occurred through those "literature review articles", which already had some certain constraints to guarantee qualities of references, such as published by organizations like IEEE. ***Therefore, the academic nature of references for this article was highly guaranteed. And this paper could be served as a starting point for subsequent students utilizing "Snowballing".***

5.5 Summary of Key Research Findings

1. After the specific preprocessing, imbalanced category situations were identified. Negative emotions were more abundant and the length of reviews with them tended to be greater.

2. In multi-class classifications, PLMs might not be as sensitive to NLP pipelines as traditional ML. Since they had learned extensive language knowledge from large corpora, including various forms of a word. Excessively applying them probably affected performance. But in complex assignments such as multi-label classifications, techniques such as lemmatization showed promising trends on certain metrics such as F1-score and recall.

3. Encoder-only architectures were indeed more suitable for SA on RE, whereas "encoder-decoder" ones still required further improvements.

4. Distillation represented a novel direction for RE, offering promising insights for implementation and deployment in predicting software reviews.

5. Revealing the synonym data augmentation might be inappropriate for PLMs on SA, causing potential data leakage issues.

During the iterative research, an interesting contradiction emerged in the project: enhancing PLMs through "blank nosing" was in contrast with some NLP pipelines or general preprocessing that aimed to reduce noises for better classifier performance. After critical thinking, this phenomenon could be explained as follows:

- "Blank nosing" should be viewed as the regularization to prevent overfitting after many epochs, as evidenced by poor performance of dropout 0.0 and weight decay 1e-8 without noise, mentioned in Ablation Study 4.4, suggesting "blank nosing" could be a substitute for dropout and weight decay.

- Conversely, pipelines or preprocessing aimed to remove irrelevant elements, like filtering some stopwords or reducing vocabulary features through lemmatization, to enhance data quality before training.

Therefore, this potential contradiction was not an issue but reflected distinct objectives within processes of ML.

5.6 Reflections

During data cleaning, emoticons made of punctuation, such as ">_<" [47] and ":)" were ignored [61], potentially changing emotion distributions during manual checking. Exclamation marks, probably indicating emotions [111, 110], were also overlooked. Additionally, some reviews could be fake or auto-generated [51, 90, 25, 77], undetected during filtering.

6 Conclusion and Future Work

This chapter summarizes key contributions in 6.1, addressing both objectives claimed in 1.2 and concerns stated in 2.5. Additionally, based on validity threats in 5.4 and research findings in 5.5, suggestions for future work are provided in 6.2.

6.1 Achievements and Contributions

The summary of achievements and contributions of this project was listed as below:

1. A high-quality database was created, serving as RE corpora for fine-tuning or even pre-training language models.
2. Offering annotation guidelines for filtering representative reviews and sentiment scoring.
3. Many relevant papers were identified through Snowballing, supporting nearly every aspect sufficiently, allowing this report considered as a starting point for "Snowballing".
4. Two data augmentation methods were implemented: one achieved significant successes, for instance, the accuracy of SVM from 0.72 to 0.89, NB from 0.63 to 0.69, and KNN from 0.67 to 0.74 in benchmarks by generating reviews using synonym dictionaries; the other was introducing noises when fine-tuning PLMs, which could substitute dropout and weight decay techniques.
5. Exploring effects of different NLP pipelines on traditional ML and PLMs on multi-class and multi-label experiments with promising suggestions.
6. Revealing efficiencies of different PLM architectures in SA for RE.
7. Evaluating results under different GPU configurations and different hyper-parameters.
8. Sentiment classifications of app reviews at RE level were conducted by multi-class (positive, neutral, and negative) and granular multi-label experiments, assigning both positive and negative sentiment scores for each review.

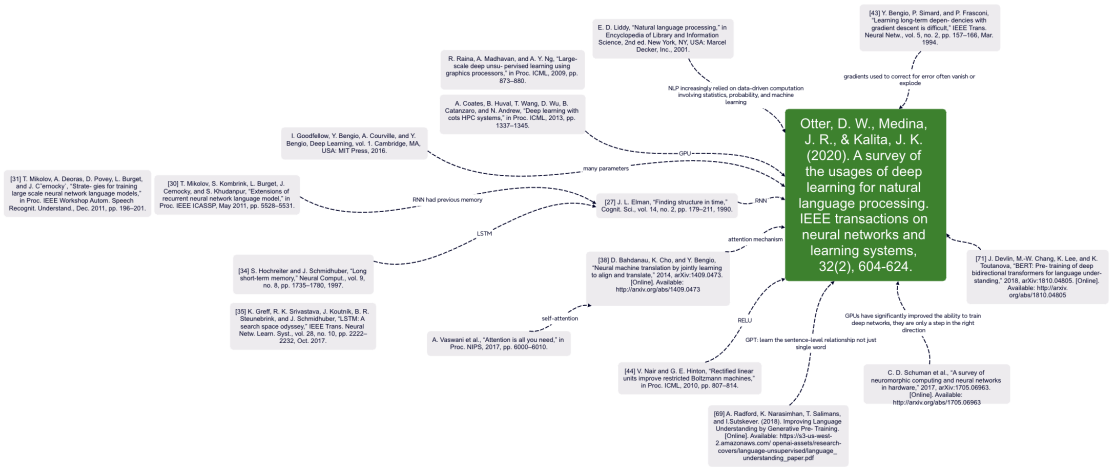
There were also some limitations, such as the substantial computational cost of GPU, which precluded CV for fine-tuning PLMs. And the absence of a suitable database for RE-level SA was also a primary concern for data sources. Furthermore, fine-tuning fixed PLM architectures probably overlooked the metadata [100], such as user information and timestamps [89, 74], indicating emotional patterns.

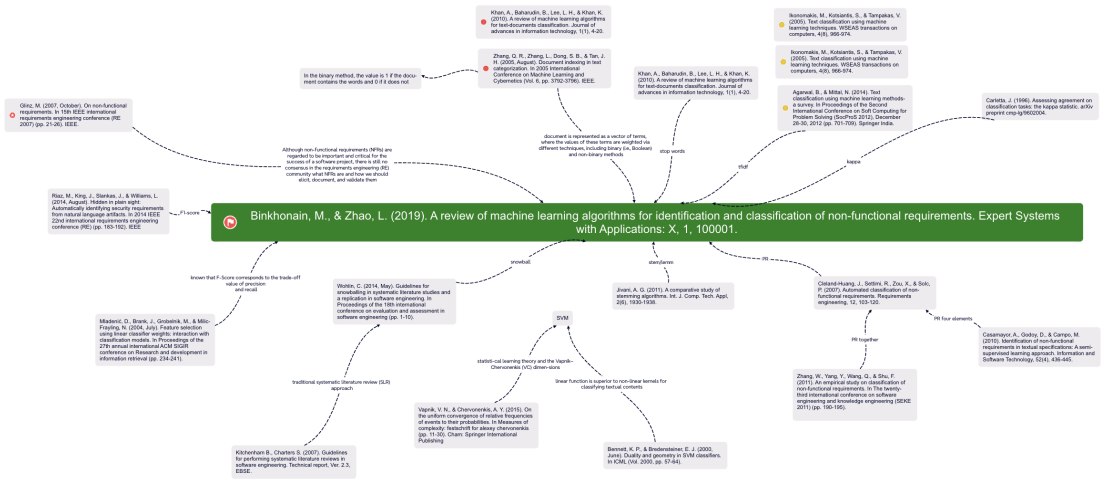
6.2 Suggestions for Future Work

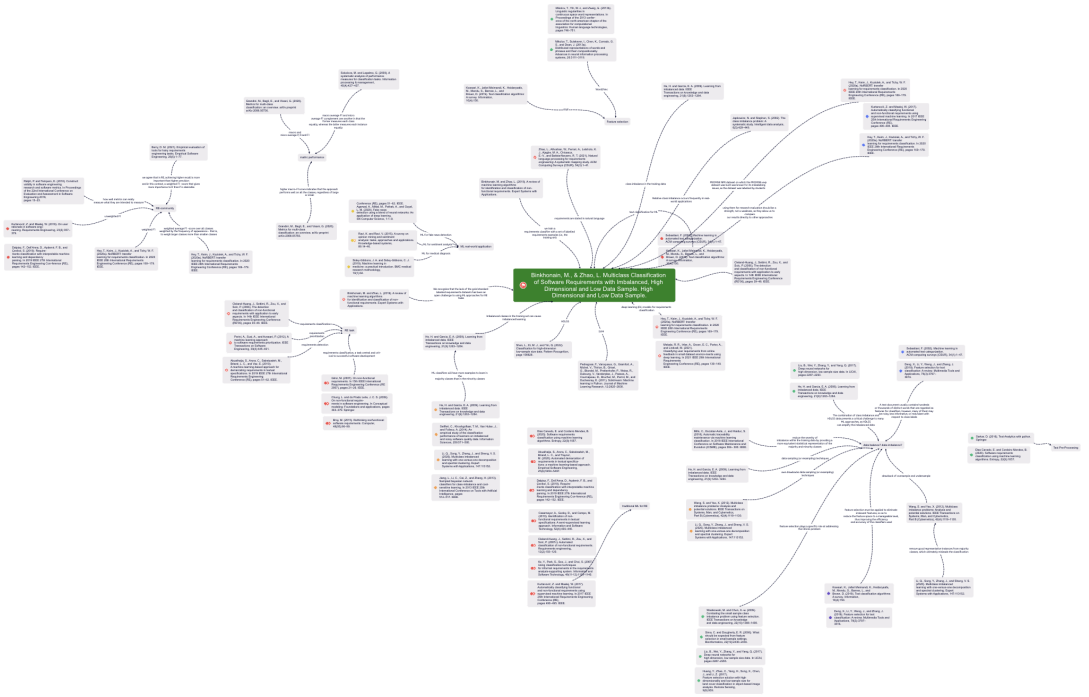
To enhance the future development and to overcome limitations of the current project, several improvements and suggestions could be:

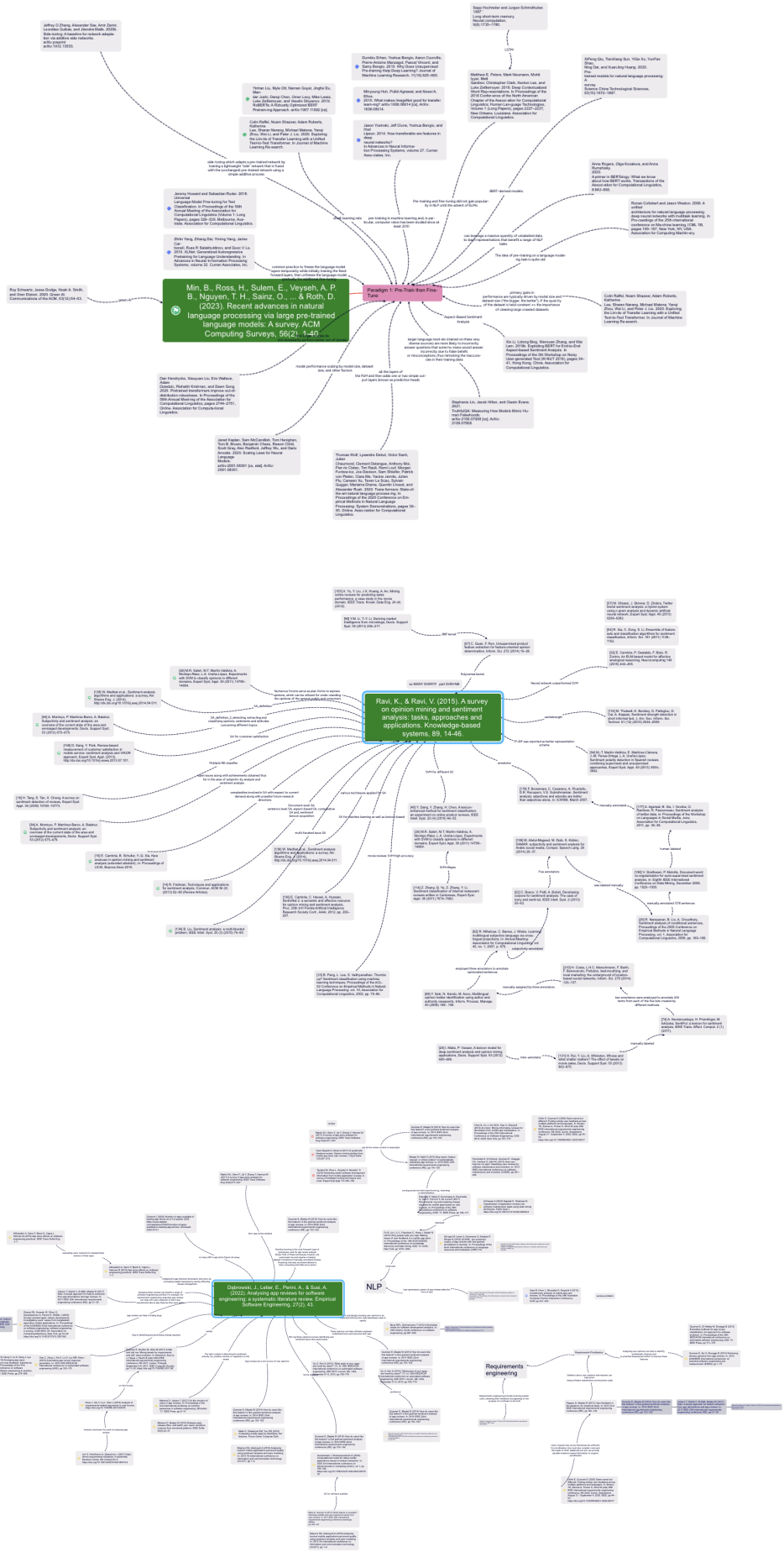
1. Applying ethical approvals and running own web crawlers for data collection like [60, 24], increasing reliability over using secondary data.
2. Developing a simple application [1] for more efficient data labelling.
3. Attempting to combine NLP pipelines instead of using a single one.
4. Advocating related stakeholders to allow users to rate others' comments [55], as directly using user-submitted scores for SA might not be ideal.
5. Encouraging software developers to regularly release minor updates rather than major replacements, deriving more consistent scores, convenient for data analyses [62].
6. Strengthening RE and ML partnerships [10] or keeping study NLP4RE [120]. For example, DistilBERT demonstrated potential capabilities of deployment on edge devices, beneficial to implementing real-time RE analysis [12, 72].
7. Tradeoffs should be considered to select models for RE: "black-box" issues [10] of PLMs significantly reduced interpretability [72], causing the negative impact on the stakeholder decision-making [12].

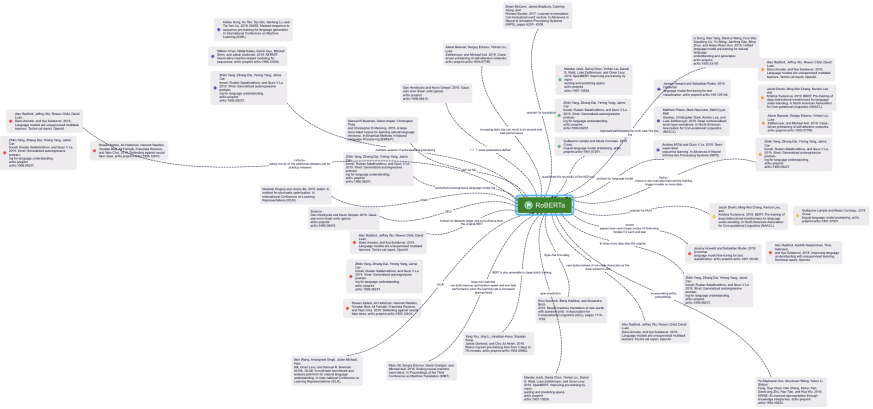
A Snowballing Flowers

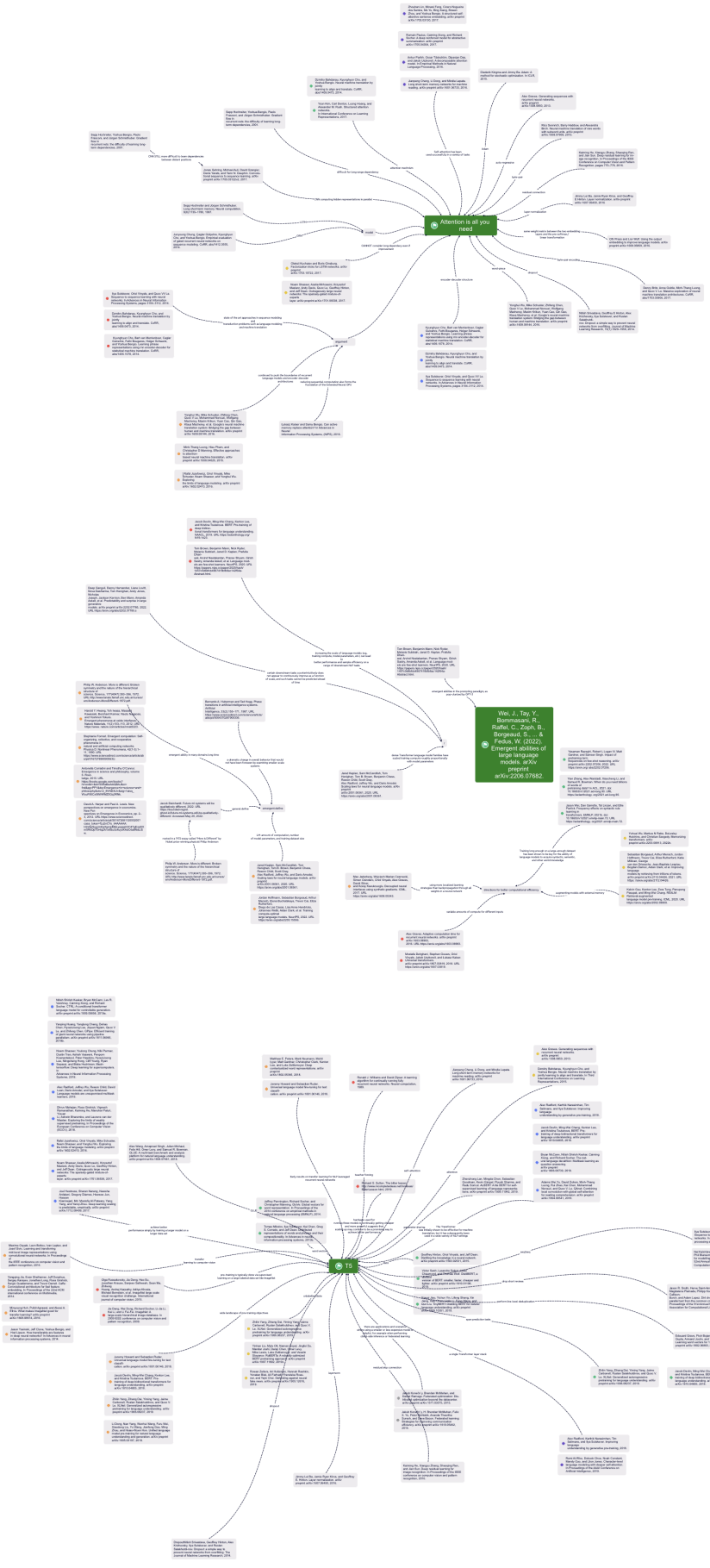


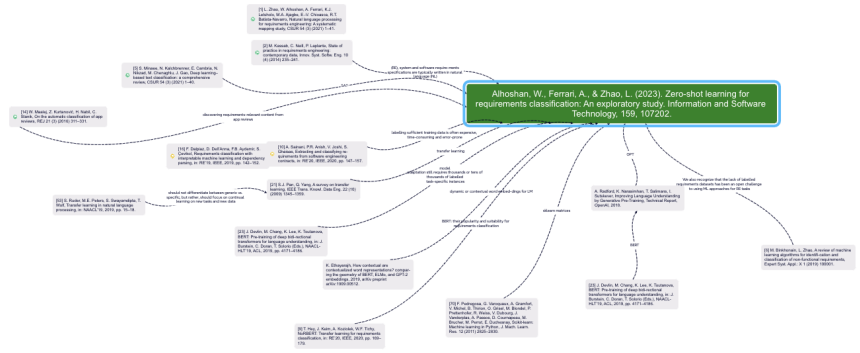


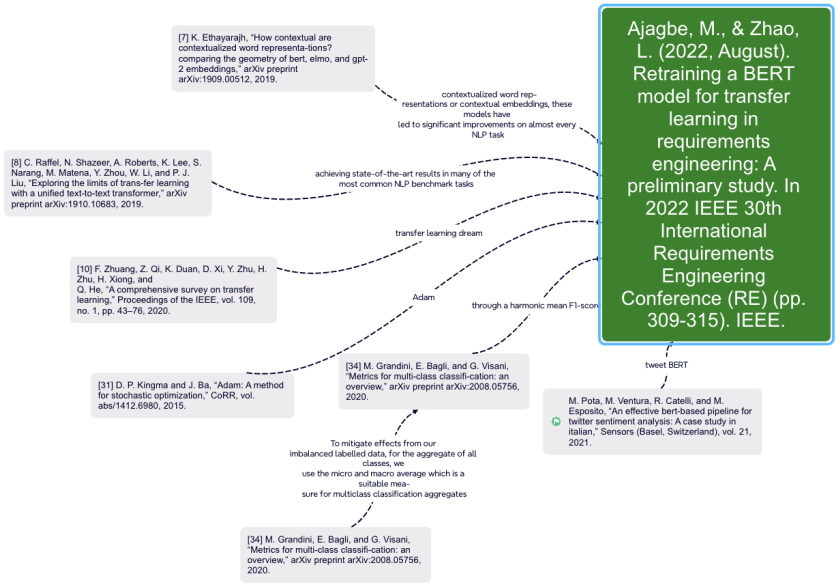


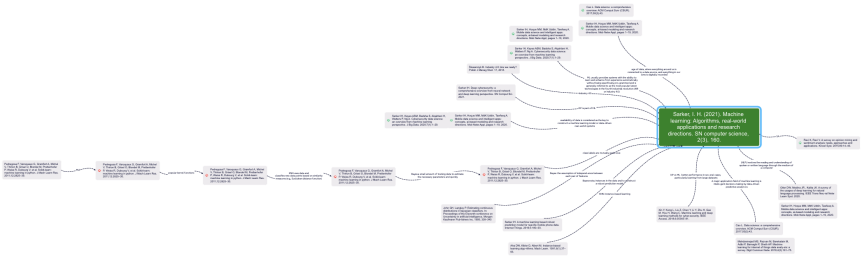

Min, B., Ross, H., Sulem, E., Veyseh, A. P. B., Nguyen, T. H., Sainz, O., ... & Roth, D. (2023). Recent advances in natural language processing via large pre-trained language models: A survey. ACM Computing Surveys, 56(2), 1-40.
Paradigm 1: Pre-Train then Fine-Tune
Ravi, K., & Ravi, V. (2015). A survey on opinion mining and sentiment analysis: tasks, approaches and applications. Knowledge-based systems, 89, 14-46.
Dąbrowski, J., Letier, E., Perini, A., & Susi, A. (2022). Analysing app reviews for software engineering: a systematic literature review. Empirical Software Engineering, 27(2), 43.
NLP
Requirements engineering

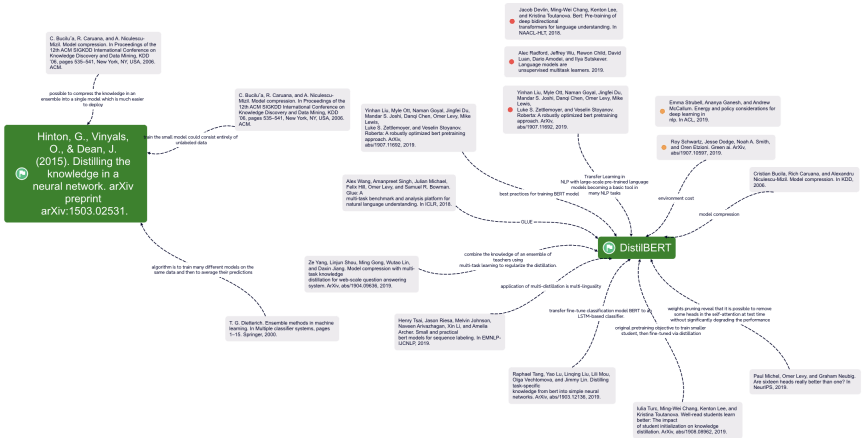

RoBERTa
Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.
Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Ruslan Salakhutdinov, and Quoc V Le. Xlnet: Generalized autoregressive pretraining for language understanding. arXiv preprint arXiv:1906.08237, 2019.
Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. arXiv preprint arXiv:1907.11692, 2019.
Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, pages 4171–4186, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1423. URL https://www.aclweb.org/anthology/N19-1423.
Mandar Joshi, Danqi Chen, Yinhan Liu, Daniel S Weld, Luke Zettlemoyer, and Omer Levy. Spanbert: Improving pre-training by representing and predicting spans. arXiv preprint arXiv:1907.10529, 2019.
Matthew E Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. Deep contextualized word representations. arXiv preprint arXiv:1802.05365, 2018.
Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). arXiv preprint arXiv:1606.08415, 2016.
Li Dong, Nan Yang, Wenhui Wang, Furu Wei, Xiaodong Liu, Yu Wang, Jianfeng Gao, Ming Zhou, and Hsiao-Wuen Hon. Unified language model pre-training for natural language understanding and generation. arXiv preprint arXiv:1905.03197, 2019.
Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. OpenAI Blog, 1(8), 2019.
Sergey Edunov, Alexei Baevski, and Michael Auli. Pre-trained language model representations for language generation. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), 2019.
Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. URL https://s3-us-west-2. amazonaws. com/openai-assets/research-covers/language-unsupervised/language understanding paper. pdf, 2018.
BERT
MLM-variant
BART
Self-supervised methods
large language models can act as unsupervised multitask models
larger batch size
GPT model which is problematic for some tasks
Text infilling inspired by spanBERT
training for longer
masking spans instead of words
tying parameters across layers
masked lan-guage modeling, which allows pre-training to learn in-teractions between left and right context words
concatenates left-only and right-only representations, but does not pre-train interactions between these features
gains by improving the distribution of masked tokens
the order of masked token
available context for replacing masked tokens

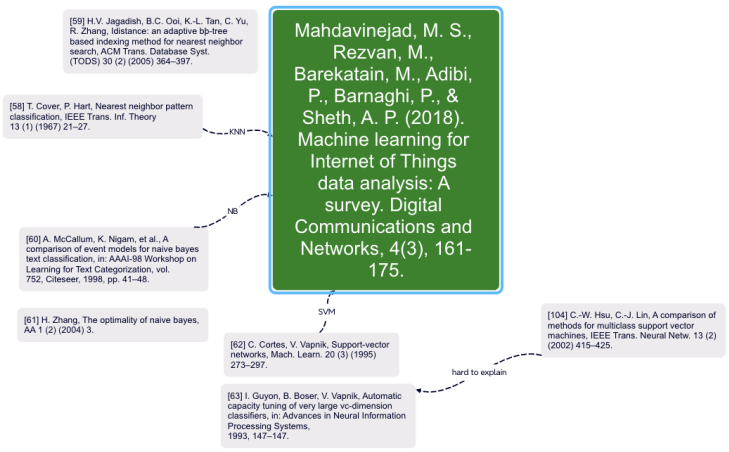

Attention is all you need
T5
Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., ... & Fedus, W. (2022). Emergent abilities of large language models. arXiv preprint arXiv:2206.07682.

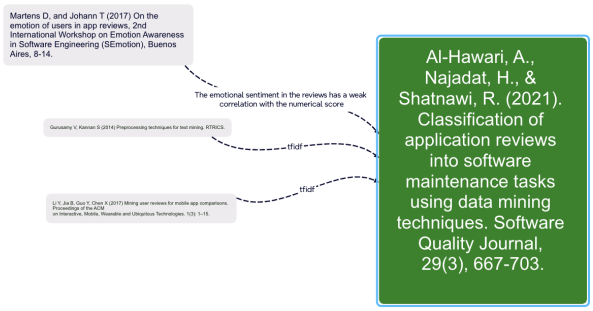

Alhoshan, W., Ferrari, A., & Zhao, L. (2023). Zero-shot learning for requirements classification: An exploratory study. Information and Software Technology, 159, 107202.

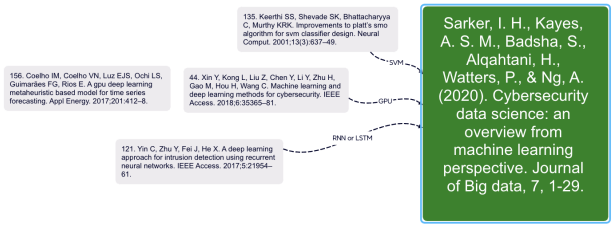

Ajagbe, M., & Zhao, L. (2022, August). Retraining a BERT model for transfer learning in requirements engineering: A preliminary study. In 2022 IEEE 30th International Requirements Engineering Conference (RE) (pp. 309-315). IEEE.
[7] K. Ethayarajh, "How contextual are contextualized word representa-tions? comparing the geometry of bert, elmo, and gpt-2 embeddings," arXiv preprint arXiv:1909.00512, 2019.
contextualized word rep-resentations or contextual embeddings, these models have led to significant improvements on almost every NLP task
[8] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of trans-fer learning with a unified text-to-text transformer," arXiv preprint arXiv:1910.10683, 2019.
achieving state-of-the-art results in many of the most common NLP benchmark tasks
transfer learning dream
[10] F. Zhuang, Z. Qi, K. Duan, D. Xi, Y. Zhu, H. Zhu, H. Xiong, and Q. He, "A comprehensive survey on transfer learning," Proceedings of the IEEE, vol. 109, no. 1, pp. 43–76, 2020.
Adam
through a harmonic mean F1-scor
[31] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," CoRR, vol. abs/1412.6980, 2015.
[34] M. Grandini, E. Bagli, and G. Visani, "Metrics for multi-class classifi-cation: an overview," arXiv preprint arXiv:2008.05756, 2020.
tweet BERT
M. Pota, M. Ventura, R. Catelli, and M. Esposito, "An effective bert-based pipeline for twitter sentiment analysis: A case study in italian," Sensors (Basel, Switzerland), vol. 21, 2021.
To mitigate effects from our imbalanced labelled data, for the aggregate of all classes, we use the micro and macro average which is a suitable mea-sure for multiclass classification aggregates
[34] M. Grandini, E. Bagli, and G. Visani, "Metrics for multi-class classifi-cation: an overview," arXiv preprint arXiv:2008.05756, 2020.

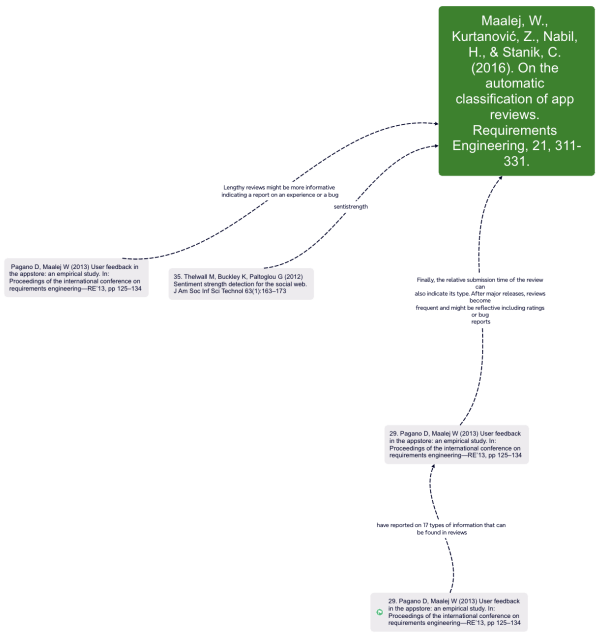

Sarker, I. H. (2021). Machine learning: Algorithms, real-world applications and research directions. SN computer science, 2(3), 160.


Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531.
DistilBERT
C. Buciluǎ, R. Caruana, and A. Niculescu-Mizil. Model compression. In Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '06, pages 535–541, New York, NY, USA, 2006. ACM.
Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In NAACL-HLT, 2019.
Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.
Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar S. Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke S. Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. ArXiv, abs/1907.11692, 2019.
Emma Strubell, Ananya Ganesh, and Andrew McCallum. Energy and policy considerations for deep learning in NLP. In ACL, 2019.
Roy Schwartz, Jesse Dodge, Noah A. Smith, and Oren Etzioni. Green ai. ArXiv, abs/1907.10597, 2019.
Cristian Buciluǎ, Rich Caruana, and Alexandru Niculescu-Mizil. Model compression. In KDD, 2006.
Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. Glue: A multi-task benchmark and analysis platform for natural language understanding. In ICLR, 2018.
Transfer Learning in NLP with large-scale pre-trained language models becoming a basic tool in many NLP tasks
best practices for training BERT model
environment cost
model compression
GLUE
combine the knowledge of an ensemble of teachers using multi-task learning to regularize the distillation
Ze Yang, Linjun Shou, Ming Gong, Wutao Lin, and Daxin Jiang. Model compression with multi-task knowledge distillation for web-scale question answering system. ArXiv, abs/1904.09636, 2019.
application of multi-distillation is multi-linguality
transfer fine-tune classification model BERT to a LSTM-based classifier.
weights pruning reveal that it is possible to remove some heads in the self-attention at test time without significantly degrading the performance
original pretraining objective to train smaller student, then fine-tuned via distillation
Henry Tsai, Jason Riesa, Melvin Johnson, Naveen Arivazhagan, Xin Li, and Amelia Archer. Small and practical bert models for sequence labeling. In EMNLP-IJCNLP, 2019.
Raphael Tang, Yao Lu, Linqing Liu, Lili Mou, Olga Vechtomova, and Jimmy Lin. Distilling task-specific knowledge from bert into simple neural networks. ArXiv, abs/1903.12136, 2019.
Iulia Turc, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Well-read students learn better: The impact of student initialization on knowledge distillation. ArXiv, abs/1908.08962, 2019.
Paul Michel, Omer Levy, and Graham Neubig. Are sixteen heads really better than one? In NeurIPS, 2019.
T. G. Dietterich. Ensemble methods in machine learning. In Multiple classifier systems, pages 1–15. Springer, 2000.


Mahdavinejad, M. S., Rezvan, M., Barekatain, M., Adibi, P., Barnaghi, P., & Sheth, A. P. (2018). Machine learning for Internet of Things data analysis: A survey. Digital Communications and Networks, 4(3), 161-175.
[59] H.V. Jagadish, B.C. Ooi, K.-L. Tan, C. Yu, R. Zhang, Idistance: an adaptive bb-tree based indexing method for nearest neighbor search, ACM Trans. Database Syst. (TODS) 30 (2) (2005) 364–397.
[58] T. Cover, P. Hart, Nearest neighbor pattern classification, IEEE Trans. Inf. Theory 13 (1) (1967) 21–27.
KNN
NB
[60] A. McCallum, K. Nigam, et al., A comparison of event models for naive bayes text classification, in: AAAI-98 Workshop on Learning for Text Categorization, vol. 752, Citeseer, 1998, pp. 41–48.
[61] H. Zhang, The optimality of naive bayes, AA 1 (2) (2004) 3.
SVM
[62] C. Cortes, V. Vapnik, Support-vector networks, Mach. Learn. 20 (3) (1995) 273–297.
hard to explain
[104] C.-W. Hsu, C.-J. Lin, A comparison of methods for multiclass support vector machines, IEEE Trans. Neural Netw. 13 (2) (2002) 415–425.
[63] I. Guyon, B. Boser, V. Vapnik, Automatic capacity tuning of very large vc-dimension classifiers, in: Advances in Neural Information Processing Systems, 1993, 147–147.


Al-Hawari, A., Najadat, H., & Shatnawi, R. (2021). Classification of application reviews into software maintenance tasks using data mining techniques. Software Quality Journal, 29(3), 667-703.
Martens D, and Johann T (2017) On the emotion of users in app reviews, 2nd International Workshop on Emotion Awareness in Software Engineering (SEmotion), Buenos Aires, 8-14.
The emotional sentiment in the reviews has a weak correlation with the numerical score
Gurusamy V, Kannan S (2014) Preprocessing techniques for text mining. RTRICS.
tfidf
L. V, Jia B, Guy Y, Chen X (2017) Mining user reviews for mobile app comparisons. Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies. 1(3): 1–15.
tfidf


Sarker, I. H., Kayes, A. S. M., Badsha, S., Alqahtani, H., Watters, P., & Ng, A. (2020). Cybersecurity data science: an overview from machine learning perspective. Journal of Big data, 7, 1-29.
135. Keerthi SS, Shevade SK, Bhattacharyya C, Murthy KRK. Improvements to platt's smo algorithm for svm classifier design. Neural Comput. 2001;13(3):637–49.
SVM
156. Coelho IM, Coelho VN, Luz EJS, Ochi LS, Guimarães FG, Rios E. A gpu deep learning metaheuristic based model for time series forecasting. Appl Energy. 2017;201:412–8.
44. Xin Y, Kong L, Liu Z, Chen Y, Li Y, Zhu H, Gao M, Hou H, Wang C. Machine learning and deep learning methods for cybersecurity. IEEE Access. 2018;6:35365–81.
GPU
121. Yin C, Zhu Y, Fei J, He X. A deep learning approach for intrusion detection using recurrent neural networks. IEEE Access. 2017;5:21954–61.
RNN or LSTM


Maalej, W., Kurtanović, Z., Nabil, H., & Stanik, C. (2016). On the automatic classification of app reviews. Requirements Engineering, 21, 311-331.
Lengthy reviews might be more informative indicating a report on an experience or a bug
sentistrength
Pagano D, Maalej W (2013) User feedback in the appstore: an empirical study. In: Proceedings of the international conference on requirements engineering—RE'13, pp 125–134
35. Thelwall M, Buckley K, Paltoglou G (2012) Sentiment strength detection for the social web. J Am Soc Inf Sci Technol 63(1):163–173
Finally, the relative submission time of the review can
also indicate its type. After major releases, reviews become
frequent and might be reflective including ratings or bug
reports
29. Pagano D, Maalej W (2013) User feedback in the appstore: an empirical study. In: Proceedings of the international conference on requirements engineering—RE'13, pp 125–134
have reported on 17 types of information that can be found in reviews
29. Pagano D, Maalej W (2013) User feedback in the appstore: an empirical study. In: Proceedings of the international conference on requirements engineering—RE'13, pp 125–134

References

[1] Kashif Ahmad, Firoj Alam, Junaid Qadir, Basheer Qolomany, Imran Khan, Talhat Khan, Muhammad Suleman, Naina Said, Syed Zohaib Hassan, Asma Gul, et al. Sentiment analysis of users' reviews on covid-19 contact tracing apps with a benchmark dataset. *arXiv preprint arXiv:2103.01196*, 2021.

[2] Muideen Ajagbe and Liping Zhao. Retraining a bert model for transfer learning in requirements engineering: A preliminary study. In *2022 IEEE 30th International Requirements Engineering Conference (RE)*, pages 309–315. IEEE, 2022.

[3] Assem Al-Hawari, Hassan Najadat, and Raed Shatnawi. Classification of application reviews into software maintenance tasks using data mining techniques. *Software Quality Journal*, 29(3):667–703, 2021.

[4] Afnan A Al-Subaihin, Federica Sarro, Sue Black, Licia Capra, and Mark Harman. App store effects on software engineering practices. *IEEE Transactions on Software Engineering*, 47(2):300–319, 2019.

[5] Waad Alhoshan, Alessio Ferrari, and Liping Zhao. Zero-shot learning for requirements classification: An exploratory study. *Information and Software Technology*, 159:107202, 2023.

[6] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[7] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014.

[8] Elsa Bakiu and Emitza Guzman. Which feature is unusable? detecting usability and user experience issues from user reviews. In *2017 IEEE 25th international requirements engineering conference workshops (REW)*, pages 182–187. IEEE, 2017.

[9] Abayomi Bello, Sin-Chun Ng, and Man-Fai Leung. A bert framework to sentiment analysis of tweets. *Sensors*, 23(1):506, 2023.

[10] Manal Binkhonain and Liping Zhao. A review of machine learning algorithms for identification and classification of non-functional requirements. *Expert Systems with Applications: X*, 1:100001, 2019.

[11] Manal Binkhonain and Liping Zhao. Multiclass classification of software requirements with imbalanced, high dimensional and low data sample. *High Dimensional and Low Data Sample*, 2023.

[12] Raymond PL Buse and Thomas Zimmermann. Information needs for software development analytics. In *2012 34th International Conference on Software Engineering (ICSE)*, pages 987–996. IEEE, 2012.

[13] Erik Cambria, Björn Schuller, Yunqing Xia, and Catherine Havasi. New avenues in opinion mining and sentiment analysis. *IEEE Intelligent systems*, 28(2):15–21, 2013.

[14] Ning Chen, Jialiu Lin, Steven CH Hoi, Xiaokui Xiao, and Boshen Zhang. Ar-miner: mining informative reviews for developers from mobile app marketplace. In *Proceedings of the 36th international conference on software engineering*, pages 767–778, 2014.

[15] Igor M Coelho, Vitor N Coelho, Eduardo J da S Luz, Luiz S Ochi, Frederico G Guimaraes, and Eyder Rios. A gpu deep learning metaheuristic based model for time series forecasting. *Applied Energy*, 201:412–418, 2017.

[16] Jacek Dąbrowski, Emmanuel Letier, Anna Perini, and Angelo Susi. Analysing app reviews for software engineering: a systematic literature review. *Empirical Software Engineering*, 27(2):43, 2022.

[17] Robert Dale. Gpt-3: What's it good for? *Natural Language Engineering*, 27(1):113–118, 2021.

[18] Yan Dang, Yulei Zhang, and Hsinchun Chen. A lexicon-enhanced method for sentiment classification: An experiment on online product reviews. *IEEE Intelligent Systems*, 25(4):46–53, 2009.

[19] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

[20] Kawin Ethayarajh. How contextual are contextualized word representations? comparing the geometry of bert, elmo, and gpt-2 embeddings. *arXiv preprint arXiv:1909.00512*, 2019.

[21] Pedregosa Fabian. Scikit-learn: Machine learning in python. *Journal of machine learning research 12*, page 2825, 2011.

[22] Ronen Feldman. Techniques and applications for sentiment analysis. *Communications of the ACM*, 56(4):82–89, 2013.

[23] Bin Fu, Jialiu Lin, Lei Li, Christos Faloutsos, Jason Hong, and Norman Sadeh. Why people hate your app: Making sense of user feedback in a mobile app store. In *Proceedings of the 19th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 1276–1284, 2013.

[24] Cuiyun Gao, Jichuan Zeng, Xin Xia, David Lo, Michael R Lyu, and Irwin King. Automating app review response generation. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 163–175. IEEE, 2019.

[25] Necmiye Genc-Nayebi and Alain Abran. A systematic literature review: Opinion mining studies from mobile app store user reviews. *Journal of Systems and Software*, 125:207–219, 2017.

[26] Manoochehr Ghiassi, James Skinner, and David Zimbra. Twitter brand sentiment analysis: A hybrid system using n-gram analysis and dynamic artificial neural network. *Expert Systems with applications*, 40(16):6266–6282, 2013.

[27] Margherita Grandini, Enrico Bagli, and Giorgio Visani. Metrics for multi-class classification: an overview. *arXiv preprint arXiv:2008.05756*, 2020.

[28] Xiaodong Gu and Sunghun Kim. " what parts of your apps are loved by users?"(t). In *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 760–770. IEEE, 2015.

[29] Emitza Guzman, Omar Aly, and Bernd Bruegge. Retrieving diverse opinions from app reviews. In *2015 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pages 1–10. IEEE, 2015.

[30] Emitza Guzman, Muhammad El-Haliby, and Bernd Bruegge. Ensemble methods for app review classification: An approach for software evolution (n). In *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 771–776. IEEE, 2015.

[31] Emitza Guzman, Mohamed Ibrahim, and Martin Glinz. A little bird told me: Mining tweets for requirements and software evolution. In *2017 IEEE 25th International requirements engineering conference (RE)*, pages 11–20. IEEE, 2017.

[32] Emitza Guzman and Walid Maalej. How do users like this feature? a fine grained sentiment analysis of app reviews. In *2014 IEEE 22nd international requirements engineering conference (RE)*, pages 153–162. Ieee, 2014.

[33] Mohammad Abdul Hadi and Fatemeh H Fard. Aobtm: Adaptive online biterm topic modeling for version sensitive short-texts analysis. In *2020 IEEE international conference on software maintenance and evolution (ICSME)*, pages 593–604. IEEE, 2020.

[34] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[35] Tobias Hey, Jan Keim, Anne Koziolek, and Walter F Tichy. Norbert: Transfer learning for requirements classification. In *2020 IEEE 28th international requirements engineering conference (RE)*, pages 169–179. IEEE, 2020.

[36] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

[37] Shu Huang, Wei Peng, Jingxuan Li, and Dongwon Lee. Sentiment and topic analysis on social media: a multi-task multi-label classification approach. In *Proceedings of the 5th annual ACM web science conference*, pages 172–181, 2013.

[38] Muhammad Hussain, Summrina Kanwal Wajid, Ali Elzaart, and Mohammed Berbar. A comparison of svm kernel functions for breast cancer detection. In *2011 eighth international conference computer graphics, imaging and visualization*, pages 145–150. IEEE, 2011.

[39] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. pmlr, 2015.

[40] Timo Johann, Christoph Stanik, Walid Maalej, et al. Safe: A simple approach for feature extraction from app descriptions and app reviews. In *2017 IEEE 25th international requirements engineering conference (RE)*, pages 21–30. IEEE, 2017.

[41] Daekook Kang and Yongtae Park. based measurement of customer satisfaction in mobile service: Sentiment analysis and vikor approach. *Expert Systems with Applications*, 41(4):1041–1050, 2014.

[42] Subbu Kannan, Vairaprakash Gurusamy, S Vijayarani, J Ilamathi, Ms Nithya, S Kannan, and V Gurusamy. Preprocessing techniques for text mining. *International Journal of Computer Science & Communication Networks*, 5(1):7–16, 2014.

[43] Javed Ali Khan, Yuchen Xie, Lin Liu, and Lijie Wen. Analysis of requirements-related arguments in user forums. In *2019 IEEE 27th International Requirements Engineering Conference (RE)*, pages 63–74. IEEE, 2019.

[44] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[45] Vladimír Kunc and Jiří Kléma. Three decades of activations: A comprehensive survey of 400 activation functions for neural networks. *arXiv preprint arXiv:2402.09092*, 2024.

[46] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. *arXiv preprint arXiv:1910.13461*, 2019.

[47] Yung-Ming Li and Tsung-Ying Li. Deriving market intelligence from microblogs. *Decision Support Systems*, 55(1):206–217, 2013.

[48] Wenxiong Liao, Bi Zeng, Xiuwen Yin, and Pengfei Wei. An improved aspect-category sentiment analysis model for text sentiment analysis based on roberta. *Applied Intelligence*, 51:3522–3533, 2021.

[49] Elizabeth D Liddy. Natural language processing. 2001.

[50] Sachiko Lim, Aron Henriksson, and Jelena Zdravkovic. Data-driven requirements elicitation: A systematic literature review. *SN Computer Science*, 2(1):16, 2021.

[51] Bing Liu. Sentiment analysis: A multi-faceted problem. *IEEE intelligent systems*, 25(3):76–80, 2010.

[52] Bing Liu et al. Sentiment analysis and subjectivity. *Handbook of natural language processing*, 2(2010):627–666, 2010.

[53] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

[54] Alexandra Sasha Luccioni, Sylvain Viguier, and Anne-Laure Ligozat. Estimating the carbon footprint of bloom, a 176b parameter language model. *Journal of Machine Learning Research*, 24(253):1–15, 2023.

[55] Walid Maalej, Zijad Kurtanović, Hadeer Nabil, and Christoph Stanik. On the automatic classification of app reviews. *Requirements Engineering*, 21:311–331, 2016.

[56] Walid Maalej and Hadeer Nabil. Bug report, feature request, or simply praise? on automatically classifying app reviews. In *2015 IEEE 23rd international requirements engineering conference (RE)*, pages 116–125. IEEE, 2015.

[57] Walid Maalej, Maleknaz Nayebi, Timo Johann, and Guenther Ruhe. Toward data-driven requirements engineering. *IEEE software*, 33(1):48–54, 2015.

[58] Mohammad Saeid Mahdavinejad, Mohammadreza Rezvan, Mohammadamin Barekatain, Peyman Adibi, Payam Barnaghi, and Amit P Sheth. Machine learning for internet of things data analysis: A survey. *Digital Communications and Networks*, 4(3):161–175, 2018.

[59] Md Shihab Mahmud, Afrin Jaman Bonny, Uchchhwas Saha, Mehrin Jahan, Zannatul Ferdhoush Tuna, and Ahmed Al Marouf. Sentiment analysis from user-generated reviews of ride-sharing mobile applications. In *2022 6th international conference on computing methodologies and communication (ICCMC)*, pages 738–744. IEEE, 2022.

[60] Haroon Malik, Elhadi M Shakshuki, and Wook-Sung Yoo. Comparing mobile apps by identifying 'hot'features. *Future Generation Computer Systems*, 107:659–669, 2020.

[61] Daniel Martens and Timo Johann. On the emotion of users in app reviews. In *2017 IEEE/ACM 2nd International workshop on emotion awareness in software engineering (SEmotion)*, pages 8–14. IEEE, 2017.

[62] Daniel Martens and Walid Maalej. Release early, release often, and watch your users' emotions: Lessons from emotional patterns. *IEEE Software*, 36(5):32–37, 2019.

[63] William Martin, Federica Sarro, Yue Jia, Yuanyuan Zhang, and Mark Harman. A survey of app store analysis for software engineering. *IEEE transactions on software engineering*, 43(9):817–847, 2016.

[64] María-Teresa Martín-Valdivia, Eugenio Martínez-Cámara, Jose-M Perea-Ortega, and L Alfonso Ureña-López. Sentiment polarity detection in spanish reviews combining supervised and unsupervised approaches. *Expert Systems with Applications*, 40(10):3934–3942, 2013.

[65] Nestor Maslej, Loredana Fattorini, Erik Brynjolfsson, John Etchemendy, Katrina Ligett, Terah Lyons, James Manyika, Helen Ngo, Juan Carlos Niebles, Vanessa Parli, et al. Artificial intelligence index report 2023. *arXiv preprint arXiv:2310.03715*, 2023.

[66] Riefvan Achmad Masrury, Andry Alamsyah, et al. Analyzing tourism mobile applications perceived quality using sentiment analysis and topic modeling. In *2019 7th international conference on information and communication technology (ICoICT)*, pages 1–6. IEEE, 2019.

[67] Walaa Medhat, Ahmed Hassan, and Hoda Korashy. Sentiment analysis algorithms and applications: A survey. *Ain Shams engineering journal*, 5(4):1093–1113, 2014.

[68] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

[69] Tomáš Mikolov, Anoop Deoras, Daniel Povey, Lukáš Burget, and Jan Černockỳ. Strategies for training large scale neural network language models. In *2011 IEEE Workshop on Automatic Speech Recognition & Understanding*, pages 196–201. IEEE, 2011.

[70] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. Distributed representations of words and phrases and their compositionality. *Advances in neural information processing systems*, 26, 2013.

[71] Bonan Min, Hayley Ross, Elior Sulem, Amir Pouran Ben Veyseh, Thien Huu Nguyen, Oscar Sainz, Eneko Agirre, Ilana Heintz, and Dan Roth. Recent advances in natural language processing via large pre-trained language models: A survey. *ACM Computing Surveys*, 56(2):1–40, 2023.

[72] Shervin Minaee, Nal Kalchbrenner, Erik Cambria, Narjes Nikzad, Meysam Chenaghlu, and Jianfeng Gao. Deep learning–based text classification: a comprehensive review. *ACM computing surveys (CSUR)*, 54(3):1–40, 2021.

[73] Andrés Montoyo, Patricio Martínez-Barco, and Alexandra Balahur. Subjectivity and sentiment analysis: An overview of the current state of the area and envisaged developments. *Decision Support Systems*, 53(4):675–679, 2012.

[74] Kuncahyo Setyo Nugroho, Anantha Yullian Sukmadewa, Haftittah Wuswilahaken DW, Fitra A Bachtiar, and Novanto Yudistira. Bert fine-tuning for sentiment analysis on indonesian mobile apps reviews. In *Proceedings of the 6th International Conference on Sustainable Information Engineering and Technology*, pages 258–264, 2021.

[75] Emanuel Oehri and Emitza Guzman. Same same but different: Finding similar user feedback across multiple platforms and languages. In *2020 IEEE 28th International Requirements Engineering Conference (RE)*, pages 44–54. IEEE, 2020.

[76] Daniel W Otter, Julian R Medina, and Jugal K Kalita. A survey of the usages of deep learning for natural language processing. *IEEE transactions on neural networks and learning systems*, 32(2):604–624, 2020.

[77] Dennis Pagano and Walid Maalej. User feedback in the appstore: An empirical study. In *2013 21st IEEE international requirements engineering conference (RE)*, pages 125–134. IEEE, 2013.

[78] Fabio Palomba, Pasquale Salza, Adelina Ciurumelea, Sebastiano Panichella, Harald Gall, Filomena Ferrucci, and Andrea De Lucia. Recommending and localizing change requests for mobile apps based on user reviews. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*, pages 106–117. IEEE, 2017.

[79] Sinno Jialin Pan and Qiang Yang. A survey on transfer learning. *IEEE Transactions on knowledge and data engineering*, 22(10):1345–1359, 2009.

[80] Bo Pang, Lillian Lee, and Shivakumar Vaithyanathan. Thumbs up? sentiment classification using machine learning techniques. *arXiv preprint cs/0205070*, 2002.

[81] Sebastiano Panichella, Andrea Di Sorbo, Emitza Guzman, Corrado A Visaggio, Gerardo Canfora, and Harald C Gall. How can i improve my app? classifying user reviews for software maintenance and evolution. In *2015 IEEE international conference on software maintenance and evolution (ICSME)*, pages 281–290. IEEE, 2015.

[82] Arti Patle and Deepak Singh Chouhan. Svm kernel functions for classification. In *2013 International conference on advances in technology and engineering (ICATE)*, pages 1–9. IEEE, 2013.

[83] Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. Deep contextualized word representations. In Marilyn Walker, Heng Ji, and Amanda Stent, editors, *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 2227–2237, New Orleans, Louisiana, June 2018. Association for Computational Linguistics.

[84] Marco Pota, Mirko Ventura, Rosario Catelli, and Massimo Esposito. An effective bert-based pipeline for twitter sentiment analysis: A case study in italian. *Sensors*, 21(1):133, 2020.

[85] VS Prasatha, Haneen Arafat Abu Alfeilate, AB Hassanate, Omar Lasassmehe, Ahmad S Tarawnehf, Mahmoud Bashir Alhasanatg, and Hamzeh S Eyal Salmane. Effects of distance measure choice on knn classifier performance-a review. *arXiv preprint arXiv:1708.04321*, page 56, 2017.

[86] Changqin Quan and Fuji Ren. Unsupervised product feature extraction for feature-oriented opinion determination. *Information Sciences*, 272:16–28, 2014.

[87] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. 2018.

[88] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140):1–67, 2020.

[89] Sakshi Ranjan and Subhankar Mishra. Comparative sentiment analysis of app reviews. In *2020 11th international conference on computing, communication and networking technologies (ICCCNT)*, pages 1–7. IEEE, 2020.

[90] Kumar Ravi and Vadlamani Ravi. A survey on opinion mining and sentiment analysis: tasks, approaches and applications. *Knowledge-based systems*, 89:14–46, 2015.

[91] Sebastian Ruder, Matthew E Peters, Swabha Swayamdipta, and Thomas Wolf. Transfer learning in natural language processing. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: Tutorials*, pages 15–18, 2019.

[92] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *nature*, 323(6088):533–536, 1986.

[93] Hassan Saif, Miriam Fernandez, Yulan He, and Harith Alani. On stopwords, filtering and data sparsity for sentiment analysis of twitter. 2014.

[94] Abhishek Sainani, Preethu Rose Anish, Vivek Joshi, and Smita Ghaisas. Extracting and classifying requirements from software engineering contracts. In *2020 IEEE 28th international requirements engineering conference (RE)*, pages 147–157. IEEE, 2020.

[95] M Rushdi Saleh, Maria Teresa Martín-Valdivia, Arturo Montejo-Ráez, and LA Ureña-López. Experiments with svm to classify opinions in different domains. *Expert Systems with Applications*, 38(12):14799–14804, 2011.

[96] Mario Sänger, Ulf Leser, Steffen Kemmerer, Peter Adolphs, and Roman Klinger. Scare—the sentiment corpus of app reviews with fine-grained annotations in german. In *Proceedings of the Tenth International Conference on Language Resources and Evaluation (LREC'16)*, pages 1114–1121, 2016.

[97] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

[98] Iqbal H Sarker. A machine learning based robust prediction model for real-life mobile phone data. *Internet of Things*, 5:180–193, 2019.

[99] Iqbal H Sarker. Deep cybersecurity: a comprehensive overview from neural network and deep learning perspective. *SN Computer Science*, 2(3):154, 2021.

[100] Iqbal H Sarker. Machine learning: Algorithms, real-world applications and research directions. *SN computer science*, 2(3):160, 2021.

[101] Iqbal H Sarker, ASM Kayes, Shahriar Badsha, Hamed Alqahtani, Paul Watters, and Alex Ng. Cybersecurity data science: an overview from machine learning perspective. *Journal of Big data*, 7:1–29, 2020.

[102] Rifat Ara Shams, Waqar Hussain, Gillian Oliver, Arif Nurwidyantoro, Harsha Perera, and Jon Whittle. Society-oriented applications development: Investigating users' values from bangladeshi agriculture mobile applications. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Software Engineering in Society*, pages 53–62, 2020.

[103] Beata Ślusarczyk. Industry 4.0–are we ready? *Polish Journal of Management Studies*, 17(1):232–248, 2018.

[104] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929–1958, 2014.

[105] Amira Samy Talaat. Sentiment analysis classification system using hybrid bert models. *Journal of Big Data*, 10(1):110, 2023.

[106] Kian Long Tan, Chin Poo Lee, Kalaiarasi Sonai Muthu Anbananthen, and Kian Ming Lim. Roberta-lstm: a hybrid model for sentiment analysis with transformer and recurrent neural network. *IEEE Access*, 10:21517–21525, 2022.

[107] Huifeng Tang, Songbo Tan, and Xueqi Cheng. A survey on sentiment detection of reviews. *Expert Systems with Applications*, 36(7):10760–10773, 2009.

[108] Jie Tao and Xing Fang. Toward multi-label sentiment analysis: a transfer learning based approach. *Journal of Big Data*, 7(1):1, 2020.

[109] Mohammadali Tavakoli, Liping Zhao, Atefeh Heydari, and Goran Nenadić. Extracting useful software development information from mobile application reviews: A survey of intelligent mining techniques and tools. *Expert Systems with Applications*, 113:186–199, 2018.

[110] Mike Thelwall, Kevan Buckley, and Georgios Paltoglou. Sentiment strength detection for the social web. *Journal of the American Society for Information Science and Technology*, 63(1):163–173, 2012.

[111] Mike Thelwall, Kevan Buckley, Georgios Paltoglou, Di Cai, and Arvid Kappas. Sentiment strength detection in short informal text. *Journal of the American society for information science and technology*, 61(12):2544–2558, 2010.

[112] Rajesh Vasa, Leonard Hoon, Kon Mouzakis, and Akihiro Noguchi. A preliminary analysis of mobile app user reviews. In *Proceedings of the 24th Australian computer-human interaction conference*, pages 241–244, 2012.

[113] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[114] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, 2022.

[115] Claes Wohlin. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In *Proceedings of the 18th international conference on evaluation and assessment in software engineering*, pages 1–10, 2014.

[116] Rui Xia, Chengqing Zong, and Shoushan Li. Ensemble of feature sets and classification algorithms for sentiment classification. *Information sciences*, 181(6):1138–1152, 2011.

[117] Ziang Xie, Sida I Wang, Jiwei Li, Daniel Lévy, Aiming Nie, Dan Jurafsky, and Andrew Y Ng. Data noising as smoothing in neural network language models. *arXiv preprint arXiv:1703.02573*, 2017.

[118] Yang Xin, Lingshuang Kong, Zhi Liu, Yuling Chen, Yanmiao Li, Hongliang Zhu, Mingcheng Gao, Haixia Hou, and Chunhua Wang. Machine learning and deep learning methods for cybersecurity. *Ieee access*, 6:35365–35381, 2018.

[119] Ziqiong Zhang, Qiang Ye, Zili Zhang, and Yijun Li. Sentiment classification of internet restaurant reviews written in cantonese. *Expert Systems with Applications*, 38(6):7674–7682, 2011.

[120] Liping Zhao, Waad Alhoshan, Alessio Ferrari, Keletso J Letsholo, Muideen A Ajagbe, Erol-Valeriu Chioasca, and Riza T Batista-Navarro. Natural language processing for requirements engineering: A systematic mapping study. *ACM Computing Surveys (CSUR)*, 54(3):1–41, 2021.

[121] Fuzhen Zhuang, Zhiyuan Qi, Keyu Duan, Dongbo Xi, Yongchun Zhu, Hengshu Zhu, Hui Xiong, and Qing He. A comprehensive survey on transfer learning. *Proceedings of the IEEE*, 109(1):43–76, 2020.